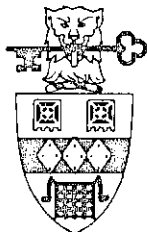


VECTOR

Interfaces: Special Issue

- * PC APL to Oracle on DEC Vax
- * Design of the APL*PLUS to DB2 link
- * Ken Iverson on APL Development
- * using Mac windows from APL.68000
- * VSAM with John Sullivan



*The Journal of the
British APL Association*

A Specialist Group of the British Computer Society

Vol.5 No.1 July 1988

Contributions

All contributions to VECTOR may be sent to the Editor at the address on the inside back cover. Letters and articles are welcome on any topic of interest to the APL community. These do not need to be limited to APL themes, nor must they be supportive of the language. Articles should be submitted in duplicate and accompanied by as much visual material as possible (ideally with a photograph of the author and a brief biographical note). Unless otherwise specified, each item will be considered for publication as a personal statement by the author.

Please supply as much material as possible in machine-readable form, ideally on an IBM PC compatible (DSDD) diskette. APL code should be kept separate from the text, and can be accepted as camera-ready copy, or as APL*PLUS/PC workspaces or IBM .AIO files.

Except where indicated, items in VECTOR may be freely reprinted with appropriate acknowledgement. Please inform the editor of your intention to re-use material from VECTOR.

Membership Rates 1988-89

Category	Fee	Vectors	Passes
Non-voting Student	5	1	1
UK Private	10	1	1
Overseas Private	18	1	1
(Supplement for Airmail)	8		
UK Corporate Membership	85	10	5
Overseas Corporate	140	10	
Sustaining	360	50	5

The membership year runs from 1st May to 30th April. Applications for membership should be made to the Treasurer on the form on the inside back page of VECTOR. Passes are required for entry to some association events, and for voting at the Annual General Meeting. Applications for student membership will be accepted on a recommendation from the course supervisor. Overseas membership rates cover VECTOR surface mail, and may be paid in sterling, or by Visa or Mastercard at the prevailing exchange rate.

Corporate membership is offered to organisations where APL is in professional use. Corporate members receive 10 copies of VECTOR, and are offered group attendance at association meetings. A contact person must be identified for all communications.

Sustaining membership is offered to companies trading in APL products; this is seen as a method of promoting the growth of APL interest and activity. As well as receiving public acknowledgement for their sponsorship, sustaining members receive bulk copies of VECTOR, and are offered news listings in each issue.

Advertising

Advertisements in VECTOR should be submitted in typeset camera-ready A5 portrait format with a 20mm blank border. Illustrations should be black-and-white photographs or line drawings. Rates are 250 per full page, 100 for half-page or less (subject to layout constraints).

Deadlines for bookings and copy are given under the Quick-reference Diary

Advertisements should be booked with, and sent to, Cathy Dargue, whose address is given with the Index to Advertisers.

CONTENTS

EDITORIAL:	Adrian Smith	Page 3
APL NEWS		
Topics for Future Meetings	Jonathan Martin	4
Quick Reference Diary	Adrian Smith	5
APL-ication: Latest Details	Philip Goacher	6
General Correspondence		11
British APL Association News		
Report on the 1987/88 year	Graham Parkhouse	19
Accounts and Minutes of the 1988 AGM		21
APL Bibliography	David Eastwood	26
News from Sustaining Members	Cathy Dargue	28
The Education VECTOR	Norman Thomson	33
Survey of Attitudes to APL	David Eastwood	34
 REVIEWS		
APL Product Guide	Cathy Dargue	39
Lauer Software's APL2C	Romilly Cocking	49
Algorithmics by Harel	Anthony Camacho	56
 RECENT MEETINGS:		
Through IRMA to IMS	David Alis	60
Real-time Processes in a Brewery	Robert Poate	64
Developing Business Systems	Romilly Cocking	66
APL in a Large Organisation	Maurice Jordan	68
Fast PC to MF File Transfer	David Toop	70
 GENERAL ARTICLES		
A Commentary on APL Development	Ken Iverson	78
Simple Icon Design for EGA and VGA	Adrian Smith	85
AP727: APL*PLUS to DB2	Allan Gay	91
 TECHNICAL SECTION: INTERFACES		
Hacker's Corner: a little bit of C	Adrian Smith	102
Technical Correspondence		103
VSAM Processing in APL	John Sullivan	104
APL*PLUS/PC to Oracle on Vaxes	Adrian Smith	119
APL through Mac Windows	Richard Nabavi	133
 British APL Association Software Library		
Software Submission Details	David Ziemann	140
Software Order Form	David Ziemann	142
Index to Advertisers		143

dyalog APL

C INTERFACE

```
#include "apl.h"
#include "support.h"

main()
{
    ARRAY *rslt, *lrg, *r;
    *getarray(), *frc
    *idn d(), *d_idn
    fncode;

    startup(2);
    define("IDN_TO_DATE", RSI
    define("DATE_TO_IDN", RSI

    lrg = rarg = rslt = 0;
    setjmp(errbuff);

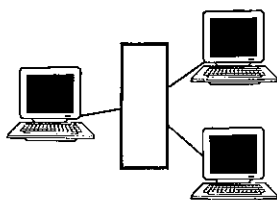
    while(1)
    {
        lrg = freearray()
        rarg = freearray()
```

UNIX INTERFACE

CHOICE OF HARDWARE

IBM Ps/2 Model 80
Compaq 386
Altos 3068
Apollo
AT&T 382
Bull SPS7, SPS9
Dec Vax
Hewlett-Packard HP9000
IBM 6150
Masscomp
NCR Tower 32
Pyramid
Sun
Unisys Series 5000

MULTI-USER



FILE SYSTEMS

SQL INTERFACE

EDITORS

```
[0] Z←DISPLAY Z;
[1] □IO←0 □ SZ←x
[2] Z←' 'z(□FMT
[3] NSS→(1 0≠DZ
[4] NOR:Z←(1pZ)p
[5] →(1=DZ)/SMP
[6] Z←' ',(QTv~1
[7] SMP:QB+(2i0
[8] FMT:Z←□FMT Z
[9] BOX:R C←~1+pZ
[10] C←~1+12 07
```

FULL-SCREEN INPUT

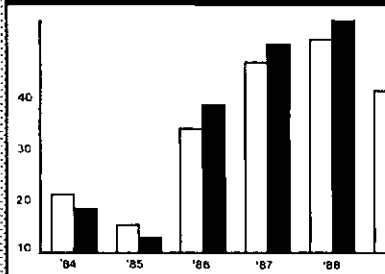
Share Name

Number of Shares

Unit Cost

Buyer

GRAPHICS INTERFACE



EMPNO	ENAME	JOB	MGR
7329	SMITH	CLERK	7902
7499	ALLEN	SALESMAN	7698
7521	WARD	SALESMAN	7698
7566	JONES	MANAGER	7839
7654	MARTIN	SALESMAN	7698
7698	BLAKE	MANAGER	7839
7782	CLARK	MANAGER	7839
7788	SCOTT	ANALYST	7566
7839	KING	PRESIDENT	
7844	TURNER	SALESMAN	7698
7876	ADAMS	CLERK	7788

DEVICE SUPPORT

NESTED ARRAYS

DISPLAY REPORT

	North	South	East
Cakes	14	76	46
Buns	22	5	68

...gives you all this – and APL too!

For further information, contact
Dyadic Systems Limited
 Park House, The High Street, Alton, Hampshire, GU34 1EN
 Telephone: (0420) 87024. Telex: 858811

Editorial: Don't Lose Touch

by Adrian Smith

This VECTOR is all about interfaces. I hate interfaces! At least 90% of the hassle generated during any APL project is at the edges, where the array structure of APL fails horribly to dovetail with the files and records used by everyone else.

The projects I like best are the little hole-and-corner ones, where my file system is called)SAVE and I don't have to bother my head with the niceties of packed-decimal, or the inner secrets of the Lotus worksheet. So ... why a VECTOR about all the things I detest most in this life?

- it is an unfortunate fact that systems generally have users, and users soon get tired of copying figures off their CICS screens so they can type them in to your APL model. They start saying unreasonable things like "... but it's all on the same computer ...", as if this made life any easier for you.
- the PC DOS standard has made a huge psychological difference. A PC user assumes that any package he buys will (with a bit of frigging around) read data from any other package. He is much less willing to accept excuses from the mainframe brigade about incompatible formats, or variable length records, or whatever.
- things are gradually getting better. As mainstream data starts to find its way into databases like DB2 and Oracle, the APLer's life suddenly becomes bearable. At long last, we get data back in tables, organised into rows and columns!

In summary: we can't ignore this problem; it won't go away; and our best excuses don't wash any more. We have to face up to it, and we need all the tools and techniques we can get. In the mainframe world we use things like AP123 to read VSAM data, and where necessary we get new interfaces made to order, for example from APL*PLUS to DB2. At the VAX and PC level, I am more and more convinced that we need to accept C as our programming partner. Unix is written in C, and so is almost any PC package you care to name. If you want to talk to them, you are going to have to talk in C. (You could try shouting in APL, but I don't think it would work.)

This all means broadening our horizons quite dramatically; I hope the papers in our technical section give you the start that you need.

Suggested Topics for BAA Meetings

by Jonathan Martin

The following is my first attempt at collating the suggestions I received at the AGM on 29th April.

- Tutorial session. Two speakers before tea on fairly technical (specific) topics, then an open session (maybe with an informal panel) on anything anyone wants to ask.
- APL and the Outside World. Topics such as parallel processing, relational databases, expert systems.
- APL in Education. Teaching APL to newcomers, further training in advanced techniques, APL in further education. This will probably be an evening meeting, to give teachers more chance to attend.
- APL - the next 5 years. Representatives from the major suppliers to tell us what they are working on, and what they expect to bring to the market soon.
- Applications in APL. Four speakers, relating their experiences in implementing a variety of APL systems.

As regards the actual dates, October 21st will be the Tutorial Session, since this follows so closely on the heels of Canterbury. The mapping of the remaining topics to the remaining dates will be published in VECTOR 5.2 (November 88).

Quick Reference Diary

Date	Venue	Event
16-20 August	Syracuse Univ. New York	ACM SigAPL workshop on APL in Expert Systems.
2 September	London (ROSL)	Evening Meeting (6pm - 8pm) on Education and I-APL. Speakers to include Alan Sykes.
28-30 September	Canterbury	APL-ication the Practical State of the Art APL Conference
21 October	London	BAA Meeting: APL Tutorial
18 November	London	BAA Meeting
13 January 1989	London	BAA Meeting
17 February	London	BAA Meeting
17 March	London	BAA Meeting
21 April	London	BAA Meeting

All British APL Association meetings are to be held at the Royal Over-Seas League, Park Place, near Green Park Tube station. Afternoon meetings begin at 2.00pm and close at 5.30pm.

Dates for future issues of VECTOR (note 5.2 dates)

	Vol.5 No.2	Vol.5 No.3	Vol.5 No.4
Copy date	1st Oct 88	1st Dec 88	1st March 89
Ad booking	10th Oct 88	12th Dec 88	10th March 89
Ad Copy	24th Oct 88	23rd Dec 88	25th March 89
Distribution	November 88	January 89	April 89

APL-ication

The Practical State-of-the-Art APL Conference

University of Kent

Canterbury, England

28-30 September 1988

APL-ication has been planned as an exciting, outward looking conference of interest to all APL practitioners - both newcomers to APL and those with more experience in the use of the language - and it will also provide an insight to the effective use of APL for those wishing to evaluate the use of the language in particular application areas.

APL-ication is designed to give a careful balance between the presentations given by invited speakers and contributed papers, with the accent on the practical application of APL in a variety of business disciplines. Indeed, we believe that the conference will be of interest to those who wish to evaluate the use of APL in specific business areas.

The conference will have an international flavour, with contributions from overseas, particularly our APL colleagues in Europe.

The aim is to make the event highly participative. 'Live' presentations will accompany papers, supported by the provision of a wide range of computing and audio-visual equipment.

Although there will be no formal exhibition, you will be able to meet consultancies and vendors at their hospitality suites and at their presentations of software and hardware in the product forums.

An APL bookstall and a software exchange will be open throughout the conference.

The British APL Association invites applications for places at the conference from its members, to whom it offers an advantageous rate. A most important feature of conferences such as **APL-ication** is the ready made forum for meeting fellow members with whom to exchange ideas and discuss techniques.

We look forward to meeting you there!

University of Kent

The conference will be held at the University of Kent which overlooks the Cathedral City of Canterbury. The University campus, opened in 1965, offers outstanding facilities for staging conferences, with on-site accommodation and social amenities. The University can be reached conveniently by road and rail. Canterbury is close to the motorway network affording rapid access to all parts of the UK and Europe via nearby Dover. The rail link to London also provides access from all parts of the country and from Heathrow and Gatwick airports.

Programme

We were surprised at the number of interesting papers submitted for this conference, and it has not been an easy task to assemble a three-day programme to amply reflect the diversity of topics offered. The schedule of papers with abstracts is available in our conference brochure - if you have not yet received your copy these may be obtained from the BISL Conference Department at the BCS (for address see Registration Information & Terms).

Canterbury

The City's roots can be traced back to a prehistoric iron age settlement, though the story of Canterbury really starts after the Roman invasion of AD 43. After successive occupation by the Vikings and the Danes, the Normans arrived and it was they who began building the Cathedral in 1070. Despite the later attentions of Henry VIII and Cromwell the Cathedral survives to this day as the 'Mother Church of English Christianity', and is famed for its collection of stained glass windows.

Leeds Castle

To ensure that you see something of the local heritage, we are taking delegates to Leeds Castle for the conference banquet.

The castle, not far from Canterbury, was originally a Norman stronghold, later converted into a Royal Palace by Henry VIII. The castle is built on two islands in the middle of a lake within the wooded rolling Kent landscape.

A Royal residence for over 300 years, the Castle was a favourite home for eight of England's medieval Queens. The grounds of the Castle extend for some 500 acres with lakes, streams, gardens and woodland and include an ancient vineyard from which the Castle's own wine is produced. There is a museum at the Castle and the grounds also boast some of the world's most unusual and exotic waterfowl and birds.

Registration Information and Terms

Registration: Delegates are requested to register as early as possible using the Registration Form. A separate form is required for each delegate - please photocopy the form as required. Completed forms with payment in full should be sent as soon as possible to:

BISL Conference Department (APL)

The British Computer Society

13 Mansfield Street

London W1M 0BP, England

Telephone 01-637 0471

Fees: Reduced fees are available to delegates holding membership of the British APL Association or The British Computer Society. The rates in pounds sterling (exclusive of VAT) are:

	Before 1 Sept	From 1 Sept
Member	230.00	255.00
Non-Member	255.00	290.00

Delegate fees include accommodation at the University of Kent on Wednesday 28th and Thursday 29th September, and all meals from lunch on Wednesday 28th to lunch on Friday 30th September. In addition, the delegate fee includes attendance at conference sessions, documentation and attendance at the conference banquet. All fees are subject to VAT at 15% which must be added to your remittance.

Payment: All fees are due with the application and must be in £ sterling. Cheques should be drawn on a British clearing bank and be made payable to the 'British Informatics Society Limited'. Confirmation of booking will be sent to each delegate along with relevant maps and instructions. Early payment discounts will apply according to the date of the postmark on the envelope in which the application has been sent.

Payments still outstanding after 30 September will be subject to a 5% surcharge.

Substitution: A registered person may nominate a substitute providing that the conference office is notified no later than 23 September 1988. The amount of fees then due is according to the eligibility of the substitute, who will be invoiced for any difference owing.

Cancellations: Cancellations will be accepted only if received in writing not later than 1 September 1988. A 10% handling charge will be deducted from any refund.

Programme: The organisers reserve the right to make changes to the advertised programme.

APL-ication Registration Form

Delegate Name

Title (please tick) Mr ☐ Mrs ☐ Miss ☐ Ms ☐ Dr ☐ Prof ☐

Organisation:

Address:

.....

Telephone: Membership: BAA ☐ BCS ☐

Special requirements: Please state here any special dietary/other needs

.....

Residential conference fees for APL-ication 28th-30th September 1988 are shown below. Delegates arriving the night before the conference may book bed and breakfast at a charge of £15.

	Before 1 Sept	After 1 Sept
Member Fee	230.00	255.00
Non-Member Fee	255.00	290.00
Accommodation for 27 Sept		15.00
Fee Payable		
VAT : Add 15% of Fee Payable		
Total remittance		

Signed Date1988

Please note that completing and signing this form implies acceptance of the applicable terms and conditions.

Please send details ☐ I will travel by Rail ☐ Road ☐ Air ☐
of hotel accommodation

APL *Innovators*

SALADIN COMPUTER SYSTEMS is a small and rapidly growing software house specialising in decision support systems for the oil industry. Based in London, our business is international, and our expanding client portfolio includes 'household name' companies throughout the world.

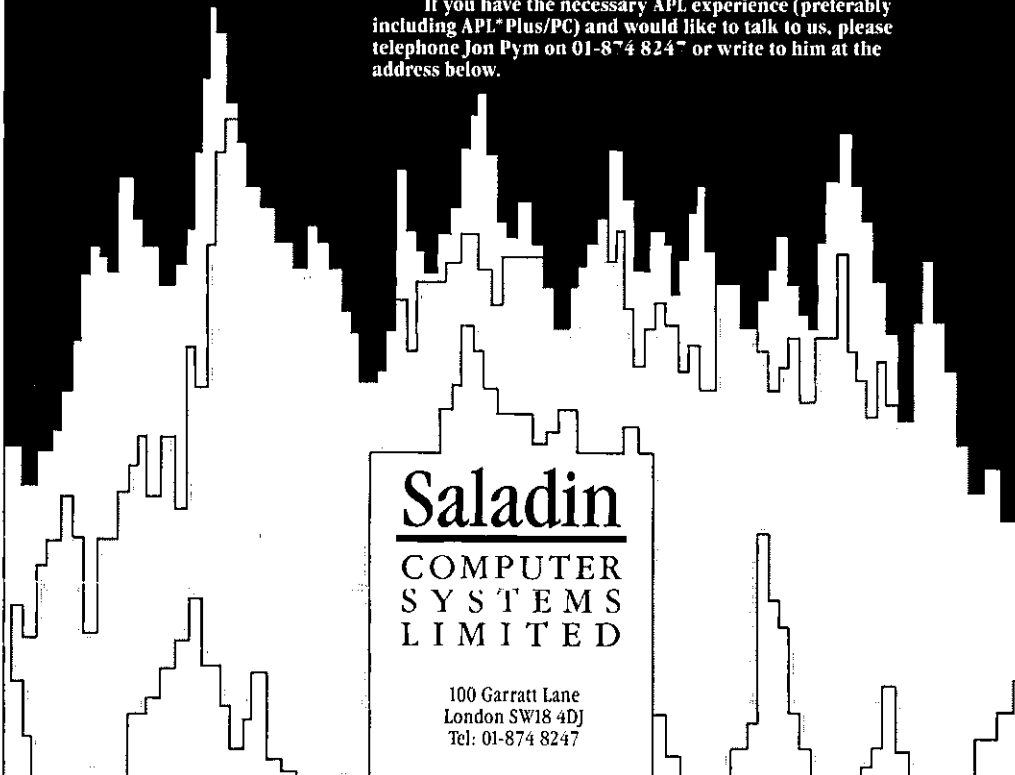
We use APL to provide both custom and package software systems of the highest standard, with a state-of-the-art user interface which rivals any available software.

We are currently seeking to expand our development, consultancy and support team.

You should be more than just a fluent APLer. You will need creativity, initiative and determination, coupled with a flexible approach, the ability to take responsibility and well developed personal skills.

In return, we can offer a challenging and highly motivated environment, exceptional rewards and international career opportunities.

If you have the necessary APL experience (preferably including APL* Plus/PC) and would like to talk to us, please telephone Jon Pym on 01-874 8247 or write to him at the address below.



Saladin
COMPUTER
SYSTEMS
LIMITED

100 Garratt Lane
London SW18 4DJ
Tel: 01-874 8247

General Correspondence

The VECTOR working group welcomes correspondence on any topic affecting the APL community. All letters should be addressed to the Editor, and should indicate whether they are for the general or technical section. (Letters containing APL code will normally be classed as 'technical'.) The editor reserves the right to edit any letter unless the writer states that it is to be published in full or not at all.

Re: letters published in VECTOR 4.2

From: Mark Cranny

18th May 1988

After they had sat on my desk for several weeks, I finally got round to reading editions 4.2 and 4.3 of VECTOR, and was so amazed at some of the correspondence in 4.2 that I felt the need to stop revising and put pen to paper - or finger to keyboard even.

The first letter was from Dick Bowman claiming that APL goes further than any 4GL in that it is far more flexible. He mentions ADRS, yet doesn't seem to make the connection between APL and 4GLs. In several issues of VECTOR, there have been full-page adverts for IC/1, which IBM has consistently refused to market as a 4GL, preferring to call it an "integrated multifunction center environment", yet by the traditional criteria for a 4GL it certainly is one:

- non-procedural (menu driven)
- database query language
- input validation routines / tables
- report generator
- financial planning/analysis facility
- graphics facility
- complete application generation facilities
- interfaces to other systems (OS, DBMS, TP etc.)

For the uninitiated, IC/1 is ADRS II the reporting system, APLDI the APL data interface, ADRS/BG business graphics, FPS II financial planning and APE (application prototyping environment) all thrown together and called IC/1. It emerged from Sweden, was taken over by the Americans when 4GLs first started to become fashionable, and when it didn't take off was handed back to the Swedes to continue to support and develop (probably after causing quite a lot of damage). IC/1 allows rapid or evolutionary prototyping to take place, with its own inbuilt indexed sequential filing system, or it can act as a front end to other DBMSs such as SQL/DS. The ADRS sections of code provide the reporting and data-validation capabilities, and APE allows screen design for interface creation. The graphics facility is probably the weakest part of the package, and is

certainly nowhere near as well developed as the graphics of IBM's flagship 4GL, Application System (AS). IC/1 is far from being the best 4GL available, but the ideas are there, and (as estate agents are fond of saying) it has potential.

I completely agree with Mr Bowman that current 4GLs only go part of the way to presenting effective application generation, but IC/1-type environments give the best of both worlds: a non-procedural language for fast prototyping, but also the ability to get out of the menus and to get to the actual source code that the package is written in, allowing complete customisation of the application to fit the design requirements. Maybe a future development could be that when an application is completed, all internal documentation and unused functions could be erased to save space, and then the code compiled to create the finished production workspace, only being recompiled after updates and system changes to the original development workspace. Perhaps the "sustaining members" of the BAA are already developing advanced APL tools along these lines?

I first came into contact with APL at university, but it was not until I worked for IBM for an industrial experience year that I really started to use APL and its associated products, such as IC/1. At first I couldn't understand why IC/1 was not successfully marketed by (probably) the best marketing organisation in the world. However, I think there can be only one conclusion - IBM doesn't want to market products like IC/1 when it can market the more glamorous AS. As with DB2, IBM tries to get customers interested in AS and then informs them that it needs their biggest and most powerful boxes in order to run it.

There was also a letter from Richard Connor, who claimed that:

"One of APL's great virtues is that its effective use doesn't require technical knowledge of computers and software. It is an excellent language for the end-user."

Of all that I have read about APL, this has to be the biggest load of horse s**t I have come across. Yes, APL is a powerful tool when hidden behind interfaces such as ADI, but Mr Connor seems to be suggesting that end-users get down to the nitty-gritty in raw APL code. I'd love to know what he does for a living, and what experiences he has had of end-user computing! I spent 13 months actually sitting next to the people who were using the systems I had designed, and several that I inherited and had to maintain. It was difficult enough to get users to use IBM's PROFS office system (based around PF keys), let alone get them to accept a programming language per se.

One particular user had a habit of leaning backwards and hitting me over the back of the head whenever something went wrong or there was something he didn't understand. If he had been given raw APL to use as an analysis tool I should have had to wear a crash-helmet! There are many computer-illiterate people out there with a very real fear of the keyboard, and if the BAA has been spending its time over the years trying to encourage

those people to use APL in its raw form, and not as completed applications, then it is not surprising that APL is still pretty much a dead duck. APLers seem to expect everyone else in the computer world to share their enthusiasm for the aesthetic qualities of the language: in the real world people just want to see results, or to have effective and straightforward end-user tools made available to them.

Barrie Webster's letter contained an accurate insight into the current position and problems of APL - the language has just not been sold, and has always had the image of being a mathematician's toy. If 4GLs written in APL were actively marketed, then users could be introduced to the power of APL without realising it, but the facility to go further and to get more involved with the source code would be there if required.

I am constantly surprised when I discover yet another company who use APL - Midland Bank, British Airways, IBM etc - and yet the language still accounts for only a minute proportion of all systems development. It is this business community which needs educating as to the benefits of APL, rather than schoolchildren. Companies have too much investment in COBOL to rewrite their complete applications portfolio in another language, especially one that is also a 60s love-child. However, a move towards prototyping in APL can be encouraged: after all, C started to be more widely used as a result of Unix, so why not use the 4GL environment as a vehicle to get APL more widely accepted?

Commercial organisations are interested in the business advantages to be gained (increased speed of development, flexibility of design, reduced maintenance costs etc.), and at what cost, not how pretty or elegant the code is! Maybe the powers that be in the BAA could "encourage" Computing or Computer Weekly to give a couple of pages over to articles looking into the pros and cons of APL. If APL doesn't move with the 4GL environment, it will remain with an ever-decreasing band of enthusiasts.

After leaving IBM and returning to college to finish my degree in systems analysis, I joined the BAA in order to keep up with developments in APL. I think that I am addicted to the language, in that I enjoy using it more than any other 3GL I have come across, but I am not a disciple or APL bigot. The BAA have been the Jehovah's Witnesses of the computer world for too long; when a member of the APL True and Faithful puts their foot in the door of a disbeliever, and starts talking about dyadic iotas and Atomic Vectors, who can blame them for switching off? As a business computing person, I will use any tool necessary to achieve efficient and cost-effective results, and I am not brainwashed to the extent that I see APL as the ultimate systems cure-all. However, I do think that its power has been completely overlooked and underestimated by the business community. The power of APL is there provided it is implemented with good in-house standards for readability and maintainability, prototyping and documenting tools.

The BAA has the image of a group of mathematicians masquerading as computer professionals; developing ever more intricate and useless utilities for other computer scientists. I would suggest that the business-minded members of the BAA form a breakaway party to concentrate on promoting the business advantages of APL, and leave the academics to argue over whatever they want to argue over. However, in light of what has happened to the SDP, maybe that is not such a good idea?!

I would love to see APL become a popular and widely-used language, but to be honest, I don't think it ever will.

Yours in deepest perspiration (exams only 2 weeks away)

Mark Cranny

Dept for Business Systems Analysis
City University, London

P.S. E J Sargeants's letter suggesting an APL-based calculator was a joke, wasn't it?

Public Letter to P Chapman

From: F.H.D. von Batenburg

3rd May 1988

This month I received my copy of I-APL, and was impressed by what is distributed for just £11. I got a comprehensive language manual by Garry Helzer, describing all the functions clearly with just the right amount of examples to illustrate them. There was also an attractive tutorial by Linda Alvord and Norman Thomson, and a (fortunately) small technical manual.

A small disappointment was the print quality of the language manual, which was in dot-matrix format (granted in a not-too-bad NLQ font, but ugly nevertheless); a pity that this could not be printed on the same laser as the other booklets were.

I consider it a small miracle that Paul Chapman (and his supporters) developed such an interpreter in less than a year, and I hope this product will achieve what it set out to do: distributing the gospel of APL. As for myself, I certainly intended to help distributing it to my students, and by starting to write some regular series in Atari and PC magazines.

So I put the floppy in the drive, typed in "IAPL /C" (a character generator for all graphics cards was provided too!), and in response to the first 5 spaces I typed "[10T]10" and a VALUE ERROR was my reward. Then I discovered that the character set is distributed over the various keys in a new way. Although the JOT is still Shift-J (after pressing [INSERT] however), most of the other characters were not in their familiar places. The

next 5 minutes showed an heroic struggle to find all the familiar symbols in order to get some simple expressions done.

Problem

This brings forward the point I wish to make. I wholeheartedly support the I-APL project and I hope that it will attract new adepts for APL, but if the character set is not brought into line with the standard keyboard quickly, I think it will seriously hamper the project.

In the discussions about I-APL it was often stated that compatibility with standard APL was of utmost priority. For that reason I read that fractional replicate was removed because it would frustrate any user who might upgrade to a commercial APL. That decision, although painful, was a sensible thing to do.

Upward Compatibility

Now the position of the characters over the keyboard is a similar case. If people are accustomed to the position of the characters in I-APL they will experience an undesirable frustration when they change to commercial APL.

For example I had the intention to start all my APL courses with I-APL. This is a very strong incentive for beginners; it is my experience with other packages that a package which my students are allowed to copy and use at home for study (and to give away to friends) immensely increases the popularity of that program. But after a few weeks I teach file handling and graphics; at that moment I need to switch to a commercial APL.

However, with the character set incompatibility, I don't dare to burden relative novices with switching to another character set. So unfortunately I had to drop this idea.

Downward Compatibility

You may wonder who will 'downgrade' from commercial APL to I-APL? The answer is: "the most important people you have", experienced APL users and APL professionals who try to promote APL. These people are familiar with the current position of the symbols, and they will not have a one-time mental conversion, but will be frustrated again and again whenever they switch to I-APL for a moment. As experienced APL users devote their time mainly to the professional implementations, they know the standard keyboard layout by heart. As they switch to I-APL only occasionally, they will never be comfortable with the alternative layout of I-APL.

A single source for APL*PLUS

APL*PLUS
PC



for PCs
and the PS2

APL*PLUS
UNIX



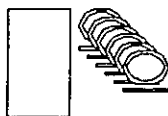
for UNIX
systems

APL*PLUS
PRO



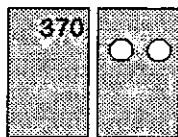
for the PS2
model 80

APL*PLUS
VMS

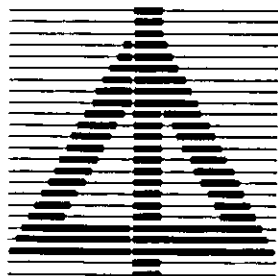


for the
VAX range

APL*PLUS
VM & MVS



for IBM
mainframes

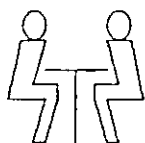


COCKING & DRURY LTD.

THE APL PROFESSIONALS

180 Tottenham Court Road,
London W1P 9LE
Tel: 01 - 436 9481

A single source for APL



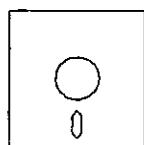
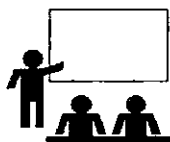
Advice

Design



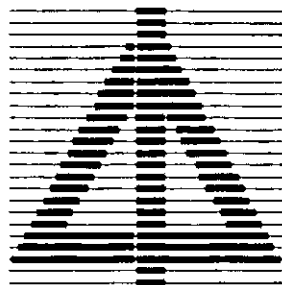
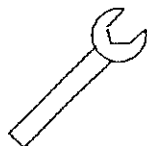
Application
Development

Education



APL software

Support



COCKING & DRURY LTD.

THE APL PROFESSIONALS

180 Tottenham Court Road,
London W1P 9LE

Tel: 01 - 436 9481

They will always need to fetch the paper with the layout of I-APL; this is very frustrating and it will thwart the quick 'playing' in the few minutes left between two big projects. Even when they have the I-APL layout at hand, they have to spend too much time locating the proper symbols.

Even if such a professional doesn't give up finally, at least his interested visitor will not get a favourable impression of APL if he sees his tutor struggling with 'a minor detail' such as the search for a proper symbol. That APL has a lot to offer will probably be lost by this bad impression.

By the way, it will also require that all existing manuals and books have to devote extra sections to the character set location. This is an unjust burden for writers too, and it makes me hesitate to do that for a 'non-standard' APL feature.

Remedy

For these reasons I strongly advise a return to standard APL for the position of the symbols. The best thing to do would be to distribute a patch with which to return to the familiar keyboard layout. However, if you don't want to drop this new layout, at least give the user an option to choose between the new and the standard keyboard.

I feel very positive about this project and I think that Paul Chapman and all the other contributors did a marvellous job. So it's a shame if such a relatively small thing were to hamper the success of this project. With all respect for the big achievement which I-APL is, please don't let this small thing of a non-standard character placement endanger the final aim of popularizing APL.

Yours sincerely

F.H.D. van Batenburg

Instituut v.Theoretische Biologie
Groenhovenstraat 5
2321BT Leiden
Holland

British APL Association News: Report on the Achievements of the 1987/88 Committee

by Graham Parkhouse

The committee met seven times altogether. Never did we manage all to be present at once. Average attendance at each of the 3 hour sessions was 8 of the 10 strong committee. Everybody attended most of the meetings.

What struck me about the meetings, being a newcomer, was how serious a business it all seemed to be. All members had to circulate written reports before each meeting, and there was an on-going actions list which was reissued after every meeting. I have to report that I think the 168 hours plus travelling time, report writing and minute writing might have been more constructively spent actually doing some of the things that we talked about all year.

What did we achieve? Jill Moss drew up a new BAA brochure which was printed in the autumn, which has been sent out by I-APL Ltd to enquirers, and is freely available. The teachers' course which Norman Thomson organised in Winchester is reported in VECTOR 4.2, and two similar courses are being organised for Winchester and Swansea this summer. A postal ballot was voted in at the Extraordinary General Meeting held on 20th November and the new procedures were put into effect this April. Two delegates have been sponsored to attend the ICME Conference on Mathematical Education at Budapest in July. They are both teachers who said they were prepared to speak out for APL at the conference.

A special committee meeting was convened to discuss further support to the I-APL project. A further £2,000 was given to I-APL Ltd to make BAA's total contribution £8,000. This meeting is reported in more detail in VECTOR 4.4.

The committee made considerable progress towards getting paid administrative help. If current plans meet with approval from the next committee we shall be employing someone to work up to 10 hours per week doing the many jobs that are handled by committee members which would be better done and done better by one person with time available. This might begin on an experimental basis for a year. Watch this space for developments! You may have noticed a sluggishness in the system, such as wrong dates in VECTOR for forthcoming meetings, late notification of meetings, and one very late VECTOR. (Did you realise you got 4.3 before 4.2?) I am partly to blame, but we are sure that the right kind of help will get the system running better.

Preparations for APL-ication (28-30th September) have been handled entirely by the sub-committee set up for the purpose. We hear there are going to be some good speakers.

So what didn't we achieve? A BAA bulletin board was considered but was dismissed as inappropriate to our needs at the moment. A leaflet on APL applications was beginning to be drafted. A conference for senior management was talked about, as was a regional meeting, and these may still be coming. There was discussion of producing an APL bibliography, even of sponsoring a researcher to produce one. This has not been pursued and maybe it has already been done for us; David Eastwood did a nice little bit of research when he called in at the Institution of Electrical Engineers to investigate their on-line information services. He is going to report his findings in VECTOR.

I have tried to give you a realistic picture of what your committee has been up to over the year. If you found yourself reacting to anything I have written, or are left shocked by what I have failed to mention, please ring me or write to VECTOR and let us have your suggestions.



SOLUTIONS FOR THE CITY

We are a specialist firm of consultants with a prestigious list of clients including major City institutions.

For one of these clients we have recently developed a foreign exchange dealing system with the following features:— deal entry, deal enquiry, multi-currency position keeping, basket currency calculators, arbitrage calculators and ticket production.

If you are interested in foreign exchange dealing, treasury operations, portfolio management, commodity broking or similar areas we would be delighted to discuss how we could help. Call Stan Wilkinson, HMW COMPUTING LIMITED Hamilton House 1 Temple Avenue London EC4Y 0HA Telephone 01-353 4212

British APL Association - 1988 Accounts

Income and Expenditure for year to:		30/4/88	30/4/87
	£	£	£
Income			
Subscriptions:			
Full (including students)	5,523	4,304	
Corporate	3,488	3,266	
Sustaining	3,600	3,600	
Library	300	465	11,635
	-----	-----	
Special Events (APL86)		2,856	50,440
Sales:			
Vector	302	203	
Books	100	738	941
	-----	-----	
Vector (less production costs)		885	1,227
Bank Interest		4,690	1,598
		=====	=====
Total Income:		21,744	65,841
		=====	=====
Expenditure			
Meetings	1,475	1,356	
Committee	1,317	1,094	
Mailing and Publicity	1,167	655	
Education	747	106	
Recruitment	425	589	
Projects (APL-ication current A/C)	2,000	157	
BAA Software Library	47	3	
Donations: I-APL	2,010	6,000	
Bursaries	500	250	6,250
	-----	-----	
Professional Fees	75	40	
Bank Charges (Credit card fees)	156	nil	
		=====	=====
Total Expenditure:		9,919	10,250
		=====	=====
Surplus of Income over Expenditure:		11,825	55,591
		=====	=====
Balance Sheet at Year End			
Cash at bank		70,043	58,051
Debtors: APL-ication	2,000		
Other	3,607	5,607	6,830
	-----	-----	-----
		75,650	64,881
Less Creditors		(108)	(1,164)
		-----	-----
Net Assets:		75,542	63,717
		=====	=====
Accumulated surplus previous year		63,717	8,126
Surplus current year		11,825	55,591
		-----	-----
Net Reserves:		75,542	63,717
		=====	=====

RENAISSANCE DATA SYSTEMS
BOOK DIVISION

*** Enlightenment Thru Information Processing ***
1988 Catalog Excerpts (as of July 1, 1988)
+++ All APL Books in Print +++

	Quantity	Total
	Price	Ordered Cost
APPLIED APL PROGRAMMING, LaPage - Excellent APL style. Particularly good for academic use in the sciences. 1978, 308 p.	\$30.00.	.
APL - AN INTRODUCTION, Peelle - Especially for the computer-shy. 1986, 459 p.	\$29.95.	.
A PROGRAMMING LANGUAGE, Iverson - The book that started it all. A computer science classic. APL's roots. Reprinted by RDS on acid free paper in a high quality, soft cover edition. Lowest price available. 1962, 286 p.	\$49.95.	.
DIGITAL SYSTEM IMPLEMENTATION, Blaauw. Computer architecture - from design to implementation. Adders, decoders, memory, processing, control, storage, communication. All described using APL. 1976, 384 p.	\$45.00.	.
APPLIED MATHEMATICS FOR PROGRAMMERS, Iverson. Classification and sets, elementary functions, directed graphs and trees, modeling, inverse and linear functions. 40 pages of exercises. 1986, 158 p.	\$12.00.	.
ABSTRACT ALGEBRA - A COMPUTATIONAL APPROACH, Simms. A one year introductory course in which algorithmic questions and computations are stressed. A methodology for investigating computational questions. 1984, 491 p.	\$49.95.	.
COMPUTING IN STATISTICAL SCIENCE THRU APL, Anscombe. - Lots of statistics and APL. His workspaces are available from Yale University. 1981, 426 p.	\$39.00.	.
MATHEMATICAL EXPERIMENTS ON THE COMPUTER, Grenander. Case studies using APL in a number of fields, including statistics, linear algebra, geometry, asymptotics, neural networks, invariant curves. Detailed description and analysis of APL functions in all topics discussed. 1982, 323 p.	\$59.50.	.
PROBABILITY IN APL, Alvord. A delightful, yet serious development of 31 functions for fun and learning with roll, deal, binomial distributions, combinations, permutations, geometric distributions, and the World Series. Friendly examples. No previous APL required. 1984, 142 p.	\$17.50.	.
Diskette for above with functions covered in book.	\$22.00.	.
APL - ADVANCED TECHNIQUES AND UTILITIES, Bergquist. Good discussion of alternative approaches to a wide range of programming tasks - efficiency, searching, dates, workspace documentation, file design, boolean techniques, financial utilities. Assumes knowledge of the basics. Many idioms. 1986, 450 p.	\$44.95.	.
Diskette for above. Contains all functions described in text. Format is for STSC APL+PLUS/PC but is uploadable to Sharp APL, VS/APL, APL2.	\$15.00.	.
systems, overall design considerations, relative efficiencies. 125 p.		.
APL AS A TOOL OF THOUGHT, PROFESSIONAL DEVELOPMENT SEMINARS, NY/SIGAPL. They cover a wide range of topics in business and education.		.
1987 - Logic, insurance, statistics, A.I., accounting, fractals, linked lists.		.
1983 thru 1987 as one package	\$60.00.	.

Sub-total: \$

Postage and handling - U.S. and Canada (\$2.50 each item over \$25):
Postage and handling - International (please add 10 %surface mail, 50% airmail):
NYC Residents please add 8 1/4% sales tax, NY State, 4%:

Total: \$

Please send complete catalog: _____

Date: _____

Please make check or money order payable
in \$US and send with this form to:

Name: _____

Renaissance Data Systems
660 Amsterdam Avenue - Suite 124
New York, New York 10023

Street: _____

(212) 864-3078

City, etc.: _____ Postal
Code: _____

Country: _____

**Minutes of the AGM of the British APL Association
29th April 1988 at the Royal Over-Seas League, London SW1.**

by Graham Parkhouse

Agenda

1. Approval of the minutes of the last AGM (5th June 1987) published in VECTOR Vol 4 No.1 p.9.
2. Approval of the minutes of the EGM (20th Nov 1987) published in VECTOR Vol 4 No.3 p.10
3. Chairman's report
4. Treasurer's report
5. Election results
6. Special Achievement Award

Approval of Previous Minutes

The minutes of the previous meetings were approved by the forty people present.

Chairman's Report

This has been another very successful year for the BAA; as you will see from the Treasurer's report membership is significantly up and our financial status is healthier than ever.

Looking at our major activities, we have run the regular series of technical meetings (barring one where adverse weather caused cancellation); our meetings organisers have an exciting forward programme including plans for regular events outside the London area and an upmarket 'APL for Management' event.

By contrast, the Vector group has not had such a strong year, although we would hope that all the production difficulties are now behind them. The committee has given a lot of thought to the content of Vector, particularly to the amount of material of lasting value to non-London members - for whom this is the chief benefit which the BAA has so far provided.

As we continue to grow administration becomes an increasing burden and the committee has decided that it is now appropriate for us to take on someone as (initially part-time)

paid help handling many of the tasks we have hitherto relied on volunteer effort for. This we expect to see improving the quality of membership service and freeing the committee for more strategic planning.

Among our other activities has been the first of what could be a regular series introducing APL to teachers, and launching the BAA Software Library.

Something of a disappointment has been the response to our Members Projects Scheme, which we launched last year as one of the ways we wanted to invest the APL86 income in the future of APL. A few projects were proposed, but the only one we wanted to back seemed to go on permanent hold. We have a Projects Officer, whose main mission in life is to give financial backing to worthy projects; we shouldn't let this opportunity go to waste.

More positive progress has been made in the other scheme launched from APL86 - the APL-ication conference to be held at Canterbury in late September is looking like the major APL event of the Northern Hemisphere this year. The Organising Committee (myself, Christine McCree, Mel Chapman, Philip Goacher and Dave Parker) have been at work all year, we are more than happy with the papers received; publicity mailing will begin shortly and we look forward to seeing you all in Canterbury.

In closing I would like to record my appreciation of the work of the whole BAA Committee this year - as I said earlier, we have had a very successful year to a degree which I hope our successors can emulate.

Treasurer's Report

Mel Chapman reported membership figures as currently being 12 sustaining members, 32 corporate members, 7 library members, 445 full members, and 16 freebies. This he said compared to 11,33,7,333 & 16 for last year. An interim statement of accounts was distributed, which showed a total income of £21,542, £12,965 of which was from subscriptions, and an expenditure of £27,726. This included a sum of £20,000 which had been given to underwrite APL-ication, the conference taking place in Canterbury in September, which is expected to be recovered. Net reserves stand at £53,572. Mel reported that £30,000 had been invested on the London money market for one year at just over 9% fixed interest rate, which would mature next February.

In answer to questions from the floor, Mel said that the BAA does not contribute to the British Computer Society, to which the BAA is affiliated as a special interest group. The opposite is true, in that the BAA can take advantage of BCS services to the tune of £500 p.a.

The £20,000 for APL-ication was queried; was it expected to come back? Mel replied it was, with profit.

Income from book sales was noticed to be low this year, at only £100, compared to £738 last year. Last year's figures included book sale income from APL86, it was explained.

In view of the £20,000 earmarked for APL-ication, and the £30,000 locked up in the London money market, was there only £3,572 left for the incoming committee to spend? No, the £20,000 was not included in what had been declared as the assets; the incoming committee had £23,572 to spend plus subs.

There was a query about what the bank charges of £138 were for. They were for getting the facility of being able to receive subscriptions by Access and Visa credit cards. This facility will be especially useful for overseas members. The BAA loses 4% commission on subscriptions paid this way.

The summary of accounts submitted had not yet been audited. An audited version will be published in VECTOR.

Election Results

Phil Goacher read out the results of the postal ballot, which he had just received from the Electoral Reform Society. He reported that the number of votes was up 80% on last year, having received 222 voting papers. Voting was as follows:

Chairman	Anthony Camacho 105
	Peter Donnelly 117
Treasurer	Philip Goacher 137
	John Sullivan 83
Recruitment	Val Lusmore 119
	Paul Chapman 103
Publicity	Jill Moss 81
	Cathy Dargue 139
Education	Jake Ansell 91
	Alan Sykes 126

The other posts were uncontested.

The new committee is made up as follows:

Chairman	Peter Donnelly
Secretary	Graham Parkhouse
Treasurer	Philip Goacher
Meetings	Jonathan Martin
Journal	Adrian Smith
Technical	David Piper
Recruitment	Valerie Lusmore
Publicity	Cathy Dargue
Education	Alan Sykes
Projects	David Eastwood

Special Achievement Award

This award was introduced a year ago, and was awarded at the meeting for the first time. Dick Bowman handed the award, a glass apple richly embellished in gold and silver, to John Scholes of Dyadic Systems Ltd, with the citation "... in recognition of his work as leader of the team that has developed Dyalog APL. For his far-sightedness in 1981 when he anticipated the potential of a second generation APL allied to Unix running on the machines of the future. For his skill in creating innovative software which is at the same time practical."

Graham Parkhouse (Hon. Sec.)
2nd May 1988

Finding out about APL

Bibliographical notes by David Eastwood

One of the perennial enquiries we all get is for APL reference material. There is, of course, a small number of well-known APL reference books that are always mentioned. The BAA Committee produced a 'Top Six' list last year:

- APL - An Interactive Approach (Gilman and Rose)
- Sharp APL Reference Manual (Paul Berry)
- A source book in APL (Ken Iverson)
- APL Design Handbook for Commercial Systems (Adrian Smith)
- APL - the language and its usage (Polivka and Pakin)
- FinnAPL idiom library

... and these are all stocked by the University of Warwick bookshop who run the APL book service for us. Please note that 'Design Handbook' is now out of print; if you have trouble finding it the residual stock (14 copies!) is obtainable from the author. The University of Warwick also carry a number of other general APL texts.

Whilst the books mentioned above are good for a general grounding in APL and themselves carry further references to APL material, we still receive a number of enquiries along the lines of: 'do you know of anything that covers APL as used in..'. I have made some enquiries about further sources of reference material which may be of interest to the membership.

IP Sharp

IP Sharp have a bibliography workspace (1000 BIBLIOG) which is still on the Sharp system and was kept up to date until a couple of years ago.

INSPEC

The Institute of Electrical Engineers run a number of online information services, the largest of which is the INSPEC database which is continuously being updated and covers physics, engineering, computers and control, maths and information technology. I visited the IEE at Savoy Place in London to see what sort of representation APL has in the INSPEC database.

In May 1988 there were in excess of 600 references in abstracts to 'APL' (A Programming Language). This covers virtually all published material that carries some library reference, be it of UK or overseas origin. For example all ACM publications are included.

I think that with so many APL references listed, there must be something which deals with almost any application area.

Access to the INSPEC database is open to all, IEE members or not and current average prices range from £60-70 for non-members (less for members). The library staff will assist in the composition of the enquiry so that you get what you wanted! You can visit, ring or write to:

Technical Information Unit
IEE, Savoy Place, London, WC2R 0BL
01 240 1871

News from Sustaining Members

Compiled by Cathy Dargue

HMW Computing Limited

HMW Computing Limited have relocated to Hamilton House on Victoria Embankment. Here we will be nearer our customers in the City and will be able to bring together the future development of our "4XTRA" Foreign Exchange Dealing and Position Keeping system. In addition to providing demonstration and conference facilities the Hamilton House location will enable us to carry out our maintenance contract commitments more efficiently as we will be much closer to our prestigious clients.

"4XTRA" our front end dealing system for the Banking community is achieving acclaim, and interest has been expressed by Banks, equipment manufacturers/suppliers as well as systems houses. We would be pleased to meet anyone working in the banking and finance fields to discuss how our systems can revolutionise the dealing function.

HMW are also pleased to announce that Phil Last has joined us. He brings his APL2 knowledge to us adding great strength to our growing team of consultants. Our consultancy team is always available for long and short term contract work, and we can supply consultants with experience in other languages in addition to APL, for example C, Oracle and JSP-COBOL. Call Stan Wilkinson for an initial discussion.

MicroAPL Ltd

MicroAPL has continued to record healthy levels of sales of our entry-level software - APL.68000 for the Atari ST, Apple Mac and Commodore Amiga. The distribution of these sales worldwide mirrors quite closely the sales of these machines themselves. Thus, for example, APL.68000 for the Commodore Amiga is selling strongly in the USA whereas the Atari ST version is by far and away the most popular of the three in Germany. The UK sales of the Commodore and Atari versions are roughly equal, but we have noticed some increase in Apple Mac sales recently. We have been particularly encouraged by the take-up on our 'free' run-time version. Software developers are issued with a free run-time version of APL.68000 for use with packages they may develop in APL. The only condition is that APL.68000 receives an acknowledgement in the eventual product. At the moment I am aware of a couple of packages that are being launched in the USA and at least one in Germany with a lot more still in development. Perhaps people can use APL for packaged software!

Users of MicroAPL's own Mirage-based computers, the Spectrum and the Aurora, have been seeing a steady increase in the quantity of non-APL software they can run - they

now have a choice of 7 programming languages (as well as APL of course!), wordprocessing, spreadsheet and database packages. The latest addition is a CAD package which is proving highly successful in the scientific area.

MicroAPL's consultancy operations continue to grow. We are currently recruiting extra staff to enable us to meet the demand, and an advertisement appears on page 32 of this issue.

Cocking and Drury Ltd

Steven Weller joins the company as a trainee in late July. Steven is studying for a degree in Mathematics for Business at Middlesex Polytechnic, and will spend his industrial placement year in both support and consultancy areas. We are again on the lookout for permanent staff with APL skills, to be based in our London office.

There have been several new sales of APL*PLUS Mainframe and Compiler software. The advanced features, efficiency and compatibility with its PC, Unx and Dec Vax counterparts is encouraging its use as the strategic APL for the future.

Testing of the new APL*PLUS System II for the 80386 is going very well; the involvement of selected clients in the Beta-test program is proving particularly beneficial. To remind Vector readers, the new interpreter is a full second-generation system, which takes advantage of the 80386 architecture; 32-bit addressing permits workspace and array sizes that are limited only by installed memory. 15 Megabyte workspaces with nested arrays on your desk are now a reality.

APL*PLUS System II for the 80386 is compatible with both the APL*PLUS PC and APL*PLUS Mainframe systems, further extending the family of APL*PLUS interpreters. The new interpreter is viewed more as a mainframe APL system than a PC system, and will be fully supported under licence with Cocking and Drury acting as the sole UK source. If you require more details about this exciting new development before its launch in September, contact Aelred Tobin, our Micro Products Manager, in our London office.

Statgraphics is proving so popular that we have had to schedule overflow courses to meet the demand. We have also developed a new corporate deal for customers purchasing any of our PC software products.

In order to promote the spread of the APL language Cocking and Drury will include a copy of I-APL/PC, the free international APL interpreter, with each copy of APL*PLUS PC that we send out. In this way APL*PLUS PC programmers will be able to encourage other PC users to try out their favourite language.

I.P. Sharp Associates Ltd

In January 1988 Philips Telecommunication and Data Systems and I.P.Sharp Associates announced the signing of a multi-million dollar agreement for the provision of a Unix implementation of IPSA's decision support tool "Viewpoint". Under the terms of the agreement Philips will market the product, known as "Allround View", worldwide on its Unix-based P9000 computers.

Allround View provides users with a number of features, such as:

- Access to MVS-based databases such as DB2, Adabas, and IBM OS files.
- Access to Unix database systems and files.
- Electronic mail, graphics, ad hoc enquiry facility, and word processing.
- Integration of multiple P9000's in an Ethernet network.
- PC integration under Unix.
- DISOSS support through SNA
- National language version support.

I.P.Sharp Associates have adopted the 'shareware' concept for Sharp APL/PC; 50 copies of Sharp APL/PC diskettes may be made without paying a royalty or fear of copyright infringement. This does not mean that it is free. Registered copies of the software are available from I.P.Sharp and it is these copies that people are free to reproduce and distribute.

Dyadic Systems Ltd

Dyadic is pleased to announce the appointment of Objectif APL as the new exclusive Dyalog APL distributor for France. Objectif APL has previously specialised in APL training and consultancy, as well as in the development of APL applications. The company is run by Bernard Legrand, a well-known author of APL books, including 'APL et GDDM' reviewed in VECTOR 4.1. In a reciprocal arrangement, Dyadic will market Objectif APL's MERLIN, an APL spreadsheet package which is installed in over 50 VS APL sites in France. MERLIN has recently been converted to run under Dyalog APL, and will be demonstrated at APL-ication in September.

Dyadic is nearing completion of its exciting new 'windowing' software for Dyalog APL. This is designed to provide all of the functionality of the APL*PLUS/PC screen software, but will have some important new features such as horizontal and/or vertical scrolling fields. The new windowing features will be on display at APL-ication.

Dyadic has announced new implementations of Dyalog APL for the Hewlett-Packard HP9000-800 series, the Sun-4 and Sun 386i, and the Unisys 5000 models 85 and 95.

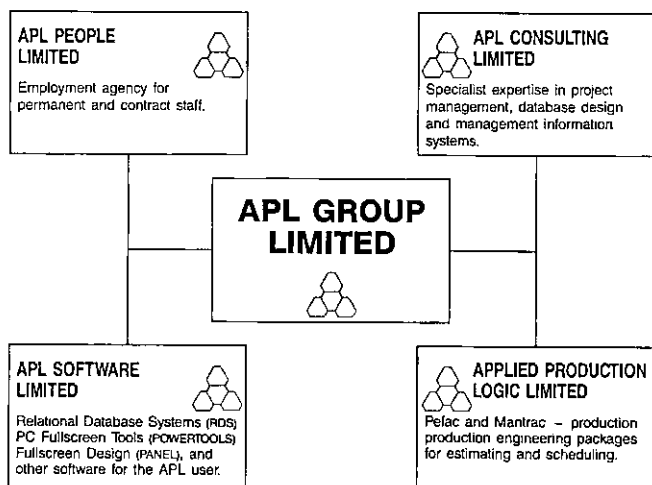
APL People Ltd

Since the last issue of VECTOR, APL People has undergone something of a re-organisation, as many of you will already know. Jill Moss has been appointed managing director to take full control of both the permanent recruitment side of the business and the company's contract work.

With this re-organisation comes a move into larger, more prestigious premises in Bath, and a complete overhaul of the company's image. Our new offices will give us much needed space to expand and take on more staff to cope with the ever-increasing workload, as the agency side of the business continues to flourish. This will be especially important now that we are branching out into C and Unix.

Another area that Jill plans to grow is in helping clients to fill senior managerial positions by means of an 'Executive Search'. At the time of writing, we are assisting a house-hold name in the fast-moving consumer goods industry to fill a vacancy crucial to the success of their IT strategy. Jill hopes to build up a reputation for the company of being able to find people with the right management skills, business background and technical knowledge for senior vacancies vital to the success of major organisations.

Tim Perry's extensive experience in the retail industry has been a major factor in winning several important contracts for the company, thus building on the success we have already had with one of the UK's biggest furniture manufacturers.



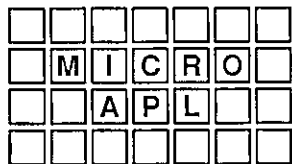
APL Experts

MicroAPL are one of Europe's leading APL specialists and our activities include the supply of the popular APL.68000 interpreter for a wide range of computers; specification and installation of complete turnkey computer systems; and a full APL programming and consultancy service.

We are currently expanding our successful consultancy activities and are seeking to recruit APL programmers, consultants and account managers to meet our existing commitments and to further expand our consultancy operations.

Our work covers all areas of computing from Management Information to Process Control and we work in most APL environments including APL.68000, APL*PLUS, VSAPL and APL2. Our work is mainly based in London and the South-East.

MicroAPL offers competitive salaries which will reflect the level of entry. If you would like to join the team at MicroAPL, please apply to David Eastwood at the address below.



MicroAPL Ltd.
South Bank Technopark
90 London Road
London SE1 6LN

Tel : 01 922 8866

The Education Vector

by Norman Thomson

In this, my concluding article in this series, I am happy to record how confident I am that my successor, Dr Alan Sykes of the Department of Management Science and Statistics at the University College of Swansea, will fill the post of Education Officer with distinction. I am sure that he will bring to it an infusion of new ideas, to hasten the day when APL is widely seen in the microcomputer world as the computer language which excels all others in the field of education. The I-APL movement provides a focus for growth and an entry point for all, however limited their means. In the longer view it would be good to see APL, seeded from this base, spread through the entire education world, with I-APL acting as a catalyst to fuel ever greater reactions.

Continuing in valedictory mode, I relate how on sifting through the belongings of an elderly relative, I came on a dusty old book entitled "Tales of Many Tongues". Opening it at random, I came upon:

"Once upon a time a great wizard made a wonderful invention which helped the workmen in his village make their tools with greater speed and precision, and thereby enabled them to trade favourably with their counterparts in neighbouring villages who did not have this advantage. At first the wizard was content that the invention should remain in his own village, but some of the elders persuaded him that not only the tools, but also the invention itself might be traded more widely. When the word of the great invention spread, people came from all around to marvel at it, and to convey it back to their own villages where they were able to improve their own tool-making skills.

Enthusiasm for the Great Invention grew, so that people from all over the land came to regular joyful gatherings to devote themselves to instruction from the great inventor's disciples. Many wonders and signs were performed, and many lesser inventions were made which sprang from the Great Invention.

As the gatherings grew in size, two types of followers began to emerge: first merchants whose skills were able to transform the Great Invention to profit, and secondly missionaries for whom it became a symbol of salvation from an outmoded culture. In time, the Great Invention spread through all the lands, and merchants and missionaries alike would come together to form local cells of the Great Invention Movement. The teachings of the founder would be written down in books giving guidance to fresh converts, and codes of right conduct to existing believers.

One day the Great Inventor departed from his own village and went to live in a neighbouring one, where he made an even greater invention. Not to be outdone, the disciples who had remained in his old village also made another and greater invention, which they spread through the whole world. Back in the villages, and undeterred by this new division, the missionaries continued to preach, ardent that lack of riches should not deter the enrolment of new believers who continued to increase daily. The merchants sought more books to advance their business, although they were too busy pursuing their trade to find time to write them. The scribes therefore came mainly from amongst the missionaries, and their writings

At this point, a page had been ripped out of the book - perhaps readers would like to continue the story for themselves ...

A Survey of Attitudes to APL in Higher Education

summary by David Eastwood

Last year the BAA commissioned a market research survey into the usage of APL in higher education. Six students at Humberside College of Further Education, in their final year of an HND in Business Studies, carried out the survey as part of course options covering Marketing Operations and Market Research.

The idea for the survey was initially discussed by David Eastwood and David Hewson who lectures at Humberside CFE and is also a BAA member.

The Terms of Reference

The students were given these objectives::

1. Determine the level of awareness of APL in Higher Education
2. Identify areas of usage of APL
3. Identify levels of usage for various programming languages
4. Identify hardware used
5. Gather opinions on APL

The survey was concluded towards the end of the academic year and a presentation was made to the BAA Committee in London. The students involved were: Mark Alston, Jill Southwood, Helen Corcoran, Neil Cotton, Nadeen Vigus and Alan Clifford. Three of the Humberside CFE lecturers also attended - both to listen and to award marks to the students!

I shall outline the way in which the survey was carried out and highlight some of the results presented. The BAA holds a full copy of the report which we will make available to members on request.

The Survey

The survey was conducted by means of a postal questionnaire, with a local trial run, and then a mailing to 311 separate Departments in Universities, Polytechnics and Colleges of Further Education. Departments selected included Accountancy, Business Studies, Computer Science, Mathematics, Statistics and Engineering. 149 replies were received, about 60% from Universities.

Analysis of the results was, of course, carried out in APL, using a program written by David Hewson (thank you David) and the results and conclusions were presented by the students jointly. All the results were presented in graphical form on overhead foils, although I shall confine myself to reporting some of the numbers. As you will see from the various results, the questions asked were aimed at giving the BAA information about what might make more people use APL, and in this context some of the comments sent in, in reply to the final question - 'any other comments?', were very interesting. Some of them are quoted at the end.

Of those surveyed, 90% had heard of APL although only 35% had or did use it. This rather goes against the common feeling that more people would use APL if they knew about it. Perhaps they know the wrong things about APL? Of those who were aware of APL, most categorised themselves as 'Novices'.

The most popular languages used were, in order: Pascal, FORTRAN, BASIC, followed by LISP, COBOL, FORTH. Others used include Prolog, C, Modula-2, Ada and Assemblers. Usage of LISP was high, with about 80 users against the 170 or so for Pascal and FORTRAN. In most cases usage was split evenly between use for teaching and non-teaching purposes. The survey enquired about why a given language was used and the five most important factors were, in order:

- Relevance to an Application
- Importance for teaching
- Importance for research
- Availability
- Familiarity

A similar question was asked, to try and establish what influenced the choice of a language for teaching purposes, and here the answers were ranked:

Relevance to Course
Relevance to Industry
Ease of Use
Machine Compatibility
Versatility
Price
Availability of training packages
Statutory Course Requirement

Just to check that people actually still used programming languages, a question was put about the importance of learning a programming language. The highest response here was 'Very Important', with more Engineers thinking it important than other categories and more Business Studies thinking it not important to learn a computer language. Interestingly, those involved with computer studies were split evenly between the 'Very Important' and 'Not Important' ends of the spectrum.

Hardware used ranged, in order of importance:

IBM PC - BBC Micro - Vax - Prime - Apricot - Other

Conclusions

The students' conclusions can be summarised as seeking to redress what is seen as an image problem for APL, and broadly they suggested:

- Simple introductory material on APL should be prepared
- Training packages relevant to specific subjects should be devised
- Publicity material should be placed in mainstream magazines and papers
- Get key decision makers on education software to attend residential courses

Other Notes and Comments

Given the time available for the students to find out about APL and carry out their survey, I think the results were both interesting and informative. In hindsight, the BAA could have set better questions for the survey and we are certainly planning to repeat the exercise in the near future. I should conclude by handing over to the respondents themselves (I have avoid identifying them in case they are known to readers of Vector):

"What developments would you like to see in the programming languages you use?"

- Site licences
- The ability to write programs like APL using the normal character set
- More international standards
- Few; stability is precious
- Flexibility in linking to other packages
- Cheap APL2
- Improved user interface
- Parallel implementations
- Explicit looping in APL
- More standard APL packages
- Built-in graphics

"I would like to use APL but our computing centre refused facilities"

"Major problems with APL are lack of an effective standard, unusual keyboard requirements"

(The problem of needing a special terminal is mentioned many times)

"APL should be allowed to die, we have too many computing languages"

"APL has a reputation of being a computer boffin's language"

"APL is a dead duck because it is unreliable"

"The major problem is the strange character set and the time taken to master the language"

"How would APL benefit business computer users. Is APL available for UNIX?"

"APL encourages tricky thinking style and ignores questions of efficiency"

"APL is a horrid language. It violates most of the principles of language design. Programs are usually unmaintainable."

"I have yet to find anyone who has learnt APL who has continued to use it once FORTRAN and BASIC were available with reasonable support."

"Many people in APL are too 'into' the language to realise that the rest of the world is passing them by."

"The operators of APL are an attractive and useful feature, but in other respects APL is very dated and lacking in features such as support for data abstraction and structured programming."

MicroAPL for ...

APL on the widest range of micros

PCs

MultiAPL - APL.68000 under PCDOS

Sample configurations:

68000/1MB	£ 995
68000/1MB/Maths processor	£1295
68020/1MB/Maths processor	£2995
68020/4MB/Maths processor	£5495

Prices include: Plug-in card, APL.68000, Keyboard stickers, Manual, STSC emulation software

STSC's APL*PLUS

£450

Single user micros

APL.68000 for the Apple Macintosh

APL.68000 for the Atari ST

APL.68000 for the Commodore Amiga

All versions £86.91

Prices include: APL.68000, Keyboard stickers, Manual, Reference card

Multi-User systems

APL.68000 runs under the following operating systems:

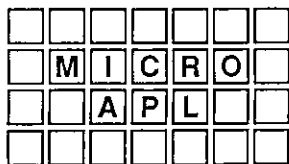
Unix, CP/M68K, Wicat MCS, Mirage

on computers which include:

Sun, NCR, Wicat, HP200/300, Spectrum, Aurora, Stride

Prices range from £1200 to £2000

All prices quoted exclude VAT



MicroAPL Ltd.
South Bank Technopark
LONDON, SE1 6LN
Telephone: 01 922 8866

Access and Barclaycard
Orders accepted

APL Product Guide

Compiled by Cathy Dargue

VECTOR's exclusive APL Product Guide aims to provide readers with useful information about sources of APL hardware, software and services. We welcome any comments readers may have on its usefulness and any suggestions for improvements.

We do depend on the alacrity of suppliers to keep us informed about their products so that we can update the Guide for each issue of VECTOR. Any suppliers who are not included in the Guide should contact me to get their free entry - see address below.

We reserve the right to edit material supplied for reasons of space or to ensure a fair market coverage.

The listings are not restricted to UK companies and international suppliers are welcome to take advantage of these pages. Where no UK distributor has yet been appointed, the vendor should indicate whether this is imminent or whether approaches for representation by existing companies are welcomed.

For convenience to readers, the product list has been divided into the following groups:

- * Complete APL Systems (Hardware & Software)
- * APL Timesharing Services
- * APL Interpreters
- * APL Visual Display Units
- * APL character set printers
- * APL-based packages
- * APL Consultancy
- * APL Training Courses
- * Other services
- * Vendor addresses

Every effort has been made to avoid errors in these listings but no responsibility can be taken by the working group for mistakes or omissions.

Note: 'poa' indicates 'price on application'.

All contributions to the APL Product Guide should be sent to:

Cathy Dargue,
60 Downhall Ley,
BUNTINGFORD,
Herts SG9 9TL.
Tel: 0707-325161 ext 2418

COMPLETE APL SYSTEMS

COMPANY	PRODUCT	PRICES (£)	DETAILS
Dyadic	APL Coprocessor	3,500+	32-bit coprocessor board for IBM PC. NS32000 cpu with FPP, up to 4Mb RAM, 16Mb virtual memory. Software includes Unix V.2, Dyalog APL, graphics support, DOS interface. Provides multi-user Unix/DOS environment.
	IBM 6150	15,000+	Multi-user Dyalog APL system with Fast 32-bit RISC processor, FPP, up to 8Mb RAM, 210Mb Disk, 16 users. Interface to SQL, graphics and APL support for standard IBM peripherals.
	Altos 3068	25,000+	Multi-user Dyalog APL system with MC68020 cpu & MC68881 FPP. Also features a LAN which supports IBM PCs as Dyalog APL terminals.
	Sun 3	15,000+	Multi-user Dyalog APL systems which can be configured as a network of workstations and/or a traditional time-sharing cpu. With its 25MHZ 68020 cpu, the Sun 3/200 is the fastest APL microcomputer on the market.
Inner Product	IBM PC	2,000-6,000	IBM PCs supplied for turnkey applications
M.B.T.	MBT Series 10	poa	UNIX/68010 based multi-user APL system
	TORCH	poa	68000/Z80 multiprocessor
MetaTechnics	-	poa	Details on application -- IBM PC compatible
MicroAPL	Aurora	20,000+	Multi-user APL computer using 68020 CPU. Std configuration 2Mb RAM, 16 RS232 ports, 68 Mb hard disc, 720K diskette
	SPECTRUM	7,000+	Expandable multi-user APL computer using Motorola 68000 Std configuration 1 Mb RAM, 12/36 Mb disc, 12 ports.
	Atari 1040ST	799	1 Mb Mono/Colour System, includes 1 Mb disc drive & mains transformer built into Console.

APL TIMESHARING SERVICES

COMPANY	PRODUCT	PRICES (£)	DETAILS
Mercia	APL*PLUS	poa	STSC's Mainframe Service - MAILBOX etc.
I.P. Sharp	SHARP APL	poa	International Network application systems and public databases.
Uniware	APL*PLUS	call	STSC's mainframe service

APL INTERPRETERS

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL Software	Dyalog APL	1,000-8,000	See Dyadic Systems entry
Cocking/Drury	APL*PLUS/PC Rel 7	475	STSC's full featured APL for IBM PC, PC/AT and PS/2. Now with virtual workspace, quad-MF, GSS graphics, Network and VGA support
	Upgrade 6 to 7	poa	
	Upgrade 5 to 7	poa	Extension upgrades to release 7
	Run-time	poa	Closed version of APL*PLUS/PC which prevents user exposure to APL
	APL*PLUS UNX	poa	STSC's 2nd generation APL for IBM PC/AT, DEC, AT&T and other Unix computers.
	APL*PLUS VMS	poa	Integrated 2nd generation APL for DEC m/cs under VMS
	APL*PLUS mainframe	poa	Component files, quad-fns, nested arrays for VS APL under VM/CMS and MVS/TSO
Dyadic	Dyalog APL	795-10,000	2nd gen. APL for UNIX systems, e.g. IBM 6150, Sun, Vax, NCR, HP9000, AT&T, Altos, Apollo, Whitechapel, Sperry, etc
Gen. Software	APL*MYRIADE	poa	Runs on Texas Instruments TI990 range.
IBM UK	IBM PC APL	poa	Event-handling & APs for full-screen I/O disks, diskettes, asynch. comms

IBM USA	IBM PC APL2	500	APL2 for the IBM PC. Program 5799-PGG/1, PRPQ number RJ-0411. USA distribution only - call your IBM representative for more details
Inner Product	VIZ::APL	250-350	8-bit Zilog Z-80 CP/M
	APL*PLUS/PC	600	See under Cocking & Drury
M.B.T.	Dyalog APL	poa	See Dyadic Systems entry
	MBTAPL	poa	Enhanced Dyalog APL for MBT hardware.
	VIZ::APL	poa	Customized for TORCH hardware
Mercia	APL*PLUS/PC Rel 7	475	STSC's full-feature APL for IBM PC, and compatibles. No 64K object size limit.
	Upgrades 5 to 7	325	
	Upgrades 6 to 7	145	
	APL*PLUS/UNIX	poa	Interpreter for UNIX systems. WICAT, CADMUS, CALLAN, FORTUNE 32:16, HP, 9000/500, OLIVETTI 3B2, SUN etc.
	APL*PLUS/MAC	350	APL for the Macintosh. Big workspaces, big objects, mouse, icons etc
MetaTechnics	APL*PLUS Rel 6	475	Discount on quantity.
MicroAPL	APL.68000	1,000-2,000	Full implementation with component files, error trapping etc. for SPECTRUM, HP300, SUN, NCR etc.
	QL/APL (keyword)	87	Full keyword APL for QL with many extra features.
	QL/APL (APL chars)	87	VSAPL compatible APL for QL with many extra features
	APL.68000 Apple Macintosh	87	
	APL.68000 Amiga	87	Full APL interpreters with support for windows, mouse, graphics etc
	APL.68000 for Atari ST	87	
	APL.68000 for IBM-PC	995	
	APL*PLUS/PC - REL 7	450	
Portable	PortAPL	£195	IBM PC Software
		£275	Macintosh
		£2,995	DEC VAX
I.P. Sharp	APL-LAB/PC + Q pack	480	STSC APL plus object packaging for the PC
	Upgrade V6 to V7	120	
	Upgrade any to V7	200	
	RTS	70	Closed version of APL-LAB/PC
	Sharp APL/PCX V1	1,650	For IBM XT/AT
	Sharp APL/PC V2	40	shareware for IBM PC/XT
	" diskettes only	10	
	" manuals only	30	
		1,000+	For IBM mainframes
	Sharp APL/PC	325	For IBM PC or PC/XT
Uniware	APL*PLUS/PC	495	STSC's full feature APL for IBM PC/XT/AT, Compaq, Olivetti
	Run-Time	call	Closed version of APL*PLUS/PC which prevents user exposure to APL
	APL*PLUS/UNIX	call	STSC's full feature APL for UNIX based computers.
	PortaAPL	280	PORTABLE SOFTWARE's APL for APPLE MACINTOSH.
		2,995	PORTABLE SOFTWARE's APL for the DEC VAX.

APL VISUAL DISPLAY UNITS

COMPANY	PRODUCT	PRICES (£)	DETAILS
Dyadic	Lynwood j300	1,560	Monochrome ANSI 3.64 APL vdu, 15-inch high quality screen, Tek graphics, local macro keys.
	Lynwood j500	2,295	Colour ANSI 3.64 APL vdu, 15-inch high quality screen, Tek graphics, local macro keys.
	IBM 3163	791	Low-cost Monochrome APL vdu. Supports downloaded Dyalog APL font.
	IBM 3164	1,093	Low-cost Colour APL vdu. Supports downloaded Dyalog APL font.

Farnell	Tandberg TDV 2221	995	Ergonomic design APL terminal, 50-19200 baud, 15/8in5 anti-reex screen, low profile keyboard
	Tandberg TDV 2271	1,195	Combined APL/ANSI ergonomic terminal as above.
Gen. Software	Mellordata	400	Second-hand Elite 3045A
M.B.T.	various		Contact MBT for details
Meta Technics	IBM EGA compatible	299	Emulates EGA & Hercules. Half Card
MicroAPL	Insight VDT-1	795	Inexpensive APL VDU
	Insight GDT-1	1,450	With monochrome graphics
	Concept201	1,295	APL VDU with 8 page memory
	Concept 201G	1,650	Graphics VDU
Shandell	HDS2010	1,215	ANSI3.64 DEC VT52/100/220 compatible. 15" tilt/swivel screen, low profile keyboard 8 page memory, windows, viewports, 80/132 columns, full overstrike, 2 or 3 comms. ports, 55 PF keys. NVM storage
	HDS2010G/GX	1495+	As above plus Tektronix 4014, Retrographics VT640/D0640 and Visual 500 compatible. 1024 x 390 or 1024 x 780 resolution.
Tektronix	4114B	13,500+	19" O.V.S.T.:Graphics: 3120 x 4096 displayable; Intelligent. up to 800K memory; APL keyboard (option 4E)
	4125	21,550+	19" 2D colour graphics, Workstation (1280 x 1024), Intelligent: up to 800K memory; APL keyboard (mod AP)
	4128	26,822+	As 4125 plus 3D wireframe

APL PRINTERS

COMPANY	PRODUCT	PRICES (£)	DETAILS
Datatrade	Datasouth DS180+	1,295	180 cps matrix printer with 4K buffer, 9 x 7 dot matrix and APL option.
	Datasouth DS220	1,695	Letter quality; graphics capability, APL option (both available with IBM Twinex or Coax interface).
Dyadic	IBM 4201 Proprinter	poa	100, 200, 40(inlq) cps, matrix printer, with graphics. Supports downloaded Dyalog APL font.
	Toshiba P351	poa	24 pin high-quality matrix printer 100 cps letter quality, 192 cps draft.
Inner Product	Epson FX80	500	Soft char. set, 160 cps, 80 column
	Anadex 9620	1,150	200 cps., 132 col., tractor feed
	Siemens PT88	620	180 cps., 80 col., silent
	TGC Starwriter	1,180	40 cps., letter quality
M.B.T.	Facit 4565	poa	40 cps letter-quality
	Facit 4510/11/12	poa	Matrix printers
MetaTechnics	Quen-data	295	Low-cost APL Daisy-wheel printer
MicroAPL	Datasouth DS180+	1,195	See Datatrade entry
	Philips GP480	2,137	Matrix printer with letter & draft quality and APL (480 cps).
	Qume Letterpro20	549	APL/ASCII Daisy-wheel printer

APL PACKAGES

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL-385	APL-385	50(PC), 125(mf)	including ...
	FSM-385		Screen development
	DRAW-385		Screen design
	DB-385		Relational WS
	GEN-385		Miscellaneous Utilities
APL Software Ltd (mainframe)	AFM/AP	11,035	Interprocess Software for VM/CMS & MVS/TSO.
	- Keyed Access	2,650	Component File Management System (VSAPL/APL2)
	- Interactive Link	1,325	

	- Mail Exchange	2,650	
	CALL/AP	4,030	Non-APL program execution (VSAPL/APL2)
	APLPRINT	2,205	Output to high speed line printer or 328x devices (VSAPL/APL2)
	ENHANCED FORMAT	2,205	Extends Format operator to full "Quad-FMT" status (VSAPL/APL2)
	ISP	750	Input and Output Stack Processors for manipulating terminal I/O
	OSP	2,205	with facilities for Error Trapping (VSAPL)
	DISPLAY CAPTURE	poa	Allows terminal output to be collected and held for retrieval by an APL function (APL2)
	UCF	poa	User Communication Facility for data transfer between users (APL2)
	RDS	poa	Relation Data Base System
	PANEL	poa	Fullscreen management system
	PFS	poa	Program File System - APL Systems development aid
	IPLS	poa	Project Management System
	REGGPAK	poa	Regression Analysis Package
(microcomputer)	POWERTOOLS	295	Assembler written replacement function for commonly used CPU-consuming APL functions, includes a Forms Processor.
	REGGPAK	poa	Regression Analysis Package
	RDS	990	Relational Database System
Beta-plan	BETA-FONT	poa	Multiple font PC character generator. Dealers required for non-Scandinavian countries.
APL IMPETUS	Impetus	poa	Hierarchical Planning System
Butel	Merlin	5,000	Mainframe APL spreadsheet runs under VM/CMS, TSO, VSPC
	Merlin/PC	poa	Version for APL+PLUS/PC
Cocking/Drury (for VSAPL)	E'EMENTS & SHAREFILE	poa	Component files, quad- functions & nested arrays for IBM VSAPL under VM/CMS & MVS/TSO
	SHAREFILE only	poa	
	ENHANCEMENTS only	poa	
	COMPILER	poa	First APL compiler. Available with APL+PLUS enhancements and Sharefile under VM/CMS & MVS/TSO
	FILEPRINT	poa	Print APL component files
	FILESORT	poa	Sort APL component files
	FILECONVERT	poa	Convert non-APL files to APL files
	FILEMANAGER	poa	Extends APL primitives to database management
	TOOLS + UTILITIES	poa	APL Software development tools
	DATAPORT	poa	Powerful Information Centre spreadsheet incorporating data exchange between APL and FOCUS, IFPS, SAS, APL/DI, ADRSII, LOTUS123, VISICALC, MULTIPLAN, DIF files
(for APL2)	SHAREFILE/AP	poa	STSC's sharefile for APL2
	FMT	poa	Full featured FMT for APL2
	WSDOC	poa	
	FILEMANAGER	poa	
(microcomputer)	STATGRAPHICS Rel 2.6	645	Integrated Statistics and graphics on IBM PC, AT, PS/2 and compatibles
	Update 2 to 2.6	140	
	Update 1 to 2.6	330	
	APL+PLUS PC Tools	325	Utilities including: RAM disk, full screen data entry, menu input, report generation, file documentor, exception handling and games.
	IRMA Module	120	327x IRMA support.
	Fin & Stat. Library	350	Financial & statistical routines
	SPREADSHEET MGR	195	APL-based spreadsheet for APL+PLUS/PC. Cell arithmetic; transfers to ASCII, LOTUS

E&S	PROTOPAK		Packages for prototyping management information systems - consisting of: PC & mainframe modules
	RMS		Relational databases
	AMS		Multi-dimensional arrays
	RAMS		Combined RMS & AMS
	BMS		Dynamic financial modelling & forecasting
	FMS		Full-screen handler for APL*PLUS/PC. (AP124-based)
	CMS		Communications package
	SOS	poa	Scheduled ordering and stock control.
FASTCODE	FASTCODE, MONITOR	239	Assembler written monitor for APL applications in APL*PLUS/PC
Gen. Software	PROPS	500+	Spreadsheet system for Product and/or Project Planning.
H.M.W.	INPUT	poa	Matrix manipulation package for data entry & report generation
	PRINTPAK	poa	Block printing for VM/CMS
	VIEWPAK	poa	AP124 Protocol emulator for IBM/PC
	4XTRA	poa	Front-end Foreign Exchange dealing / pos keeping
	Arbitrage	poa	Arbitrage modelling
	Basket	poa	Basket currency modelling
	Menu-Bar	poa	pull-down menu for APL*PLUS/PC
Inner Product	Viewcom	150	Control Viewdata from APL
	APL/dBASE II	150	Interface APL with dBase II
	APL/dBASE III	150	Interface APL with dBase III
	APL/LOTUS	150	Interface APL with Lotus
	APL/WORDSTAR	150	Interface APL with Wordstar
	APL/MULTIPLAN	150	Interface APL with spreadsheet
	CEMAS	3,500	EEC monetary and agrimonetary analysis.
M.B.T.	RHOMBUS	poa	Integrated Ofsu3suce System
	HASLEMERE	poa	Hotel Accounting System
Mercia	STATGRAPHICS 2.6	635	Integrated stat. graphic system for PCs
	Upgrade 1.0 to 2.6	295	
	Upgrade 2.0 to 2.6	145	
	EXEC*U*STAT	395	Easy to use Statistics for management.
	APL*PLUS tools		
	- VOL 1	225	IBM PC Utilities IRMA3270 comms, full screen, RAM Disk report generator
	- VOL 2	125	File documentation, screen editing, Exception handling
	FINANCIAL AND STATISTICAL LIB.	325	Financial and Statistical analysis
	APL Spreadsheet	195	APL spreadsheet - links to popular spreadsheet software.
	MICROSPAN	250	Comprehensive APL tutor
	LOGOL	poa	Logistics management system for PC, Forecasting, Inventory Control, Scheduling, Distribution, etc
MetaTechnics	MetaScreen	99	Full-screen handler for APL*PLUS/PC, based on VSAPL AP124
	MetaPack	495	Comprehensive utilities package for APL*PLUS/PC. Includes MetaScreen, MetaWS, Browse, Toolbox, Numeric Editor.
	APL-IEEE488	99	Controls IEEE488/GPIB Bus from APL*PLUS/PC.
	PLOT/PC	99	2D & 3D Graphics package. Includes interactive diagram Editors.
	Browse	99	Scrolling of DOS files, large APL variables.
	ADAPTA DLS	poa	Production & purchasing scheduling for process manufacturing.
	ADAPTA MSP	poa	Job-shop loading & scheduling for multi-stage production
MicroAPL	MicroTASK	250	Product development aids
	MicroFILE	250	File utilities and database
	MicroPLOT	250	Graphics for HP plotters etc

	MicroLINK	250	General device communications
	MicroEDIT	250	Full screen APL editor
	MicroFORM	250	Full screen forms design
	MicroSPAN	250	Comprehensive APL tutor
	MicroGRID	poa	Ethernet & other networking
	APLCALC	400	APL spreadsheet system
	MicroPLOT/PC	250	For APL*PLUS/PC product
	MicroSPAN/PC	250	For APL*PLUS/PC product
	PC TOOLS Vol 1	295	
	STATGRAPHICS Rel 1	495	
	STATGRAPHICS Rel 2	535	
Parallax	ExecuCalc	≈5,000	Mainframe-based electronic spreadsheet for VM/CMS & MVS/TSO with links to micro products
	ExecuPlot	≈5,000	Mainframe-based colour graphics with micro links.
I.P. Sharp	ACT	poa	Actuarial system
	APS	poa	Financial Modelling
	BOXJENKINS	poa	Forecasting technique
	CONSOL	poa	Financial Consolidation
	COURSE	poa	APL Instruction
	EASY	poa	Econometric Modelling
	FASTNET	poa	Project Management
	GLOBAL LIMITS	poa	Exposure management for banks
	MABRA	poa	Record maintenance/reporting
	MAGIC	poa	Time series analysis/reporting
	MAGICSTORE	poa	N-dimensional database system
	MAILBOX	poa	Electronic Mail
	MAILBOX/PC V.2	65	Full screen front end to IPSA mailbox
	upgrade to V.2	15	
	MICROCOM	poa	Mainframe to micro link
	Q PACK V.7	45	Object packaging for APL-LAB/PC
	Q PACK upgrade	10	Upgrade v.6 to v.7
	Q PACK RTS	10	Run time version
	SAGA	poa	General graphics, most devices
	SIFT	poa	Forecasting system
	SNAP	poa	Project management
	SUPERPLOT	poa	Business graphics
	VIEWPOINT	poa	4GL - Info centre product
	XTABS	poa	Survey Analysis
Sugar Mill	Stat 1	≈129.95	Statistical toolbox, menu driven
Unware (mainframe)	STSC's ENHANCEMENTS	10,715	Quad-functions & nested arrays for IBM VSAPL under VM/CMS and MVS/TSO
	STSC's SHAREFILE	10,715	Component files for IBM VSAPL under VM/CMS and MVS/TSO and for IBM APL2
	PROGRAMMER TOOLS & UTILITIES	5,715	
	FILEPRINT	5,715	
	FILESORT	5,715	
	FILECONVERT	5,715	
	FILEMANAGER (EMMA)	5,715	STSC's database package.
	APL*PLUS COMPILER	21,430	First APL compiler. Complements APL*PLUS enhancements and Sharefile under VM/CMS and MVS/TSO.
	EXECUCALC	3,995	Mainframe spreadsheet compatible with VISICALC and part of LOTUS 1-2-3 under VSAPL (VM or TSO).
(microcomputer)	STATGRAPHICS	725	Statistics & Graphics for PCs.

STATGRAPHICS FCA	140	An add-on module to STATGRAPHICS Factorial Correspondence Analysis.
APL*PLUS/PC TOOLS		
- VOL1	325	Incl. 327 x IRMA support, RAM disk, full screen data entry, menu input, report generation, games.
- VOL2	125	Incl. File documentor, screen editor, exception handling.
SPREADSHEET MNGR	250	APL spreadsheet with built-in ASCII, LOTUS and SYMPHONY interfaces.
APL*PLUS/PC FIN. & STAT.LIBRARY	350	Collection of financial and statistical utilities.
POCKET APL	140	Smaller version of APL*PLUS/PC.
UNIASM	275	Collection of assembler routines for APL*PLUS/PC users.
UNITAB	240	APL*PLUS/PC spreadsheet-like data entry and validation system.
The APL DEBUGGER	105	First released APL*PLUS/PC debugger.
OVERLAYS	250	Fast assembler routines to handle overlays in APL*PLUS/PC.
R.BRIDGE	380	Interface between APL*PLUS/PC & R.BASE 5000.
DMA	380	A version of EMMA (APL database manager) for APL*PLUS/PC users.
APL2C	295	Interface between APL*PLUS/PC and DATALIGHT C language
ADAPTA/DLS	33,333	Production & purchasing scheduling for process manufacturing.
Warwick University	BATS	500 Menu driven system for time series analysis and forecasting using Bayesian Dynamic modelling. Price is reduced to 35 for academic institutions
FAB	free	Training program for the above

APL CONSULTANCY

(prices quoted are per day unless otherwise marked)

COMPANY	PRODUCT	PRICES (£)	DETAILS
APL Consulting	Consultancy	180-550	Expertise in APL system design, project management, prototyping, financial applications, decision support systems, MIS, links to non-APL systems, documentation.
APL People	Consultancy	from 120	Consultants available at all levels, with experience in VS APL, APL*PLUS, APL2, Sharp APL, Dyalog APL, APL 68000, C/Unix, TSO/MVS, VM/CMS, graphics. Additional skills available. OR, statistics, financial, commerce, retail, engineering.
APL Software Technology	Consultancy	poa	Technical & business systems, micros, networking & communications a speciality
Buckland Management Systems	Consultancy	poa	Business and Technical systems in commerce and industry. Designing, programming and implementing applications of O.R and statistics.
Camacho	Consultancy	poa	Specialising in programming & manual writing.
Chapman	Consultancy	150-300	24-hour programmer: APL, C, assembler, graphics; PC, mini, mainframe, network.
Cocking/Drury	Consultancy	120-150 140-200 185-300 275-400	Junior consultant Consultant Senior consultant Managing consultant
Peter Cyriax	Consultancy	100-150 120-200 160-300	Junior Consultant Consultant Senior Consultant
Delphi	Consultancy	poa	Specialising in management reporting systems and APL on microcomputers.
Dyadic	Consultancy	poa	APL system design, consultancy, programming & training for Dyalog APL, VSAPL, APL*PLUS, IPSA APL etc.

E & S	Consultancy	150-250	System prototyping: all types of information system.
FASTCODE	Consultancy	poa	Specialise in improving performance of APL applications on micros & mainframes
Gen. Software	Consultancy	100+	
H.M.W.	Consultancy	poa	System design consultancy, programming. HMW specialize in banking and prototyping work.
Inner Product	Consultancy	200	On-site micro-mainframe APL, PC/DOS & Assembler
Lloyd Savage	Consultancy	poa	Decision support, particularly specialising in Sales & Marketing systems.
M.B.T.	Consultancy	poa	
Mercia	Consultancy	poa	APL*PLUS & VSAPL consultancy
MetaTechnics	Consultancy	poa	Management Information & Production. Engineering, APL - C/Assembler custom programming
MicroAPL	Consultancy	poa	Technical & applications consultancy.
M.T.I.	Consultancy	poa	Specialise in Maintenance and development of existing APL systems
Parallax	Consultancy	£750	Introductory APL, APL for End-user & Advanced Topics in APL
QB On-Line	Consultancy	200	Specialising in Banking, Financial & Planning Systems
Rochester Group	Consultancy	poa	Specialise in MIS using Sharp APL
I.P. Sharp	Consultancy	poa	Consultancy & support service world-wide.
Unware	Consultancy	call	Junior to managing consultancy in APL.

OTHER PRODUCTS

COMPANY	PRODUCT	PRICES(£)	DETAILS
APL People	Employment Agency	poa	Permanent employees placed at all levels. Contractors supplied for short/long-term contracts, supervised or unsupervised. Executive Search service available.
HMW	Employment	poa	Contractors and permanent employees placed.
Mercia	ALLCARD	495+	Memory management unit, allowing 952K under DOS - extra 312K APL*PLUS/PC workspace
	MULTI-APL	195+	Multi-task / Multi-user / Network APL*PLUS/PC with file locking, etc.
I.P. Sharp	Productivity Tools	poa	Utilities for systems, operations, administration & analysts; auxiliary processors, comms software, international network.
	Databases	poa	Financial, aviation, energy and socioeconomic.

VENDOR ADDRESSES

COMPANY	CONTACT	ADDRESS & TELEPHONE No.
Analogic Corporation	Denise Favorat	8 Centennial Drive, Centennial Industrial Park, Peabody, Mass. U.S.A. 01961 Tel: 617-246-0300
APL 385	Adrian Smith	Brook House, Gilling East, York. Tel: 04393-385
APL Consulting	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS. Tel: 0225-462602
	Tim Perry	Jubilee House, Chapel Rd, HOUNSLOW TW3 1XT. Tel: 01-577 3541
APL Impetus Ltd	Cedric Heddle	Rusper, Sandy Lane, Ivy Hatch, SEVENOAKS, Kent TN15 0PD Tel: 0732-885126
APL People	Jill Moss	The Old Malthouse, Clarence St, BATH, BA1 5NS. Tel: 0225-462602
APL Software	David Alis	Jubilee House, Chapel Rd, HOUNSLOW TW3 1XT Tel: 01-577 3541
APL Software Technology	John Hagger	14 Rosewood Avenue, Alveston, Bristol BS12 2PP Tel: 0454415737
	Per Hultin	46 Vicarage Road, South Benfleet, Essex SS7 1PB Tel: 0374550501
Beta-plan APS	Kim Andreasen	Stengrade 75, DK-3000 Helsingor, Denmark. Tel: 45 2 21 48 48
Buckland Management Systems	John Buckland	Westwood, 9 Grove Road, Camberley, Surrey, GU15 2DN Tel: 0276 684327

Butel Technology Ltd.	Mike Munro	Butel House, 3 Great West Rd., London W4 5QJ Tel: 01-995-1433
Anthony Camacho		2 Blenheim Road, St. Albans, Herts AL1 4NR. Tel: St. Albans (0727) 50130
Paul Chapman		18, Trevelyan Road, London, SW17 9LN Tel: 01-767 4254
Cocking & Drury Ltd.	Romilly Cocking	180 Tottenham Court Road, LONDON, W1P 9LE Tel: 01436 9481
		155 Friar Street Reading RG1 1HE. Tel: 0734-588835
Datatrade Ltd.	Tony Checkseld	38 Billing Road, Northampton, NN1 5DQ. Tel: 0604-22289
Delphi Consultation	David Crossley	Church Green House, Stanford-in-the-Vale, Oxon SN7 8LQ. Tel: 03677-384
Dyadic Systems Ltd.	Peter Donnelly	Park House, The High Street, Alton, Hampshire. Tel: 0420-87024
E & S Associates	Frank Evans	19 Homesdale Road, Orpington, Kent BR5 1JS Tel: 0689-24741
Farnell International Instruments Ltd.	R. Fairbairn	Jubilee House, Sandbeck Way, Wetherby, W. Yorks Tel: 0937-61967
	Roger Attard	Davenport House, Bowers Way, Harpenden, Herts. Tel: 05827-69071
FASTCODE	Andrew Dickey	P.O. Box 281, Croton-on-Hudson, New York 10520, U.S.A. Tel:(914) 271-3200
General Software Ltd	M.E. Martin	22 Russell Road, Northolt, Middx. UB5 4QS Tel: 01-864 9537
H.M.W. Computing Ltd	Stan Wilkinson	Hamilton House, 1 Temple Avenue, Victoria Embankment, LONDON EC4Y 0HA Tel: 01-353 4212
IBM UK Ltd	Chris Self	PO Box 32, Alencon Link, Basingstoke, Hants RG21 1EJ. Tel: 0256-56144
Inner Product Ltd.	Dominic Murphy	Eagle House, 73 Clapham Common Southside, London SW4 9DG. Tel: 01-673 3354
Lloyd Savage Ltd	Philip Johnson	Cambridge House, Oxford Road, Uxbridge Middx, UB8 2UD Tel: 0895-59826
Mercia Software Ltd.	Gareth Brentnall	Aston Science Park, Love Lane, Birmingham B7 4BJ Tel: 021-359 5096
MetaTechnics Systems	John Stenbridge	Unit 216, 62 Triton Road, London, SE21 8DE Tel: 01-670 7959
MicroAPL Ltd.	David Eastwood	South Bank Technopark, 90 London Road, LONDON SE1 6LN Tel: 01-922 8866
M.T.I.	Ray Cannon	7 Pine Wood, Sunbury-on-Thames, Middx. TW16 6SH Tel: 09327 80848
Parallax Systems Inc.	Kevin Weaver	60 West 9th Street, New York, New York 10011, U.S.A. Tel: 212-475-4001
Peter Cyriax Systems	Peter Cyriax	213 Goldhurst Terrace, London NW6 3ER Tel: 01-624 7013 (Answerphone) 0860-377963 (Mobile)
Portable Software	Richard Smith	60 Aberdeen Ave, Cambridge, Mass U.S.A. 02138 Tel: 617-547-2918
Q8 On-Line Systems	Philip Bulmer	5 Surrey House, Portsmouth Rd Camberley, Surrey, GU15 1LB Tel: 0276-20789
The Rochester Group	Robert Pullman	164 Pinnacle Rd., Rochester NY 14620 Tel: 716-461-3169
Shandell Systems Ltd.	Maunce Shanahan	12 High Street, Chalfont St Giles, Bucks HP8 4QA. Tel. 02407-2027
I.P.S.A. Ltd.	David Weatherby	10 Dean Farrar Street, London SW1. Tel. 01-222 7033
Sugar Mill Software Corp.	Lawrence H. Nitz	1180 Kika Place, Kailua, Hawaii 96734 Tel: (808) 261-7536
Tektronix UK Ltd.	Paul Morgan	Fourth Avenue, Globe Park, Marlow, Bucks SL7 1YD Tel. 06284-6000
Uniware	Eric Lescasse	15 Rue Erlanger, 75016 Paris, France Tel. (1) 45-27-20-61 Telex: 648348F UNIWARE
Warwick Univ.	Dr Andy Pola	Dept of Statistics, University of Warwick, Coventry, CV4 7AL Tel: 0203-523066 x2158

Interfacing APL and C

by Romilly Cocking

Why Interface?

At first sight, you may see no need to interface APL with other languages. APL is powerful and expressive, and modern implementations are very efficient. Fast hardware is affordable, and interpreters like the new APL*PLUS II system for the 80386 give the power of a dedicated mainframe to top-end PC users.

You may still want to consider using other languages with APL, however. There are several reasons for this; one of the most common is that your application includes an algorithm that is iterative or that works on scalar values. (This often happens when you are trying to optimise something, or simulating a complex system.) You can often see in advance of coding it that an APL version of that algorithm would run too slowly, but you want to write the rest of your application in APL. If you are a mainframe APL user, you might write the relevant code in APL and then compile it; on a PC, you would need to write it in a compiled (or assembled) language which you can invoke from APL.

A second need arises when your application is written, but working too slowly. A monitoring tool like Quad-MF can help you to analyse the reasons for poor performance, but you may not always be able to speed up the APL code that is causing the problem. This may be because the code is iterative or scalar - but it can also arise if APL cannot take advantage of some known characteristics of your data. Perhaps an example will clarify this problem. Suppose that you are consolidating financial data for an international sales organisation, in which you have sales for branches, and you want to total these by district, area and country. You may well use an APL inner product to do your calculation. If so, you will often use a boolean argument to specify which branches belong to which districts, and floating point numbers for your sales figures. APL will diligently unpack your boolean values and do floating-point multiplications with them, instead of just using a 0 or 1 to decide whether to include the number in a total. If you recode this one function in a lower-level language, the speed-up can be dramatic.

You may encounter a third reason to interface APL if you want to use existing code that has been written in another language. There must be many thousands of specialist libraries available; libraries of mathematical functions, interfaces to databases and spreadsheets, drivers for graphics display devices and many, many more. If you want to use them in your APL application, you will have to interface to the language in which they are written. You may also want to use an APL front-end to existing application code.

Which Language?

You are confronted with a huge choice of languages in which to develop your own software, and many of them are now supported by commercially available libraries. Which language should you use? If you are using STSC's APL*PLUS/PC or II system for the 386, your choice will be strongly influenced by the interfacing capabilities of the interpreter that you are using. Although you can invoke application programs using quad-CMD, the most efficient way of interfacing an APL application with routines written in other languages is via the quad-CALL system function.

STSC's quad-CALL allows you to invoke assembler code (held as an array) from within an APL workspace. The assembler code is subject to certain restrictions; in particular, it must be relocatable, since the array containing it may be moved within your workspace by the APL interpreter.

You could restrict yourself to assembler as your language for use with APL, but assembler is not always ideal. Most people find it slow to write, and floating-point calculations are particularly difficult to develop. If you wish to use pre-written software, you may have no choice but to use something other than assembler; the software may be written in another language.

For many people looking for a general language to use with APL, C is an attractive choice. Most APLers who have come across C like the language, and learn it quickly. They find that C compilers are readily available, as is a large volume of public domain and commercial software. C code is portable to many environments, and many interfaces to databases and spreadsheets are implemented in C.

If you have already coded your application, and the portion to be speeded up is written in ISO APL, you might even be able to use the SOFREMI APL compiler to generate C code directly.

Overcoming the Problems with Using C and APL.

Alas, the choice of C as your interface language is not without problems. The code produced by standard C compilers does not meet the restrictions that quad-CALL imposes. Moreover, C data structures differ from those used internally by APL. Worst of all, both APL and C have their own dynamic memory management methods, which can clash.

Fortunately, these obstacles have been overcome for us by a commercially available package, developed by Lauer Software. Their APL2C interface allows you to develop applications that run under APL*PLUS/PC using both APL and C. The package contains four main components - a manual, a copy of the Datalight C compiler, libraries of C code

and an APL workspace. The workspace contains a source file editor (ED) which can also trigger the compilation and linking of your C code, so you can edit, compile, link and test from within the APL environment. In addition, registered users get a newsletter containing technical tips and news of new releases. Upgrades are available at a nominal charge.

The current version (4.11) runs under APL*PLUS release 7, and comes with v3.1 of the Datalight compiler. In addition, it can be used with version 3.2 of Lattice C. The remainder of this article deals with my experience of the interface in practice.

My situation is unusual, in that I have just upgraded from an earlier version of APL2C that did not include the Datalight compiler. Both the APL2C software and Lattice C were installed from scratch, however, as I was upgrading my hardware at the same time.

The APL2C installation instructions are comprehensive, and easy to follow. (They leave the APL2C workspace in an APL2C sub-directory, however, without telling you that you will have to gain access to this library in order to load it. I suppose the assumption is that anyone experienced enough to attempt interfacing knows enough to work this out for themselves.) The installation of Lattice C is also straightforward; when both products have been installed successfully, you can carry on with the introductory example in the manual.

I suspect that this example works fine if you are using Datalight, but the C code generates compiler errors with Lattice C. The reason is simple; return statements have been omitted from the end of each C function. Presumably the Datalight compiler inserts them for you. A corrected version of the code for Lattice users is included as Appendix A.

This is not the only setback that Lattice users will encounter. The sample compilation in the APL2C manual covers three situations: compilation and linkage of C software for the two compilers under DOS, and for the Datalight compiler using the ED APL-based editor. If you want to compile using Lattice C from within ED, you need to customise the compile and link commands; you can find out how to do this in a much later section of the manual (p98 et seq., on ED).

Two Sample Applications.

It took me about half a day to install the software and get everything properly set up; I then went on to use the interface for two sample applications. The first was a utility for use with STSC's spreadsheet manager. The second was a utility to perform character set translation.

The first utility was a C version of code that I had prototyped in APL, but which ran too slowly. The utility took a LOTUS spreadsheet name and a range name; it returned the

coordinates of the corners of the range within the spreadsheet, or an error code if something went wrong. It worked by reading through the spreadsheet file, record by record, until it came across the record that it required.

My APL original worked, but it ran rather slowly. It took about four seconds to locate a range that existed; if I gave a non-existent range name in a large spreadsheet, the APL version took about forty seconds to pass the file before giving up.

I used quad-MF to confirm that most of the time was spent reading records and checking their type. It looked as if this would run much faster in C; after half a day of coding, I was able to test the C code using the APL2C interface. It took less than two seconds to find a range, and less than four to establish that a range did not exist.

The speed-up of two to one on a four second operation was worthwhile, but the speed-up of ten to one on a forty second operation could mean failure or success for a commercial system. Encouraged by this, I took a look at a much simpler problem that I hoped C could solve.

Most of us have met the need to do character set translation. It may just be a matter of changing lower case text to upper case; it may be that we are moving data from one computer to another. This is very easy to do in APL; the idiom below takes W, a character array, and translates each character in it using a 256-byte translate table A.

```
Z←Z[⊖A⊂W]
```

Although this idiom is easy to code, it is slow. I thought that a C version would run much faster, and so it did. A large (5K) character vector took about 3 seconds to translate using the APL idiom, and less than half a second using C. The C code was easy to write, and (like the APL idiom) the code worked first time. A C listing is included as Appendix B.

I felt very pleased until Jonathan Barman (a colleague of mine from Cocking & Drury) gave me his assembler function to do the same job. It ran about ten times faster than the C code, and sixty times faster than the APL. He explained that this was as he expected, since the assembler code used a hardware instruction that did just what was wanted at hardware speed.

Looking at Trade-offs.

You may feel from all this that the trade-offs in using different languages are complex. Things are further complicated by some idiosyncrasies of C (or the APL2C interface). Firstly, C distinguishes between scalars and other arrays in a way that APL does not. This difference is echoed in the interface, which passes pointers to APL arrays, but values of APL scalars. If you want to write a function using the interface that behaves in

the same way with an argument of any rank, you will have to do some un-natural juggling to achieve this.

A second problem occurs with memory management. Much C code uses dynamic memory allocation - a method by which space is requested when the program needs it, and then freed when it is no longer required. This happens explicitly in C, but it also happens implicitly in APL. The two languages use different methods of handling the problem, and although the interface tries to deal with this it does not do so completely.

Another thing that you can take for granted when using APL is data typing. Most of us do not need to worry about whether a particular array is integer or real. In C, we have to know this - and know it in advance. This causes two problems: if we do a calculation in C on integers, and produce an answer that is too big to hold as integer, nothing will tell us that we have done so; and if we do a calculation in APL on Real numbers that happens to give us an integer answer, APL will change the type without consulting or informing us. The trick for getting around the second problem is covered by the interface (see pp22, 97 of the manual), but I know of no general solution to the first.

C code may take less time, but it does take up a lot of room. The object code for my simple translate example takes 11K bytes! Almost all of this is due to the inclusion of library code that I do not need, but it would not be trivial to eliminate this. (By contrast, Jonathan's assembler function is about 300 bytes.)

Finally, debugging C code from inside APL is not easy. We are spoilt by the excellent debugging facilities that form an inseparable part of the APL environment; these days, C programmers can use debugging aids that are comparable in convenience and power. Unfortunately, there seems to be no way to use these when running C code via the APL2C interface.

Conclusions

Hybrid systems can offer real benefits, and C is a good choice if performance needs a speed-up, and the algorithm is complex or includes floating point calculations. Moreover, C allows you to use a lot of existing code. Assembler may be better for simple integer or char manipulation.

Appendix A: Sample C program from Lauer manual

... corrected for use with Lattice C.

```

/*****
/* Timestamp: Sat Jun 25 1988 05:52:18 PM          */
/* Program Name: demo prog from Lauer Software manual */
/* Programmer: Romilly Cocking                    */
/* Copyright (C) Cocking & Drury Ltd 1988         */
*****/

#include <apl.h>

int csub1(),csub2(),csub3();
int (*rlookup[])() = {csub1,csub2,csub3};
int numfns = 3;

csub1() /* no arguments */
{
    printf("Hello world!\n");
    return(0);
}

csub2(n) /* one scalar argument */
int n;
{
    int i;

    for (i=0; i<n; i++)
        printf("Hello from C level. i=%d\n",i);
    return(0);
}

csub3(x) /* one array argument */
unsigned x;
{
    int i;

    printf("Rank of X = %d\n",RANK(x));

    for (i=0; i<RANK(x); i++)
        printf("Length of dimension %d is %d\n",i+1,peekw(x,8+i*2));
    return(0);
}

```

Appendix B: Source for Translation Function

```

/*****
/* Timestamp: Sun Jun 26 1988 06:27:45 PM */
/* Program Name: TRANS */
/* Programmer: Romilly Cocking */
/* Copyright (C) Cocking & Drury 1988 */
*****/

#include <apl.h>
#include <aplobj.h>

APL2CFN(trans); /* insert interface code for function 'trans' */

char tab[256]; /* holds local copy of translate table, for speed */

trans(inseg,tabseg,outseg) /* inseg - segment address of APL input data
                           tabseg - same for translate table
                           outseg - same for result */

unsigned inseg,tabseg,outseg;
{
    char far *ip;
    char far *tp;
    char far *op;
    int i;

    ip = apl_object[0].start; /* set ptr to data values in array */
    tp = apl_object[1].start;
    op = apl_object[2].start;

    for (i=0; i<256; i++) /* copy each table entry into local array */
        tab[i] = *(tp+i);
    for (i=0; i<numelems(inseg); i++) /* for each element of array */
        *op++ = tab[(int) *(ip+i)]; /* tranlate and store in result */

    return(0);
}

```

Algorithmics by Harel

reviewed by Anthony Camacho

This is a very good book. The author's style has a refreshing directness and clarity. The subheadings of the chapters are a good sample and also show you the contents.

Preliminaries

1. What it's all about
2. Getting it done
3. Getting it done by computer

Methods and analysis

4. Getting it done methodically
5. Getting it done right
6. Getting it done cheaply

Limitations and robustness

7. You can't always get it done cheaply
8. Sometimes you can't get it done at all
9. The simplest machines that get it done

Relaxing the rules

10. Getting it done by co-operating
11. Getting it done by tossing coins
12. Are they better at it than us?

More knowledgeable reviewers have written that the book is an excellent survey of computer science (which I never studied) and it is certainly an excellent introduction to the range of computing that I know about and to some unfamiliar bits. I think it could be read with profit by intelligent sixth formers. It does require the reader to think. It encourages him to pause and to try to do things for himself too. Do not expect to get through its 400-odd pages without effort.

The bibliographic notes give, in 55 pages, a first rate survey of the best and seminal books on computer science. With a guide like this there is no excuse to be at a loss for a book on any topic in computer science. Such a guide (but to APL books) is just what we should have in Vector (reprinted say, once a year). If there was such a survey of the APL books in print and a reliable source which stocked them all, yet another small obstacle to the spread of APL in education would be removed.

The sections in the book which most interested me were those on the efficiency of algorithms and the non-computability of some kinds of problems, either because no conceivable machine could be fast enough or large enough or because the problem turns out to be logically undecidable.

Vector readers will be glad to hear that APL has a two page summary in the section on programming languages which, although one could cavil at some things in it, is clearly written by someone who appreciates what APL is like. It is not surprising in a book essentially about programming that APL's strength as a notation is not mentioned. .pa Every subject has its traditions and prejudices. Those of computing are I was going to say what I think they are, but my view is equally prejudiced. How would we find out? An answer is to read a good survey of the subject and look for the occasions where solutions are approved of when no reason is given. This book is full of such value judgements because it is written in a lively style; the author's aim is plainly to explain things as clearly as possible and he doesn't care a bit if this clarity is used against him. In "Algorithmic methods" many of the solutions are described as 'better'. Some authors (should I say all politicians!) try to appear wise by saying things in a way which allows them to escape responsibility for what they dare not say straight.

So here is an illustration of what I found. How do we find the maximum distance across a convex polygon? Find all the distances from vertex to vertex and take the largest? No, he says, it's better to take a vertex, find the two vertices furthest from it, record the distance to the further one and then move one vertex round the polygon and see if one of the next pair of distances is larger and if so record that, until one has tested every vertex. His example allows him to demonstrate the ingenuity called for to reduce computation (which begins to be effective when the number of sides gets large), but he assumes that the solution must be a looping one.

I have said before that I think computer training too often over- emphasises looping so that students' ability to think in parallel is atrophied. I'm sure this is a mistake of the educationists. Parallel solutions (using J. J. Sylvester's 'multiple quantity' as Donald MacIntyre would say) can be just as elegant and can give even more delight than the cleverest recursion.

I do not think it is a fault of this book that parallel thinking appears so little in it. The reason is that it reflects the real world and there is too little parallel thinking there too. As Harel writes at the end of the chapter on Parallelism and concurrency:

"Many of the best parallel algorithms known cannot be implemented because existing parallel computers are inadequate in some way. On the other hand, we still do not know enough about designing concurrent programs and systems to take full advantage of the features those same computers offer. The work continues, however, and many people hope that the next few years will produce developments that will put algorithmic and computerized concurrency on a par with the more manageable notion of sequentiality."

I hope he's wrong about "more manageable", and that it is a computing prejudice; if he isn't yet, I hope he will be made wrong one day.

Algorithmics by Harel
Addison Welsey 1987
Price £14.95



APL CONSULTING

**QUALITY APL CONSULTANCY SERVICE
SPECIALIZING IN:**

- ★ Project Management
- ★ Prototyping and Database Design
- ★ Marketing Information Systems
for Management
- ★ Design and implementation of
large financial systems

We can solve your APL problems, large or small,
and will supply a single consultant or a supervised
team according to your needs.

Contact Jill Moss or Tim Perry on 0225 462602
APL Consulting Ltd, The Old Malthouse, Clarence St., Bath BA1 5NS

RECENT MEETINGS

This section of VECTOR includes notes on a number of BAA meetings which have taken place during 1988.

I would like to thank the speakers for making available such a complete text of their talk; many members find it hard to attend as many London meetings as they would like, and this section of VECTOR provides a valuable service to them.

Through IRMA to IMS

by David Alis

Introduction

This talk shows how the PC with an IRMA board can be programmed to act as an intelligent interface between separate applications running on a mainframe. We have succeeded in using a PC under function control to move between different parts of an ordinary IMS application; we were able to recognise sequences of various transaction screens and then read or update named fields in these screens. Instead of using terminal emulation programs (e.g. E78 or I78TERM) or file transfer programs we used APL to read and write to the IRMA buffer directly. To do this in a systematic and structured way we developed a data structure, a design tool and a high level syntax which we used with a function-alias file.

Our client was the Technical Director in a business with several mainframes. One of these was a large IBM 3090 designated the Production Machine to run the main commercial systems, while a smaller IBM 3081 was allocated as the End User machine, running SQL/DS, AS, APL2, PROFS etc.

What the director needed was an update to his resource requirements database using the projections about milestone events calculated in his Pert networked project planning database. But what his problem turned out to be was that the company's resource requirement and planning system (MRRP) ran on the Production Machine under an IMS database, while the Pert planning system (IPLS) ran on the End User machine in a VM environment. There were no interfaces between them! Because Management Services had estimated that it would take 2 man years to build an interface between the IMS based MRRP system and IPLS on VM he thought it worthwhile to invite our suggestions.

Because the IPLS system had a module for moving data between the VM mainframe and PC using APL programs he said "Surely we can get the data we want out of MRRP, tweak it and then put it back. Aren't we doing this already using PCs and IPLS every day?". Within a week this conversation was to lead to a spectacular row between Directors about which was the more important, production systems or End User computing!! More importantly, at least for us, it was the point of departure for work which led to the Programmable Irma Interface described below.

Outline Description

Data in MRRP about resource requirement plans was held in the IMS machine, and data in IPLS which was to be used to modify the resource plans was on the VM machine. Data

in both systems was organised in corresponding work packets. The objective was to adjust the IMS data to reflect the current position as indicated by VM data.

Solution

The 'intelligent terminal' method was selected. A PC fitted with IRMA using the minimum of operator interaction and running under PC program control would do the following:

Step 1.....download data from IMS into file A on the PC.

Step 2.....download data from the VM machine into file B on the PC.

Step 3.....use an APL*PLUS/PC workspace to combine the data in the two files and create file C with the adjusted resource requirements.

Step 4.....upload the data from file C from the PC back into IMS.

The users would be management assistants with no special computer skills and they would need to be able to update individual work packets as selected. To protect the production system from unauthorised access the user would have to logon and logoff manually in the normal way; i.e. the PC programs would have to pass control to the user at logon/logoff.

We could modify an IPLS module to get the data we wanted, but we had to use the existing MRRP/IMS system exactly as it stood, i.e. we had to access the data required from the application's existing transaction screens because no changes were allowed to MRRP!

Thus our technical requirement was to write APL programs so that the PC appeared to the IMS machine exactly as if it were a normal user doing the usual MRRP transactions. Simulating a normal user meant being able to recognise responses from the mainframe system. These responses are either expected or unexpected. Unexpected responses include messages from the operator (e.g. 'closedown in 10 mins'), or from the IMS system (e.g. 'record cannot be found', or 'input not a proper date'). If an unexpected response was received the programs had to pass control back to the user.

The system would have to be flexible enough to detect unexpected changes to the transactions screens caused by IMS maintenance programmers. It would also have to be very highly structured if we were to have any chance of tracing and debugging the programs. From our previous experience of building PC-mainframe links we knew that our greatest difficulty would be in actually getting things exactly right because, as with all inter-computer communications, even the slightest fault can bring everything to a grinding halt.

Communicating with VM and IMS

Access to the VM/PLS data was done by incorporating an auxiliary processor for controlling data flow between APL workspaces on the PC and in APL2 on the mainframe. This auxiliary processor was written by adapting some of the IRMA APL*PLUS/PC Tools; The AP comprises about 15 APL programs on both mainframe and PC.

All communication with IMS was to be controlled entirely by programs on the PC. These programs comprised a Data Structure, a Design Tool, a high level syntax and a Function-Alias file.

The idea behind the function-alias file was the need for a self-documenting system. What we wanted was a scheme for invoking APL functions by comment or description rather than by function name. The function-alias file we developed also provided the mechanism for structuring the programs. The function-alias file worked alongside a high level syntax which consisted of about ten functions, e.g. OPENSREEN, OPENVIEW, CLOSESCREEN etc.

Using the function-alias file and these functions, our APL driver for moving from the 'MAIN MENU' screen to details of the resource details looked like:

```

      ▽ DRIVER1
[1] OPENSREEN'MRRP100'
[2] OPENVIEW'MAINMENU'
[3] WAIT
[4] CHECKERROR
[5] CHECKMESSAGE
[6] EXECUTE'Select Enquiry Mode'
[7] PRESSENTER
[8] OPENSREEN'MRRP200'
[9] OPENVIEW'SELECTION'
[10] WAIT R ◊ R+CHECKERROR R ◊ R+CHECKMESSAGE
[11] EXECUTE'Select individual packages, not summary level'
[12] PRESSENTER
      ▽

```

The functions OPENSREEN & OPENVIEW refer to the function-alias file and collectively define a table of the names of APL functions (the ones with all the messy details) and a table of text, or aliases. In the above example 'MRRP100' refers to an IMS screen which had MRRP100 in its top left, i.e. it's the name of the screen. 'MAINMENU' and 'SELECTION' are names we assigned to 'views' we had defined on the screens. Line 6 EXECUTE'Select Enquiry Mode' is an example of invoking an APL function by its description. (What this particular APL function did to get the character 'E' typed into the proper field on the screen).

To be able to read from or write to the IRMA buffer and the real fields on the 3278 we needed a method of allocating sets of fields to APL variables. To help us we made

extensive use of the POWERTOOLS/PC processor, not only because it is well equipped to define and manipulate fields and matrices within larger matrices, but also because its other facilities for controlling user input at the field level made it invaluable for designing the data structures.

The term I3278 was coined to describe the Data Structure and an I3278 structure was defined for each screen (our application needed 10 such structures). Each I3278 is composed of several sections: a name to associate it with an application screen, the definition and location of special fields such as 'signature' and 'message' fields. These tell us where to look on the screen either for operator messages or to check that the screen is the one we expect to see. It also has definitions of 'view' substructures and the name of the function-alias file it uses.

The idea of a 'view' came from the fact that several of the screens in our application had more than one use; the use depending on the context. In our application we needed both the timespan and the details of the work package. The only way to measure the timespan was to set the scale to years and afterwards set it to months to get the details. Views are themselves defined as substructures consisting of a name, a table of field names which may be referred to within APL functions to get (or put) actual data and a format matrix specifying how 'real' fields on the 3278 screen are associated with each named field.

Conclusion

The idea of an intelligent interface between mainframe applications using PCs was successful and our choice of the technical design and system structures was instrumental in being able to complete a working implementation within timescales that were less than a quarter of those estimated by Computing Services. We think that the large differences in timescales continues to be a source of frustration to End User Directorates, and a source of threat to Computer Services. Our client, as far we know, has ended his battle with the other Directors with honours even; it was realised that the benefits of other necessary enhancements to the systems would justify the cost of a complete redesign of MRRP.

We are surprised at how reluctant management seems to be to grasp the genuine opportunities offered by coupling PCs to mainframes; opportunities for new types of applications, as well as for reducing operating costs by devolving significant amounts of processing to local high powered (i.e. 80386 based) PCs.

Controlling Real-time Processes in a Brewery (with APL)

notes on a talk by Robert Poate (MicroAPL)

Robert Poate of MicroAPL talked about an application in an area which ordinary APL programs don't normally reach, namely real time control of a continuous flow process. The continuous flow in this case was kegged beer in Romford Brewery, a large plant with a capacity of 1 million kegs per year in a series of highly automated lines. The task which Robert undertook on behalf of MicroAPL was to install a management information system which draws its data from 8 plc's (Programmed Logic Controllers) located at strategic points in the keg-filling operation, and feeds it via RS232 links to a MicroAPL Spectrum running the Mirage-2 operating system.

This allows event handling and multitasking between 9 jobs, one corresponding to each of the plc's and the ninth to a filer job which makes updates to a series of rolling files every 15 minutes. These files contain the material for reports on production and rejects, as well as the current status of each of the filling lanes. About 4 days worth of such files are retained on disk, and the filer job runs a daily compression program which writes a consolidated day file. The total amount of data so stored is about 6 megabytes every 4 days. The shared memory which the Mirage system allows between the communicating jobs works like APL shared variables, and is much faster than a RAM disk.

The effect has been to provide management with a massive amount of information in the form of graphs and tabular reports. Among other things, it has allowed them to monitor cumulative production closely against target patterns. It has also given them the means to discover (for example) which depots have a habit of returning kegs the wrong way up, and a recent inference from the data led to the discovery of a CO₂ leak before a substantial loss had occurred.

With factory networks appearing more and more commonly, this sort of application is clearly one of which a lot more instances will be seen, and it is pleasing to know that APL is capable of supporting its share of the action. Robert concluded by making a plea to writers of such systems to make them highly extendable, a quality which APL makes it easier than most languages to instil.

**BAA Technical Meeting
25th March 1988 at the Royal Over-Seas League**

notes by Adrian Smith

This was generally accepted to be one of the best technical meetings we have ever had. There was a good attendance, and some keen questioning, and active discussion in the tea-break. There was however, a rather sad lapse in the organisation, as Romilly Cocking waited in vain for nearly 15 minutes for someone to open the proceedings.

Romilly eventually recalled his session at Loughborough in 1983, when Dick Bowman (who was chairing his session) opened with the words "Here is Romilly Cocking, who clearly feels he needs no introduction - so he isn't going to get one!" To this Romilly wittily added "... I didn't realise it was a life sentence!!"

This had the undoubted effect of getting the audience thoroughly on the speaker's side, and the rest of the afternoon passed off without a hitch. Maurice Jordan had the benefit of an impromptu introduction from Jonathan Martin, along the lines of "Here is Maurice Jordan, who is my boss. My personal review is due next week ... he's a wonderful boss thank you."

Developing Business Systems

Romilly Cocking (notes by Adrian Smith)

Some Examples

SAM Sales and Marketing in consumer goods. Like all these big systems, it is really terribly simple in operation. Data is dimensioned by "Time,Product,Outlet", where each cell has info about product price etc. The typical question "How are we doing?" requires access to 1Gb of data, with many reports per day. The users rate this as the highest value system they have ever put in.

TULIP "... how long before we can start selling these new pension plans?" "5 years" quoth DP, so the Insurance Admin Dept wrote it in APL themselves. Eventually their system was redone in COBOL; it went from 150Mbyte in Sharefile to 1.5Gbyte in Adabas, and took about 20 times as long to write as the APL original!

Petroleum DB . Some 200Mb of data with complex joins between 17 tables. How do you relate different geologists views on stratigraphy? You have 48 hours to decide if you are going to take a share in an oil field "Let's sit down at the terminal and find out"

The developer must focus on two needs:

- systems are professionally developed. They are not end-user systems, nor can they go through the COBOL grinder.
- systems have to track the changing needs of the business.

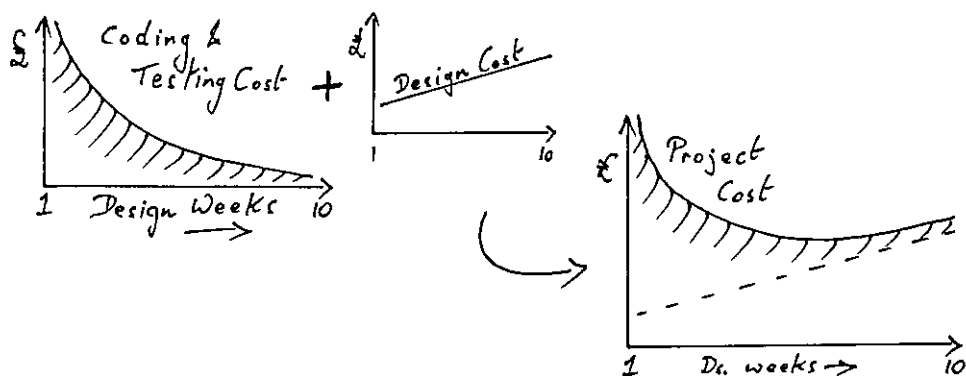
The closer the functional fit to the user's needs, the higher his productivity gain, and the more time he will spend using the system.

Key Objectives for Managing System Development

The system has to be demonstrably correct. It also has to be easy to change. You can't design these at the terminal - that's not theory; we've done it that way and it didn't work!

You need appropriate design for data, APL functions, the logic within functions. APL gives you more time on design, because the coding is cheaper!

The majority of systems are under designed ... looking at the shape of the cost curves, this is a very expensive mistake!



Coding and Testing vs Analysis and Design

In a straw poll, 7 of those present (approx 60 attended) had participated in projects which had (in their view) been over-designed. These all appeared to be defence "cost plus" contracts!

Moral: remember the last time you debugged a set of Xmas-tree lights! The highly coupled version (bulbs in series) is like a bad APL system if there are two dud bulbs you are in for a long haul. Use the principles laid out in "Myers" to write systems which behave like the parallel-wired lights. The system is robust to failure of any one element, and what's more it is quite obvious where the failure has occurred.

Building on Success

Ask ... "Do you know anyone else in this company, or anywhere else, who we should be talking to?" They answer 1/2 the time, but often with 2 - 3 other people.

Tell ... the world about it! Let the BAA spread it about!! APL has been under threat since its inception because:

1. of a shortage of people who know the language well
2. of a lack of vendor commitment. APL is not in SAA! This leaves lots of DP managers round the world wondering whether APL has a future.
3. of ignorance within DP. We tend to show the DP manager what a lousy job he's doing. For some reason this argument doesn't go down too well!

Supporting APL in a Large Organisation

Maurice Jordan (British Airways)
notes by Adrian Smith

BA spend 3% of revenues on information management, expected to be 5% in 2 years time. This leaves enough to fund 3 APLers in a full-time central support role. They have some 30 - 40 mips running APL, with around 10Gb of data, and 30 - 40 APL developers. ("How much is that in Megaflops?!") The corporate info systems "got us through Libya/Chernobyl" when the switchboard was jammed with Americans wanting to cancel.

Main Tasks of the APL Development Centre

- to be BA local experts in APL
- to develop and maintain an APL infrastructure. (2,500 functions). Makes developers more productive by doing all the nasty things at the centre, and doing them well. Examples are: language extensions; cover for APs; inverted file support and so on. Also a variety of documentation tools, code checkers etc.
- to handle archiving, efficiency checking, response-time monitoring and so on.

Minimizing the Maintenance Load

2,500 functions
12 lines each on average
= 30K lines of APL code
= 150K lines in typical 3GL!

Typically the maintenance time is about 1/25th of the development time, which is not half bad. Why?

- a lot of the functions are simple. They are easily described in a 1-line comment.
- they have 1 or 2 arguments. They return 1 result. They have no side effects.

There are some functions which do cause problems.

- Functions using stranded arguments (of many types); functions which use globals to hold results; functions which do lots of things with the screen (everything going to the screen is essentially a side-effect!) With stranding it is often hard to tell which is the function and which are the arguments!!
- over-generalized functions. Don't go for a 200% solution; you throw away all the fast development time, and the result is much harder to change to reflect what actually happens, rather than all the things you thought might, but never did!

The Cost of Poor Quality

Manufacturing industry	5%
Service industry	20%
Software development	>50%

... for every 100 bug fixes that IBM sends out, 58 produce a bug somewhere else! Moral:

- keep bug logs up to date. Put them in the WS or on Email where you can find them again.
- take time to do it right 1st time. "There is never time to do it once properly, but by God you can always find time to do it twice"
- use formal methods for debugging. ("I wonder if the A320 is fully debugged?!") Use APL as a formal notation (e.g. using the set operations) to check the spec, even if the algorithms would be hopeless in practice.

Thoughts on Efficiency

... roughly the equivalent of 1 747 (plane) would have been required if this had not been pushed as an issue.

- response time does matter. To get the "superprogrammer" effect you need small teams, no hierarchy, and good response time.
- monitor overall response, and check workspaces with $\square$ MF. Set this for all functions, write the results to file, and analyse at leisure. Typically the results are sorted by "gross time" of function, and refined in more detail by looking at the functions called by the top 10 worst offenders.
- using $\square$ MF and Fastfns, the total saving on CPU is between 20% and 40%.
- watch the Sharefile response ... a job run automatically every hour checks response, and has been calibrated with user-reaction ("fine" "upset" "hopping mad" etc) to give sensible warnings.

Editorial note ...

... there was lots more, all of it good stuff. I will try and find room in VECTOR 5.2 to continue through MJ's thoughts on training APLers, communication with the APL development teams, and exploring new environments for APL. Watch this space!

PC to MF Communications (or How to Bounce a Volvo)

by David Toop

Introduction

The transfer of data between the mainframe and PC APL environments can be greatly speeded up by using Assembler functions at each end to encode and decode the data. This article describes such a method and describes the type of application for which the programs are used.

The programs described will be made generally available, hopefully through the BAA software library.

Environment

We had a mainframe running APL2 under VM/CMS with AFM Shared File System and numerous PC/ATs, Clones etc. running APL*PLUS/PC Version 6.4. The PCs were fitted with FORTEGRAPH 3279 Terminal Emulator cards (see Adrian Smith VECTOR 4.1 pages 31-33 for a fuller description of the facilities available with this card).

The low level file transfer software was FORTE-NET, a mainframe product available from FORTE (Now DCA - the IRMA people). This software is resident on the mainframe; it starts and controls the data transfer. The software has an 'ASIS' binary transfer facility which performs no translation. It is also extremely fast with transfer speeds in excess of 2 Kilobytes per second being possible.

The Problem

A large amount of data gathered by PCs was to be transmitted to the mainframe for analysis; the transformed data would then be returned to the PC for subsequent display and further manipulation. Component files in excess of 1 Megabyte were not uncommon.

The data in both environments was to be in the form of APL component files. The prime consideration was that after transfer the destination environment should contain an identical component file to that at the source.

The files were of a very simple format - the first component being a character matrix containing a list of object names, the object type and a pointer to the component containing the object. Most of the data would be floating point numeric matrices.

First Attempt

Easy we thought! We will use an ASCII to EBCDIC translation table for characters, format and execute for numerics. It worked but was extremely slow, particularly format on the PC, a 64 x 64 floating point array (quite small for some of the data arrays we were using) taking 90 seconds to format on an AT running at 6MHz. Furthermore, execute on the PC is only able to handle character vectors of less than 32K characters - hardly sufficient. Also with `PP` set to 16 there was a large overhead in extra characters.

Think Again

We then looked at the APL2 supplied functions ATR and RTA (see David Piper VECTOR 3.4 pages 123-128). These assembler functions were extremely fast, using a pattern to perform the conversion of a string of bytes to an APL data object (or vice versa).

It was decided that we would use this pattern as the basis of a standard transfer format for data objects. Variables would be sent as bytes in internal APL2 format.

Three data types were to be considered:

- Characters
- Short Integers (16 Bits)
- Floating Point (8 Bytes)

The data was always to be in simple arrays although it could be complex (in which case we would extend the rank of the object by 1 and the first dimension would contain the real and imaginary parts).

Data Formats: Variables

Figure 1 shows the structure of a typical transfer data object. The first portion is the pattern as defined for ATR and RTA, the pattern delimited from the data by a NULL character (quadAV[quadIO] on both systems).

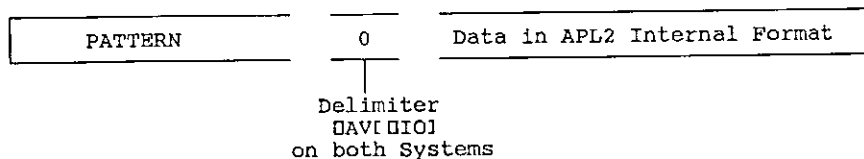


Figure 1: Variable format.

The data is held in APL2/S370 format as are the patterns which are therefore in EBCDIC characters.

Files

The transfer file format is shown in Figure 2. The first 15 bytes are reserved for the directory length, the individual data objects follow and finally a directory of the start positions for each object is at the end of the file.

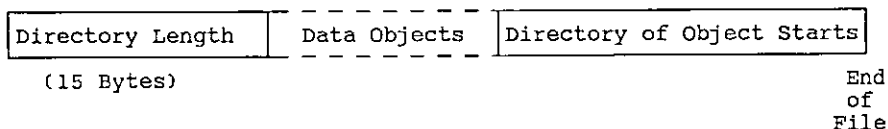


Figure 2: Transfer File Structure

All objects including the length and the directory are encoded as per the standard format.

Data Conversion Strategy

Characters: conversion of characters from ASCII to EBCDIC and vice versa is performed using translation tables. Maintaining our own control of translation enabled us to attempt some enhancements:

- Special APL characters (e.g. Diamond) could be treated separately.
- National Language Characters could be translated accurately.
- We could intelligently convert lower case ALPHAs, graphics and box drawing characters.

Short Integers: these are two bytes on both machines except that the high and low bytes are reversed. The byte reversal necessary for converting from one format to the other is extremely fast.

Floating Point: the conversion technique for floating point numbers was considerably more complex than the other data formats. On the mainframe real numbers are held in 8 bytes as floating point base 16 with a single byte exponent and 7 byte mantissa. On the PC real numbers are also held in 8 bytes but in base 2 with twelve bits exponent and 52 bits mantissa. Furthermore, the byte order of the two formats is reversed.

It is interesting to note that because of the normalisation process there is very little difference in the accuracy of the two systems, although the PC can represent a larger number range.

We evolved some quite complex algorithms involving byte reversal, shifts and bit twiddling to convert between the two formats. APL was used to prototype and test these algorithms which proved to be particularly useful in some of the boundary conditions. At this point I had hoped to show our prototype function, but it appears to have gone the route of all good prototypes and been consigned to the rubbish bin! However, we used page 224 of Gilman and Rose as a starting point for the mainframe to PC algorithms.

Using the Transfer Programs

The user is provided with four top-level functions, two for the mainframe and two for the PC. These functions are named FVSEND and FVRECV. To illustrate the mechanism of the transfer the chain of events which occurs when sending a file to the Mainframe is as follows:

- The user first establishes an APL session on both of the machines and switches to the PC APL session.
- He types FVSEND 'FILENAME' where FILENAME is the name of the file he wishes to send to the mainframe. The FVSEND(PC) function then builds the native Transmit file on the PC using the machine code function outlined above.
- He then 'Hot Keys' to the APL2 session and types: FVRECV 'FILENAME' Where FILENAME is the destination filename. The FVRECV(MF) then invokes the forte transfer software using AP100. The transfer file (now resident on the mainframe) is processed using the file facilities of AP110 and the supplied assembler function RTA.

The user finishes with an APL component file identical to the one on the PC.

Speeds

As a comparison of speed, our test variable of 64 x 64 floating point numbers takes approximately 0.5 seconds to convert (which is a factor of 180 improvement over format). The conversion of two byte integers is improved by a factor of 15.

The major bottleneck in the whole system is now the transmission time which accounts for approximately 95% of the total transfer time. Overall it takes approximately 10 minutes to transfer a 1 Megabyte component file between the two environments.

Typical Application: Vibration Analysis

The application which I have chosen to demonstrate the use of this facility is in the analysis of the vibration in cars. Data from a test car or component is gathered from

either road measurements or a test rig. The data acquisition is controlled by a PC which allows excellent facilities for the control of the measuring instruments and interactive examination of the data.

An extremely large amount of data can be obtained from a single measurement. Figure 3 is a typical display from the data acquisition facilities:

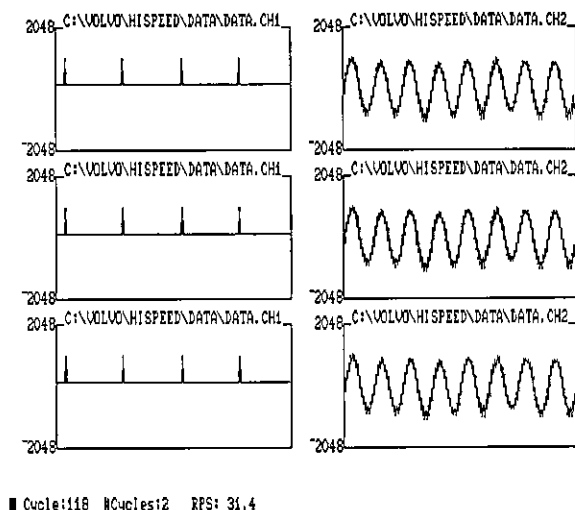


Figure 3: Typical Data Collection and Manipulation Screen

When the operator is satisfied with the data, it can be quickly transferred to the mainframe where it can be processed using the greater speed, workspace size and facilities of APL2. Typically such operations involve the identification of vibration modes, frequencies and amplitudes. The transformed data can then be transmitted back to the PC.

Other programs can now use the superior real time graphics and interactive response of the PC to animate the test results producing a display such as:

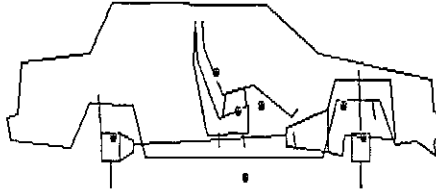


Figure 4: Animation Type Display on the PC,
produced after Mainframe Calculations.

Conclusions

The use of both mainframe and PC for the tasks at which they are best has proved to be very effective, and whilst the system is still undergoing development the results from users have been very encouraging.

The use of APL as a 'glue' to hold both systems together and provide a common user interface has proved to be one of the major benefits of the system.

The calling of assembler functions from both environments to do highly specialised yet well defined tasks has not hampered the natural flow of APL programs. In many cases, it has meant the difference between success and failure of the application. Furthermore, identifying small specific tasks to be written in assembler does not add significantly to the overall development cost, especially when APL can be used to prototype these modules.

APL CONSULTANTS

LONDON & READING

Account Managers	(6 years+)	to	25K
Senior Consultants	(4-6 years)	to	21K
Consultants	(2-4 years)	to	17K
Junior Consultants	(1-2 years)	to	13K

Are your APL skills and potential being recognised and rewarded?

Cocking & Drury consultants have been implementing successful decision support applications for 10 years, with clients who appreciate the productivity benefits of APL.

In our professional team you will experience a range of APL environments - APL*Plus, VSAPL, APL2 and Unix, on both mainframes and micros. You will also be developing systems which, increasingly, need to interface with non-APL Information Centre products.

Of course as the leading APL consultancy, in a rapidly expanding market, we offer a rewarding career with first class benefits - profit sharing, free health insurance, and a non-contributory pension.



For further details call Ralph Wilson on 0734 588835

COCKING & DRURY LTD.

155 Friar Street, Reading, RG1 1HE

GENERAL ARTICLES

This section of VECTOR is oriented towards readers who may neither know APL, nor be interested in learning it. However we hope you are curious about how, under the right conditions, such impressive results can emerge so quickly from APL programmers.

A Commentary on APL Development

by Kenneth E. Iverson

Because "A Dictionary of APL" [1] was designed for concise and convenient reference, it is important that it be supplemented by other treatments of the language, concerning matters such as style and literacy [2], discussion of the means used to eradicate anomalies that have crept into the language [3], illustrations of analytic uses of the language [4], and explorations of the uses of particular verbs, adverbs and conjunctions [5].

The present paper treats the rationale for many of the less familiar facilities defined in the dictionary, rectifies the definition of rank, provides simpler definitions for the conjunctions defined by cup and cap, and discusses the matter of treating sets and set-like operations. The order of treatment parallels that of the dictionary.

Although some of the facilities in APL have been introduced in response to demands from programmers, most have been motivated by knowledge of their utility in some branch of mathematics. Moreover, they have invariably been generalized from the special cases on which they were based.

For example, reduction, inner product, and outer product were not demanded by programmers and are still not demanded by programmers in other languages. Moreover, although they were suggested by the sigma notation, by the matrix product of linear algebra, and by the outer product of tensor analysis, it is the generalizations of these ideas that has made them fruitful in general programming.

The exploitation of these facilities has been a long process of discovery by programmers, discovery often impeded by the fact that the implementation of a given construct might well remain too inefficient for serious use until the discovery of its utility motivated more efficient implementation. Reductions and inner products on boolean functions provide a case in point.

The motivation for the newer verbs and adverbs defined in the dictionary remains largely mathematical, and the process of discovery and exploitation continues. For example, programmers have taken to the rank adverb, and its efficiency in the Sharp APL implementation has already been sharply improved by embodying its treatment within individual primitive verbs.

If the development of APL is to benefit fully from experience in use, it is essential that the process be slow and conservative. Early examples of this conservative approach (such as the adoption of notation for the circular functions) occur in Section 4 of "Evolution of APL" [6], and "The Story of o" [7]. A more recent example will now be discussed.

Although Section IIC of [1] states that adverbs and conjunctions may produce results of any class, none of those defined in the dictionary produce anything other than verbs. The potential of adverbs producing conjunctions was recognized as early as 1978 [8], but was avoided by introducing a collection of conjunctions, as in $f \nabla g \omega \leftrightarrow (f\omega) + (g\omega)$, and $p \nabla q \omega \leftrightarrow (p\omega) \wedge (q\omega)$. Somewhat later, Arthur Whitney suggested the conjunction $\tilde{\wedge}$ [9], which provided a more general facility for this purpose, but which still avoided the introduction of constructs that produce adverbs or conjunctions.

The present paper proposes an adverb (called *yoke*) that produces a conjunction; it supersedes several facilities proposed in the dictionary, the *with conjunction* ($\tilde{\wedge}$), the *union* and *intersection* ($\cup$), and the concomitant notion of *combining rank*.

Box (<). The importance of box rests on the fact that it provides a scalar encoding of any noun to which it is applied. Because the result is a scalar, arrays of boxed elements can be handled conveniently by existing selection and structural functions, and because the inverse *open* ($>$) has rank 0, any function f can be handled with the convenience of a scalar function by using the simply-related function $f^{<}$. For example, if $f := \nabla(-1 \rho a) \phi a \cdot \times \omega$, $0 \times 1 + a$, then $\phi 1 f \phi 2$ is the coefficient vector of the polynomial equal to the product of polynomials with coefficients $\phi 1$ and $\phi 2$, and $f^{<} / bc$ produces the boxed coefficient of the product of a list bc of boxed coefficients.

The convenience of scalar encodings can be illustrated by the implicit scalar encodings used for numeric values: the increasingly complex lists 12 and 12.34 and 12.34/45.6 represent integers, rationals, and complex numbers, but in APL only the scalar encodings are relevant and the details of the representations can, for most purposes, be ignored.

The analogue of box used in other dialects (where it is called *enclose*, and denoted by $\langle$), differs in two significant respects:

1. $\langle a$ differs from $\langle a$ for any argument a ; whereas $\langle a$ differs from $\langle a$ only if a is not a simple scalar. For example, if $a := 'I'$, then $\langle a$ is a one-letter word, $\langle \langle a$ is a one-word sentence, and $\langle \langle \langle a$ is a one-sentence paragraph; each distinct (i.e. distinguishable), whereas a and $\langle a$ and $\langle \langle a$ and $\langle \langle \langle a$ are not distinct.
2. In other dialects, a certain list of functions (usually characterized as "mathematical") apply to enclosed numeric arguments as if they were not enclosed; thus $\langle 3 \ 4 \rangle + \langle 5 \ 6 \rangle$ is equivalent to $\langle 3 \ 4 \rangle + \langle \langle 5 \ 6 \rangle$. The use of such an explicit list raises questions concerning the characterization of any new functions, user defined, or derived functions. For example, would the defined function $f: \alpha \mapsto \omega$ qualify? It is mathematical in the sense that $3 \ f \ 4$ produces the result 3.25, and in that f / i evaluates the continued fraction represented by the integers i .

Cycle, Mix, and Mix Index ($\leq \geq$). These functions all concern *permutations*, and their importance rests on the importance of permutations in mathematics, and on the importance of sorting (a particular case of permutation) in data processing.

Any reordering or *permutation* of the vector ω is called a *permutation vector*; it can be used with indexing to produce a corresponding ordering of the major cells of an array. For example, if $p := 0 \ 1 \ 2$, then $p \{ 'rstu' \}$ yields 'rust'.

Since $\{$ permits negative indexing (selecting from the tail end), an argument such as $p := 0 \ -1 \ 1 \ 2$ can produce the same permutation as p .

In the dictionary, the term *mix* is introduced to characterize such a vector, but since any mix q can be reduced to the equivalent permutation vector by the expression $(\rho q) | q$, we will here simplify the discussion by speaking only of permutation vectors.

The permutation $s := 4 \ 5 \ 2 \ 1 \ 0 \ 3$ used for illustration in the dictionary is said to have cycles 5 3 1 and 4 0 and 2 because when used in indexing it "cycles" the elements in each group of positions, but never intermixes different groups. For example:

```

a ← 'abcdef'
b ← s{a ↔ efabad
c ← s{b ↔ adcfab
d ← s{c ↔ ebcdaf
e ← s{d ↔ afcbad
f ← s{e ↔ edofab
g ← s{f ↔ abcdef

      2
      0  4
      1  3  5

```

Any permutation can be defined by specifying its cycles. Since any rotation of a cycle (such as 105 3 1 or 205 3 1) represents the same reordering, we can adopt a "standard" form in which the largest element is in the leading position (as was done in the example, with 5 3

1 and 4 0 and 2). Finally, if these cycles are put in increasing order according to their leading elements and then catenated to form a single vector, $\alpha = 2\ 4\ 0\ 5\ 3\ 1$, the breaks between the successive cycles can be determined by the expression $\alpha = \Gamma \backslash \alpha$, and α is called the cycle representation of the permutation α .

The cycle representation of a permutation s is given by the cycle function $(\alpha \rightarrow \alpha s)$, and the corresponding permutation is obtained from the cycle representation by the mix function $(s \rightarrow \alpha s)$.

The cycle representation is useful in identifying distinct groups that are not intermixed by a permutation, and in finding the "power" of a permutation (the number of times it must be applied to restore the original order); this is given by the least common multiple of the lengths of the cycles, that is, by $\wedge$, $(\alpha = \Gamma \backslash \alpha) 1 \dot{\vee} \rho \alpha$.

Moreover, since the function $\geq$ treats any "incomplete" argument α as if the missing elements of $\Gamma \backslash \alpha$ are single cycles (that do not move), then $\geq 5\ 3\ 1$ produces a permutation that cycles the elements 5, 3, and 1; and $\geq 4\ 1$ interchanges elements 4 and 1. Such commonly useful permutations are otherwise relatively awkward to specify.

All the permutations of order n can be arranged in a $!n$ by n table T in "lexical order", that is, the rows evaluated as numbers in a base n system ($n \geq 1$ T) are in increasing order. For example, if $n=3$, then T is the table:

```
0 1 2
0 2 1
1 0 2
1 2 0
2 0 1
2 1 0
```

A given permutation p can therefore be identified by its index in such a table; the expression $]p$ yields the index of p . Consequently, $r \rightarrow (n - 1 \rightarrow \rho p) \Gamma]p$ yields the reduced representation of p and $2 \mid + / r$ yields its parity, both as defined in [4].

The dyadic expression $\hat{z}] \alpha$ yields that permutation of the major cells of α produced by the permutation of order $1 \mid \rho \alpha$ whose mix index is $\hat{z}$. For example, if $\alpha = 'abcd'$, then $1] \alpha$ reverses the last two elements of α , and $5] \alpha$ reverses the last three, and $4] \alpha$ rotates the last three.

Finally, $\hat{z}] 1 \mid n$ produces row $\hat{z}$ of the table of permutations of order n , and $(\hat{z}] 1 \mid n)] 1 \mid n$ produces the entire table.

Nub in, $Nubsieve$, and $Nub (= \neq \neq 1)$. The expression $\neq \alpha$ produces a boolean result that defines a classification or set, the 1's in the result identifying the set of distinct major cells of α . The expression $(\neq \alpha) \neq \alpha$ produces a selection based on the classification $\neq \alpha$, yielding the nub of α . The expression 1α is equivalent to $(\neq \alpha) \neq \alpha$, and therefore produces a combined classification and selection. For example:

α	$\neq \alpha$	1α
ABC	1 0 1	ABC
ABC		DEF
DEF		

Chapter 2 of [10] may be consulted for a discussion of classifications and sets, where they are introduced by the following statement of their role in mathematics and data processing:

It is often necessary to separate a collection of objects into several classes, and then perform some operation upon each of the classes. The operation performed is often trivial compared to the complexity of the classification procedure itself, and classification is therefore an important matter. Indeed, most computation involves some classification, even though the classification process may be implicit rather than explicit.

Further examples of the use of classification may be found in the treatment of graphs and barcharts in Chapter 4 of [11], and in the treatment of directed graphs and trees in Chapter 6 of [10].

Two further examples of the use of nub , and of classifications based upon it, will now be discussed:

```
an+3 1 4 1 3 2
tan
3 1 4 2
=an
1 0 0 0 1 0
0 1 0 1 0 0
0 0 1 0 0 0
0 0 0 0 0 1
```

The last result is an "auto-classification" of an produced by the $Nub-in$ function defined by $(tan) \circ . = an$. If an is a list of account numbers, and if

```
dep+35 40 21 52 25 17
```

then the expression $(=an) \circ . \times dep$ may be seen to produce a summary of the deposits to individual accounts.

The second example concerns the use of the matrix $=an$ as a post-multiplier rather than as a pre-multiplier. If an is a list of arguments to a function F that is expensive to compute, then it is desirable to apply F only to the distinct elements (as in $F \mid an$), and then distribute the results for all arguments, as in $(F \mid an) \circ . \times =an$. An expression of this form is used in [4] to derive Newton's symmetric functions for determining the coefficients of a polynomial from its roots.

Less ($\sim$). Because *less* applies to the major cells of its two array arguments as a "set-like" function (differences), its further discussion will be deferred to the final section.

Nand, Nor, Right, and Left ($\wedge$, $\vee$, $\rightarrow$, $\leftarrow$). Boolean functions (that are defined on the boolean arguments 0 and 1, and produce boolean results) are very generally useful. Because there are four possible pairs of two arguments (shown in the columns of the table 2 2T14 for each of which two different results are possible, there are 16 (that is, 2×4) distinct dyadic boolean functions. Of these, two are constants, one is the negation of the left argument, one is the negation of the right argument, and the remaining twelve are provided in APL. The relations $<$, $\leq$, etc. provide six, the *or* and *and* provide two, and the set is completed by *nand* and *nor*, and by *right* and *left* (or, more precisely, by $\rightarrow 0$ and $\rightarrow 1$ and $\leftarrow 0$ and $\leftarrow 1$).

Nand and *nor* also have the interesting property that either can be used alone to produce all of the other boolean functions. The four missing dyadic booleans can be expressed as $0 \rightarrow 0$ and $1 \rightarrow 0$ and $\rightarrow 1 \leftarrow$ and $\rightarrow 1 \leftarrow$.

Programmers largely rely on the boolean functions *or*, *and*, and *nor*, but it should be noted that others of the set can provide briefer, and perhaps clearer, statements:

For	Substitute	and read as
$\sim a \vee b$	$a \vee b$	(neither) <i>a</i> nor <i>b</i>
$a \wedge \sim b$	$a > b$	<i>a</i> exceeds <i>b</i>
$(\sim a) \vee b$	$a \leq b$	<i>a</i> implies <i>b</i>
$(a \wedge \sim b) \vee (\sim a) \wedge b$	$a \neq b$	<i>a</i> differs from <i>b</i>

Some uses of left and right are discussed in Section III of the dictionary. A few others are worth noting:

- The expressions $\rightarrow m$ and $\leftarrow m$ replicate the leading row and the leading column of a matrix *m*.
- Expressions such as $s \times .5 \leftarrow s \div x \times 2$ and $s \div \rho x \leftarrow s \div x$ may be used instead of $(\div x \times 2) \times .5$ and $(\div x) \div \rho x$; they tend to give prominence to the overall functions (square root and division of *s*) and to subordinate the detail involved in computing their arguments.
- The foregoing use of $\leftarrow$ may be read as "where", as in "square root of *s*, where *s* is the sum of squares of *x*". The usage may be extended to reduce the use of parentheses to any desired degree.

Words (1) The definition of the monad $\perp$ formalizes an essential process that has long been ill-defined and divergent in dialects. It introduces new classes of names that might be used in various ways, as in 2×3 as a constant name for the rational number $2/3$.

A function that parses a sentence *s* and shows its execution piece-by-piece can be very useful as a teaching or de-bugging tool; use of the expression $\perp s$ could significantly simplify its definition.

The function $\perp$ might well be extended to a new dyadic case in which a boxed left argument would specify the parameter *p* used in the dictionary definition of the monadic case. This would provide considerable flexibility in the rules for word formation.

Circle (O). The definition of the dyad *kow* has been extended to include two existing primitives (13Ow for the identity function, and 14Ow for the exponential), together with appropriate inverses for the cases 13 and 14. The motivation rests on the frequent use, in applied mathematics, of phrases such as $\omega \times (\star \omega) \times (1Ow)$, which can now be written in the form $\star / kOw \rightarrow k + 13 \ 14 \ 1$.

From, All, Merge ($\{ \}$). The indexing provided by *from* can serve as a convenient replacement for all uses of the anomalous bracket-semicolon indexing. Moreover, it provides abbreviated indexing (allowing the selection of cells by specifying only the "outer" indices), negative indexing (allowing convenient selection from the tail end of an axis), and complementary indexing (allowing specification of "all but" an indicated set of indices along any axis). Hui [5] provides examples of uses of *from* as well as discussion of the related *merge* adverb.

It should be noted that a definition such as $f \leftarrow 1 \ 2 \ 2''$ cannot be made without negative indexing, because the shape of the argument to which *f* might eventually be applied is not known.

In the dictionary it is remarked that the monad *all* can be used to produce cartesian product, as in $\{a \times c\}$. It should be noted that the cartesian product cannot be used conveniently to produce the effect of *all* when applied to lists of more than two elements, since, for example, $a \ cp \ b \ cp \ c$ would produce items consisting of a two-element list made up of an item from *a* together with a boxed two-element list of items from *b* and *c*, rather than three-element lists of items from *a*, *b*, and *c*.

Ply (.). In a still unpublished paper, E.E. McDonnell has suggested the adoption of a dyadic case to be defined in terms of the monadic case as follows:

$$\alpha \ u, n \ \omega \mapsto \rightarrow (n \neq 0) \{ \alpha'' \cdot (, :) (u'' \cdot /) . n \ \alpha, \tilde{\omega} \omega$$

For example, $0 \mapsto .n \ 1$ gives the n th fibonacci number, and $1 \mapsto \nabla' \omega \times 1 + \omega \alpha' . n \ 1$ gives $!n$. The definition applies only for non-negative values of n .

The definition for the monad $u, n \omega$ and the dyad $\alpha u, n \omega$ should both be extended to non-scalar arguments n , the outer shape of the result being ρn . For example, $0 \mapsto .(17)1$ yields $(0 \ 1 \ 1 \ 2 \ 3 \ 5 \ 8)$, and $2'' \cdot \cdot (15)1$ is $1 \ 2 \ 4 \ 8 \ 16$.

Union, Intersection ($\cup$ $\cap$). These conjunctions, and the concomitant notion of combining rank, are superseded by the adverb *yoke* defined below.

Yoke (:). The *yoke* defined here is an adverb that produces a conjunction. The symbol suggested for it ($:$) is already in use for the custom adverb, and the semicolon is now suggested for that role.

If f, g , and h are verbs, then:

$$\alpha \ f \ g : h \ \omega \mapsto (f \alpha) \ g \ (h \omega)$$

$$f \ g : h \ \omega \mapsto (f \omega) \ g \ (h \omega)$$

The ranks of $f \ g : h$ are the minimum of the ranks of f and h .

For functions with scalar results, the expression $f \tau : g$ provides what was provided by $f \cup g$; more generally, one may use $f, \tau : g \tau : h \tau : i$.

The equivalents of $f \cup g$ and $f \times g$ in mathematics may now be provided by $f \tau : g$ and $f \times g$. Moreover, $p \wedge : q$, and $p \vee : q$ and $p > : q$ provide propositions that define the intersection, union, and difference of the sets defined by the propositions p and q . Finally, the effect of *with* (a variant of *til*) is given by $\vdash g : h$; in particular, $(\vdash : \vdash) / \alpha$ gives the continued fraction represented by the list α .

Sets. In mathematics, sets are commonly defined by notation of the form:

$$S = \{x | P x\} \quad T = \{x | Q x\}$$

where P and Q are propositions, that is, functions whose range comprises two elements, sometimes called *true* and *false*, but in APL more conveniently treated as 0 and 1. For example, if $P: (\omega = 1) \wedge (\omega > 0) \wedge (\omega < 100)$, then S is the set of integers lying strictly between 0 and 100.

Since a set such as S can only be used to answer the question of whether an argument y belongs to S (expressed as $y \in S$), the set definition appears to add nothing to the

proposition used in the definition; that is, the expression $P y$ provides the same result.

However, expressions for the union and intersection of sets are more convenient than the corresponding expressions in terms of the defining propositions, that is, $S \cup T$ instead of $(P \omega) \vee (Q \omega)$, and $S \cap T$ instead of $(P \omega) \wedge (Q \omega)$. What is required to make the use of propositions more palatable (and to exploit the use of booleans other than the *or* and *and* used in union and intersection) are suitable conjunctions to link the propositions.

This need is met by conjunctions of the form $b :$, where b is a boolean function. For example, $P \vee : Q$ produces the proposition analogous to the union of sets S and T , and $P \wedge : Q$ produces intersection. Moreover, $P > : Q$ provides ordinary set difference, and $P \neq : Q$ provides symmetric set difference. Finally, a "family of propositions" such as $H \mapsto P, : Q, : R$ can be used in a variety of ways such as direct evaluation, as in $H \omega$, or in an intersection of sets and their complements, as in $(b'' (\wedge :)) \tilde{\omega} H$, where the propositions corresponding to zeros in the parameter b are negated.

Set-like operations. A proposition defining a set is sometimes itself defined in terms of an exhaustive list of the elements belonging to the set, as in:

$$VOWELS: \vee / \omega \omega = 'AEIOU'$$

The union, intersection, and difference of two sets can then be represented by analogous operations on the corresponding lists, and these operations on lists are often referred to by the names appropriate to the operations on sets.

Thus, if a and b are the lists representing the sets (propositions) A and B , then $a \tau b$ (or $\vdash a \tau b$ to remove duplicate elements) corresponds to union ($A \vee B$); $a \sim b$ to difference ($A \setminus B$); and $a \wedge \sim b$ to intersection ($A \cap B$).

It should be noted that $a \tau b$ and $a \sim b$ concern the major cells of the arguments. For example:

a	b	$a \tau b$
mno	stu	mno
pqr	vwx	pqr
stu		stu
		stu
		vwx
$a \sim b$	$a \wedge \sim b$	
mno	stu	
pqr		

Finally, major cells relative to any axis k can be treated by expressions of the form $a \sim \tilde{\omega} (k \tilde{\omega} \sim) b$.

Acknowledgements

I am indebted to John Sansom and other careful readers of the dictionary for raising many of the questions addressed herein, and to Roland Pesch, Henri Schueler and E.E. McDonnell for helping provide some of the answers.

References

- [1] Iverson, K.E., "A Dictionary of APL", APL Quote Quad, Vol 18, No.1, September 1987.
- [2] Berry, M. and Pesch, R., "Style and Literacy in APL", Proceedings APL86, July 1986, APL Quote Quad, Vol 16, No.4.
- [3] Iverson, K.E., "APL87", Proceedings APL87, APL Quote Quad, Vol 17, No.4.
- [4] Iverson, K.E., "Notation as a Tool of Thought", CACM 23, August 1980.
- [5] Hui, Roger, "Some uses of { and }", Proceedings APL87.
- [6] Falkoff, A.D. and Iverson, K.E., "The Evolution of APL".
- [7] McDonnell, E.E., "The Story of ", APL Quote Quad, Vol 13, No.1, September 1982.
- [8] Iverson, K.E., "Operators and Functions", IBM Research Report RC 7091, 1978.
- [9] Iverson, K.E. and Whitney, A.T., "Practical Uses of a Model of APL", APL Quote Quad, Vol 13, No.1, September 1982.
- [10] Iversen, K.E., "Applied Mathematics for Programmers, I.P. Sharp Associates.
- [11] Iverson, K.E., "Mathematics and Programming", I.P. Sharp Associates.

Simple Icon Design for EGA and VGA

by Adrian Smith

Introduction

Some little time ago (see VECTOR 4.1 page 34) I was lucky enough to acquire a Hercules-Plus card for my monochrome system. At the time, this was supplied without an APL font (I gather from the Hercules advertising that they have now added APL to their list), so I was faced with the prospect of designing the entire APL character set from scratch. Fortunately it occurred to me that the APLCHAR.COM file supplied for APL*PLUS/PC must somehow define the characters; I duly reverse engineered it and thus built a couple of very nice variants of the Hercules-supplied fonts for use with APL.

Of course once one had had a taste of font design, one was hooked! I had the excellent excuse that I needed a variety of funny symbols for valves, taps etc for a plant information system I was building, and anyway I had promised Richard a realistic mouse to manoeuvre round the simple maze I had written him! The functions listed below are all you need to handle the character definition for the EGA and VGA; I could put them on the Software library, but it hardly seems worth the bother! If you would like the code for the Herc+, please let me know - it is almost identical to the VGA as fonts are interchangeable between the two devices.

HELP

ΔFILE←'APLCHAR.COM'	A SET UP EGA FILE NAME
ΔHFILE←'APLSTD.FNT'	A SET UP HERC+ FILE NAME
ΔVFILE←'VGACHAR.COM'	A SET UP VGA FILE NAME
ZZZ←GETCH 252	A GET GIVEN CHAR DEFINITION
ZZZ← HGETCH 'Δ'	A ... HERCULES EQUIVALENT
ZZZ← VGETCH 'A'	A ... VGA EQUIVALENT
ZZZ PUTCH 200	A REPLACE 200TH EGA CHAR
ZZZ HPUTCH 200	A REPLACE 200TH HERC+ CHAR
ZZZ VPUTCH 200	A REPLACE 200TH VGA CHAR
EDCH 200	A EDIT 200TH EGA CHAR
HEDCH 200	A EDIT 200TH HERCULES CHAR
VEDCH 200	A EDIT 200TH VGA CHAR

How are Fonts Defined?

Each character you see on the screen is built up from a pattern of dots; the number of dots determines the quality of the final image. The EGA system uses a matrix of 14 rows by 8

columns, i.e. the solid block is effectively 14 8p1. To see how APLCHAR.COM defines a particular character you need to `DNREAD` the right part of the file, then do a bit of base-2 encoding to make up a 0/1 matrix of the right shape:

```

R←GETCH N;ZZ
-----

[1]  APULL IN FROM ΔFILE ('APLCHAR.COM') AND THEN DECODE CHAR N
[2]  ⍺(0=0=1↑0ρN)/'N←ΔAV\N'
[3]  DNUNTIE DNNUMS ⋄ ΔFILE UNTIE ^2
[4]  ALOSE 1ST 44 BYTES; REMAINDER IS IN 14-BYTE CHUNKS ...
[5]  ZZ←DNREAD ^2 82 ,14,44+(N-1)×14 ⋄ DNUNTIE ^2
[6]  ANOW LOOK UP AND BREAK INTO BINARY FORM ....
[7]  R←⍺(8ρ2)τ^1+ΔAV\ZZ ⋄ R+'.'[1+R]
      ▼

```

Things are a wee bit harder for the VGA; here each character occupies a 16 by 9 grid, but the 9th column is set for you by the hardware. It is always zero for text characters (i.e. there is always space between the characters even if you set up a solid block), and it always repeats column-8 for the line-drawing set. This means that if you want a multiple character Icon (such as my mouse which uses 4 characters from nose to tail) you must use `ΔAV[180 TO 220]` for your work. This is rather a nuisance, as there are only so many spare line drawing characters to attack; if you aren't careful you will find your nice window borders suddenly acquire nose and whiskers!

Here is the corresponding code to examine VGACHAR.COM:

```

R←VGETCH N;ZZ
-----
[1]  APULL IN FROM ΔVFILE ('VGACHAR.COM') IN VGA FORMAT,
[2]  RAND THEN DECODE CHAR N
[3]  ±(0=0=1↑0ρN)/'N←ΔV\N'
[4]  ONUNTIE ONNUMS ◊ ΔVFILE ONTIE ~2
[5]  AFILE IS IN 16-BYTE CHUNKS ...
[6]  ZZ←ONREAD ~2 82 .16,47+(N-1)×16 ◊ ONUNTIE ~2
[7]  ANOW LOOK UP AND BREAK INTO BINARY FORM ....
[8]  R←Q(8ρ2)τ~1+ΔV\ZZ ◊ R←'.*'[1+R]
    v

```

Now you are ready to start changing things! First of all you should take a copy of the standard character set with a new name! Then you simply pull it into the workspace, hit it with EDIT, and reverse the process to patch it back into the original file. Here are the functions for the EGA system:

```

R←EDCH N;CHAR;INS;SHAPE
-----
[1]  AGET CHAR FROM EGA DEFINITION, EDIT AND REPLACE
[2]  CHAR←GETCH N ◊ SHAPE←ρCHAR
[3]  AENSURE KEYBD IS IN REPLACE MODE ...
[4]  INS←0 OPPOKE 149
[5]  EDIT 'CHAR' ◊ →(Rewrite,Quit)IF 20=OPPEEK 258
[6]  Rewrite:CHAR←SHAPE↑CHAR ◊ R←CHAR PUTCH N ◊ →Rest
[7]  Quit:R←'Changes abandoned ..... ready'
[8]  Rest:INS←INS OPPOKE 149
    v

```

R←MAT PUTCH N;ZZ

```
[1]  AReplace Nth char in ΔFILE ('APLCHAR.COM') in EGA format
[2]  Awith MAT (suitably encoded)

[3]  ⚡(0=0=1↑0ρN)/'N←ΔAVN'

[4]  ADefault to EGA set if nothing passed

[5]  ⚡(0=0NC 'MAT')/'MAT←ΔCH[N;;]'
[6]  0NUNTIE 0NUNNUMS ⚡ ΔFILE 0NTIE ~2

[7]  AFILE IS IN 14-BYTE CHUNKS AFTER 44-BYTE HEADER ...

[8]  ΔAV[1+CODECH MAT]0NREPLACE ~2,44+(N-1)×14 ⚡ 0NUNTIE ~2
[9]  R←'Char loaded .... ',ΔAV[N]
▽
```

R←CODECH MAT

```
[1]  A CODE EDITABLE CHARACTER INTO INTEGER FORM ...
[2]  R←121&MATε'⚡*'
▽
```

.... and for the VGA you do much the same, only please note that it is pointless changing column-9!

R←VEDCH N;CHAR;INS;SHAPE

```
[1]  AGET CHAR FROM VGA DEFINITION, EDIT AND REPLACE

[2]  CHAR←VGETCH N ⚡ SHAPE+ρCHAR

[3]  AENSURE KEYBD IS IN REPLACE MODE ...

[4]  INS←0 0POKE 149
[5]  0EDIT 'CHAR' ⚡ →(Rewrite,Quit)IF 20=0PEEK 258
[6]  Rewrite:CHAR←SHAPE↑CHAR ⚡ R←CHAR VPUTCH N ⚡ →Rest
[7]  Quit:R←'Changes abandoned ..... ready'
[8]  Rest:INS←INS 0POKE 149
▽
```

```
R←MAT VPUTCH N;ZZ
```

```
-----
```

```
[1]  AReplace Nth char in ΔVFILE in VGA format
[2]  Awith MAT (suitably encoded)

[3]  ⚡(0=0=1↑0ρN)/'N←ΔAViN'

[4]  ADefault to EGA set if nothing passed

[5]  ⚡(0=ΔNC 'MAT')/'MAT+ΔCH[N;;]'
[6]  ΔNUNTIE ΔNNUMS ∘ ΔVFILE ΔNTIE ~2

[7]  AFILE IS IN 16-BYTE CHUNKS ...

[8]  ΔAV[1+16↑CODECH MAT]ΔNREPLACE ~2,47+(N-1)×16 ∘ ΔNUNTIE ~2
[9]  R←'Char loaded .... ',ΔAV[N]
```

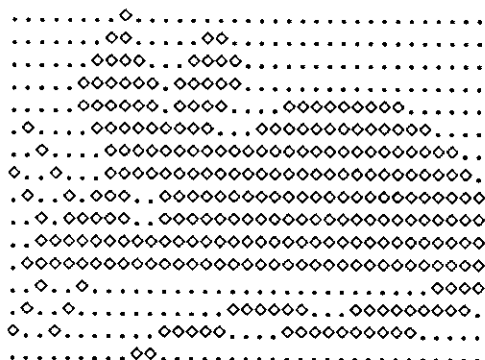
```
▽
```

If you want to do multiple-character icons (such as the mouse in the example below) you can quickly hack together the code to get and replace the appropriate block of ΔAV.

Examples of Use

The basic reason for using redefined text characters rather than genuine graphics is speed; text is at least an order of magnitude faster. For example, to put an entire plant diagram on screen takes around 1/5th of a second in text mode as against some 20 sec as a graphic! Icons also benefit from being small. I wanted a cursor-selectable 'mailbox' on every screen of my system; by redrawing just 2 characters I made a very respectable pillarbox, and two more gave me the 'signpost' to go alongside it.

You must also use text if you want true animation. The mouse keeps up very comfortably with the cursor repeat (on the PS/2 you can even use a 'fast cursor' poke) which is essential if the game is to be any sort of test of the user's dexterity!



There is only one snag I can think of; if you want hardcopy you also have to do the download characters for your Epson, which is tedious in the extreme as the character box is now 11 by 9. So far, I haven't got round to it, which is why there are no illustrations of the mouse in action!

Oh, and do be a little careful in your choice of characters. I used quad and quad-quote to make my pillarbox, as they look roughly right in their natural state. Any code with lots of quad-functions in it looks pretty peculiar when you edit it!

Auxiliary Processor 727: the Design of an APL*Plus/TSO to DB2 Interface

by Allan Gay

This article introduces AP727, an auxiliary processor presently being developed by Cocking & Drury Limited as a means of interfacing STSC Inc.'s APL*Plus/TSO with IBM's DB2. The salient points of the AP727 design are sketched and some technical aspects are described.

Users of other relational database systems and other operating systems should not be deterred by the references to DB2 and TSO. In essence, AP727 is simply a means of interfacing mainframe APL*Plus and a relational database system by means of the Structured Query Language (SQL). If a given package supports Dynamic SQL and runs under MVS or VM, a suitable variant of AP727 is likely to be feasible.

Indeed, to facilitate the production of such AP727 variants, AP727 has been carefully designed to concentrate operating system, Dynamic SQL and interface specifics into dedicated repluggable modules.

As an interface between STSC Inc's APL*Plus/TSO and IBM's DB2 relational database system, AP727 is functionally equivalent to the MVS/TSO implementation of APL2's AP127. Compatibility with AP127 has been a client requirement from the outset in order to ensure that users have the option to migrate between these two APLs.

Compatibility means that AP727 has the same repertoire of services as AP127 and that all external interfaces are close APL*Plus equivalents of their APL2 counterparts. See IBM's SH20-9217 "APL2 Programming: Using Structured Query Language (SQL)" for details of AP127 repertoire and interfaces.

Outline Description

AP727 is a demand-responsive auxiliary processor designed to run in the APL*Plus/TSO environment under MVS/370 or MVS-XA. It is re-entrant and so is capable of being installed as a resident AP if desired. Extended addressing is not used.

AP727 supports up to forty concurrently-shared variables. However, only one variable may be active at a given time. If multiple variables are offered, AP727 matches the offers provided that the shared variable quota is not exceeded. The first variable to be specified (set) by the APL*Plus user becomes the active shared variable and remains so until the user retracts it or AP727 signs off. If the user retracts the active shared variable, the next shared variable to be set by the user becomes the active shared variable.

The single active shared variable is operated bi-directionally both to receive commands and data from the APL*Plus user and to return results to the APL*Plus user. User commands are formed as simple or nested objects. The first or only simple object in the user input is a simple character vector which constitutes an AP727 operation code. Certain operation codes permit or require the provision of additional objects such as statement names, SQL statements, substitution data and so on.

Similarly, AP727 results are formed as simple or nested objects. The first or only simple object is always a 5-element integer returncode vector documenting success or failure. A second object, which may be simple or nested, is returned from many operations and contains result data.

The primary purpose of AP727 is to access DB2 and the databases which it operates. This is done through the medium of SQL. AP727 contains a precoded repertoire of Dynamic SQL statements which are issued in various combinations according to user command input.

Communication between AP727 and DB2 is performed via the Call Attachment Facility (CAF). Two independent subtasks are started by AP727 to maintain and operate the interface. The use of subtasks facilitates the wresting of control from DB2 by the simple expedient of DETACHing them. This action is performed if the user attentions out of a potential interlock or long-running DB2 operation and then retracts the active shared variable.

AP727 does not include any attention-handling logic, relying instead on the Shared Storage Manager (SSM) to process user-generated interrupts.

By default, the DB2 application plan id "APLP" is used. This may be overridden by specifying an alternative id at APL*Plus startup. The override is retrieved through conventional MVS PARM-processing logic via R1.

Structure

As Figure 1 demonstrates, AP727 comprises a 9-module main task (shown enclosed by the broken line) and two subtasks.

Module and subtask ids are in the form APPsnnni where: APP denotes APL*Plus s indicates the host operating system: T for MVS/TSO-specific modules V for VM/CMS-specific modules X for modules which are non-specific as to operating system. nnn is the Auxiliary Processor number: 727 for DB2 and SQL/DS i is an arbitrarily chosen module differentiator.

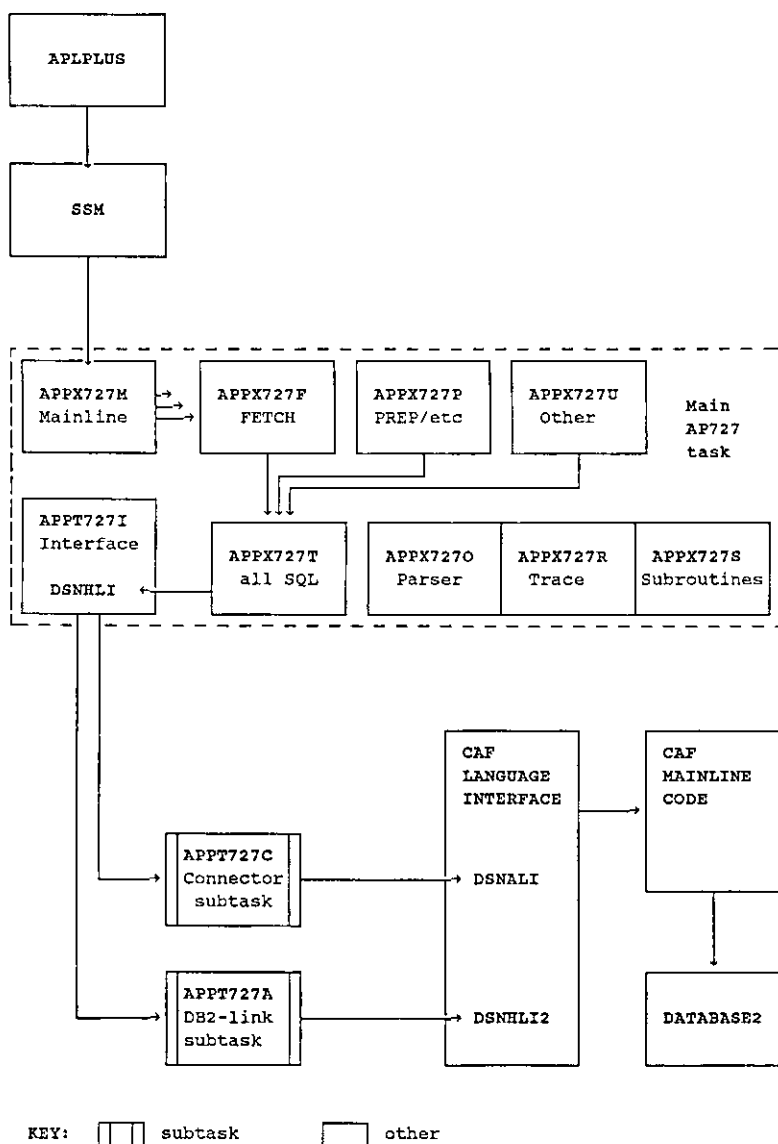


Figure 1: AP727 Module Context Chart

Processing Cycle

All shared variable operations and other SSM transactions are conducted by the Mainline module. When a user submission is received, the Parser module is invoked to validate and reform it. If the submission passes validation, the Parser module returns addressing data for the appropriate service module. If the submission is faulty, diagnostic information is returned instead.

The Mainline module regains control when parsing concludes. If a service module address has been returned, the Mainline module invokes that service module. Like the Parser module, some service routines may return diagnostic information. Finally, the Mainline module packages diagnostic information and result data together, returns them to the APL user via the SSM and issues a wait. When the wait is satisfied, the cycle reiterates.

An outer cycle commences with AP727 sign-on to the SSM and terminates with sign-off when the SSM requests it.

DB2 Interfacing

Module APPT727I (in the main task) and the two subtasks constitute the mechanism by which AP727 communicates with DB2 in the MVS context. Under VM/CMS, different tools are employed: module APPT727I would be replaced and the two subtasks would be deleted from the design. The relational database system might be SQL/DS instead of DB2.

The use of subtasks provides a ready means of severing interlocks and wresting control from DB2; one simply detaches the relevant subtask. The ensuing abend informs DB2 that the transaction is defunct.

Module APPX727T (in the main task) hosts AP727's repertoire of precompiled Dynamic SQL statements. Precompilation produces external references to entrypoint DSNHLI, which is dummied in module APPT727I. When APPX727T issues a Dynamic SQL statement, module APPT727I receives control and relays the parameter data to subtask APPT727A. The subtask invokes DSNHLI2 in the Call Attachment Facility's language interface, the CAF mainline code is triggered and the request passes to DB2.

Subtask APPT727A requests a CAF OPEN for the first Dynamic SQL statement to be issued following AP727 startup or a prior CAF CLOSE. APPT727A requests a CAF CLOSE whenever a COMMIT or ROLLBACK operation is requested. These operations are enacted by means of the CAF CLOSE request's SYNC and ABRT operation codes respectively. COMMIT and ROLLBACK are never issued as SQL statements per se.

Subtask APPT727C, the Connector, is invoked before AP727 issues the first Dynamic SQL statement of the session. It requests a CAF CONNECT via entrypoint DSNALI and subsequently, by remaining attached, maintains the connection established between the address space and the named DB2 subsystem. This eliminates implicit CONNECT processing and its associated overhead when a thread is opened via CAF OPEN. A CAF DISCONNECT is issued at AP727 shutdown.

Repertoire

AP727 repertoire consists of its own operations (services which are individually requested by the APL user), SQL statements (formulated and supplied by the APL user), and Dynamic SQL statements (the inbuilt means by which AP727 communicates user-supplied SQL statements to the relational database system and retrieves results).

Thus, for example, the PREP operation might be requested for a SELECT statement, the text of that statement being supplied by the APL user. AP727 would respond by issuing a Dynamic SQL DESCRIBE statement for the supplied SELECT statement.

Each AP727 service is requested by supplying its operation code, together with any necessary data (such as the text of an SQL statement). Operation codes and user-supplied SQL statement texts are character vectors.

The following operation codes request database services:

CALL	executes a PREP'd INSERT, UPDATE or DELETE statement.
CLOSE	closes the cursor for an OPEN'd SELECT statement.
COMMIT	files all database changes since the last synchpoint.
EXEC	a one-off combined PREP and CALL.
FETCH	retrieves data for an OPEN'd SELECT statement.
OPEN	opens a result cursor for a PREP'd SELECT statement.
PREP	prepares a supplied SQL statement for CALL or FETCH.
ROLLBACK	cancels all changes since the last synchpoint.

The remaining operation codes concern AP727 session management:

DESCRIBE	reports a SELECT statement's result format.
GETOPT	reports AP727 options extant.
MSG	gives the message text for a given returncode.
NAMES	gives the names of all extant PREP'd statements.
PURGE	removes one or all PREP'd statements.
SETOPT	specifies new AP727 session options.
STATE	reports the status of a PREP'd SQL-statement.
STMT	returns the text of a PREP'd SQL-statement.
TRACE	activates AP727 self-tracing capabilities.

Within the confines of the present article it is not possible to describe all the internals of AP727 in detail. Nonetheless, it is worth selecting and exploring one topic so that the general reader may gain some insight into the considerations entailed.

Accordingly, the remainder of the present article is devoted to a description of the mechanisms employed when retrieving data from DB2 in accordance with user-supplied SELECT statements.

Retrieving Data

SELECT statements are used to retrieve data from the relational database system. They name the relevant fields ("columns") and may optionally specify row-selection criteria based on field content. The effect of SELECT statement processing is to create - at least, conceptually - a result table containing the selected rows and columns, with a cursor indicating its first row. Rows may then be retrieved successively, the cursor always advancing downwards to indicate the next row to be retrieved.

As a general purpose tool, AP727 is faced with the problem of retrieving data from a relational database system without any prior information about the form and extent of that data. The problem is solved by issuing set sequences of Dynamic SQL statements to process SELECT statement texts provided by the APL user.

Before retrieval may commence, the relational database system must be queried so that data reception areas can be acquired and structured. These areas will vary according to the attributes of the columns in the result table built in response to the user-supplied SELECT statement. A DESCRIBE Dynamic SQL statement is issued to obtain the relevant information.

AP727's PREP operation conducts all these activities. The OPEN operation completes the preliminaries and declares a cursor id for the result table. Data retrieval may now begin. The FETCH operation retrieves result table rows and returns them in the form of an APL variable. The number of result table rows per FETCH and the organisation of the resulting APL object vary according to the AP727 options in effect. Options may be defaulted, explicitly declared via the SETOPT operation, or supplied as overrides on FETCH statements.

When all desired result table rows have been retrieved, the CLOSE operation may be used to release the cursor, reverting the statement to PREPARED status. Finally, the PURGE operation may be used to release all data structures associated with the preparation and use of the SELECT statement.

Data Retrieval Structures

AP727 supports up to forty concurrent SQL statements. Reference to statements is facilitated by associating a user-supplied name with each. The name, together with much other data, is held in an AP727 control block termed a Query Control Vector (QCV). Each extant statement has its own QCV. A QCV is created when a user-supplied SQL statement is first PREPARED. Up to forty concurrent QCVs are supported subject to storage availability. AP727's PURGE operation is supplied for the purpose of erasing QCVs after use.

As Figure 2 indicates, the address of each extant QCV is held in the QCVLIST. A pointer into this list designates the address of the QCV which is currently active (if any). This enables service routines to refer to QCV fields without repetitive searching, the Parser module having preset the pointer beforehand.

The QCV, besides containing user-supplied data such as SQL statement text and statement name, stands at the head of an expanding hierarchy of data structures used in data retrieval for a single SELECT statement. Of these, the most important is the SQL Descriptor Area (SQLDA). This control block, progressively created and completed by means of the Dynamic SQL DESCRIBE statement, contains size and attribute information for database columns, together with pointers to areas reserved in memory for column data and status indicators.

SQLDAs are used not only when receiving data from the relational database system but also when supplying data to that system. An AP727 QCV may contain a pointer to both a data-reception SQLDA (used to receive FETCHed data) and a values-description SQLDA (used to specify parameters which complete a partial SELECT statement). An SQLDA comprises a fixed-length stub followed by zero or more occurrences of a fixed length extension documenting a single database column. The number of occurrences is recorded in the stub.

To discover how many columns of data will be retrieved when a given SELECT statement is processed, a Dynamic SQL DESCRIBE statement targetting a dummy SQLDA (an SQLDA having no extensions) is issued. The necessary number of extensions - and thus the number of database columns - is reported in a field within the stub. Armed with this information, AP727 then acquires storage, builds an SQLDA having sufficient extensions and issues a DESCRIBE statement targetting that new SQLDA. This causes detailed information about the size and attributes of each column to be placed into the new SQLDA's extensions.

AP727 consults this information to determine how much space will be required for data reception purposes at FETCH time. Sufficient storage is acquired dynamically and allotted to each column. Reception field addresses are placed into the relevant SQLDA

extension fields. Indicator fields (used to signal null data) are similarly allotted and connected.

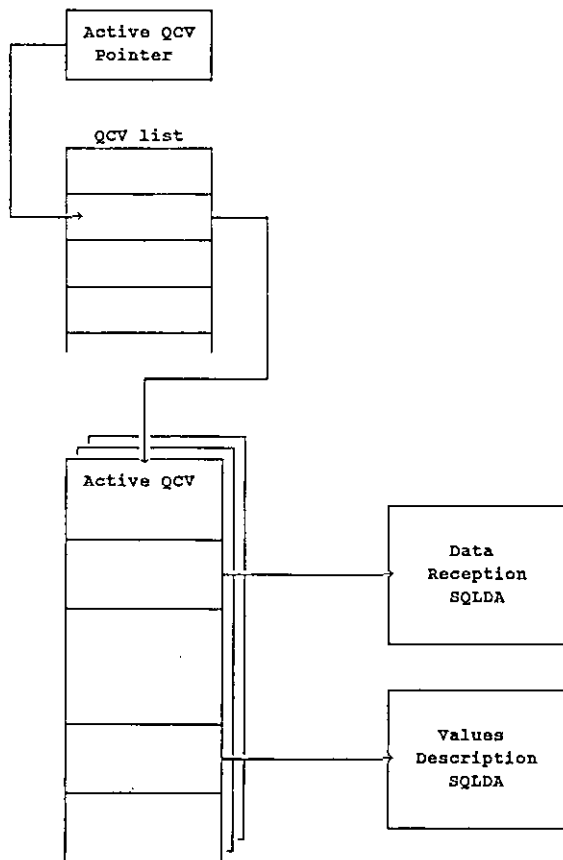


Figure 2: The active QCV

At this stage, the **SELECT** statement has attained PREPared status and AP727 is ready to retrieve the first row of the result table.

Figure 3 depicts the data-reception SQLDA for a PREPARED statement and shows how the reception fields are connected to it. Given this arrangement, a Dynamic SQL FETCH statement (issued as part of AP727's FETCH service) will place the next result-table row's column values into the reception fields associated with the SQLDA.

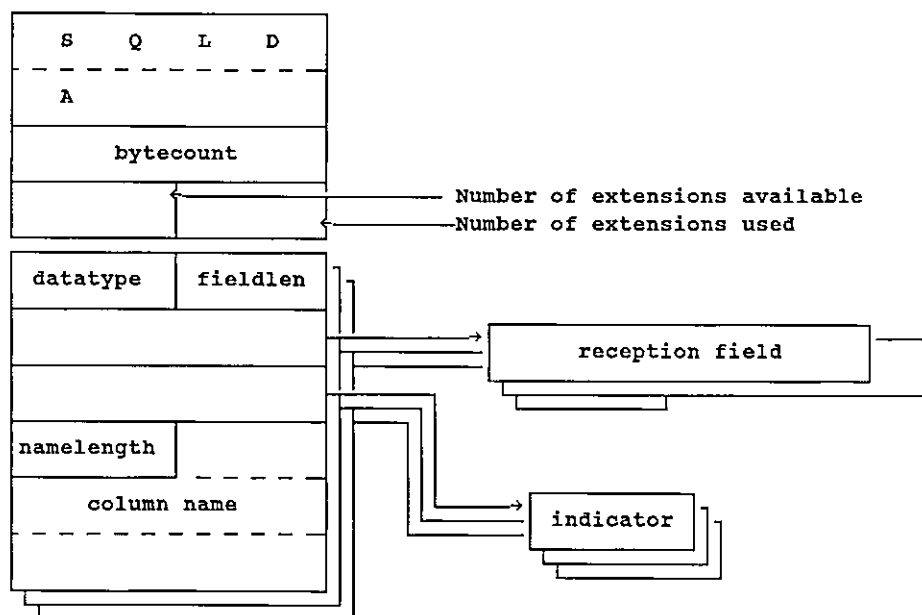


Figure 3: a data-reception SQLDA

If AP727's `FETCH` service were to return only a single row of the result-table, the APL user might have to invoke AP727 many times in order to retrieve an entire table. An extended loop of this kind, entailing shared variable communications on each iteration, would be inefficient. Instead, AP727 consults a user-modifiable count governing how many result-table rows are to be retrieved by each `FETCH` operation. AP727 loops to retrieve that number of rows and returns them as a single APL object. The object may be a vector of matrices or a matrix of vectors, according to the AP727 options specified, overridden or defaulted by the user.

The additional storage required for the consolidation of result table rows into a single APL object is acquired when the `PREPARED` statement is `OPENED` and released when it is `CLOSED`.

Conclusion

Although many APL users routinely avail themselves of the services of auxiliary processors, few gain much exposure to auxiliary processor internals. It is hoped that this article, though necessarily an incomplete description of AP727, has cast some light on an unfamiliar area and proved of interest to APL users and relational database users alike.

TECHNICAL SECTION

This section of VECTOR is aimed principally at those of our readers who already know APL. It will contain items to interest people with differing degrees of fluency in APL.



EXCITING POSITIONS AT THE LEADING EDGE

With top City institutions and leading Blue chip companies amongst our client list we have a lot to offer experienced APL programmers.

Call Stan Wilkinson to discuss your future.

HMW COMPUTING LIMITED
Hamilton House 1 Temple Avenue
London EC4Y 0HA Telephone 01-353 4212

Hacker's Corner: a little bit of C

by Adrian Smith

This little program might help you to see how the other half lives! It will take almost anything, and output a file suitable for Word; to my amazement it even ate Allan Gay's piece, which came as an APL component file! It runs in about 5 sec for a typical article.

```

/::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::/
/* tf.c  April 1988  by Adrian Smith                                     */
/* Basic text filter ... passes any input file out to STDOUT having ... */
/* - ensured CR/LF pairs (to deal with APL quad-edit vectors)          */
/* - stripped high bits from WordStar output                            */
/* - removed white space (including all spaces at the end of paragraphs)*/
/* - made tabs into visible ; characters. Word can always put them back! */
/::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::::/
#include <stdio.h>
#include <ctype.h>

main(argc,argv)  int argc; char *argv[];

{ FILE *fp;
  int ch, in, cr = 13, lf = 10, tab = 9, bl = 0, np = 0;

  if (argc != 2) {printf("Please use: tf file[>outfile]\n"); exit(1); }

  fp = fopen(argv[1],"rb");                                     /* open for binary read */
  if (!fp) { printf("Can't open file '%s'\n",argv[1]); exit(1); }

  while ((in = fgetc(fp)) != EOF)                               /* Process whole file */
  {
    ch = toascii(in);                                           /* Strip high bits (WStar) */
    if (ch == lf);                                              /* Forget linefeeds */
    else if (ch == cr)                                          /* CR/LF pair */
      { if (np)                                                /* ... 2nd in a row */
        { putchar('\n'); np=0; bl=0; }                          /* gives new para. */
      else
        { np=1; bl=1; }                                         /* ... get ready for para */
      }
    else if (ch == ' ') bl = 1;                                  /* Kill multiple blanks */
    else
      { if (ch == tab) { ch = ' '; bl=0; }                      /* Leave recognisable tabs */
        if (ch == 96) ch = '\';                                  /* Dispose of back quotes */
        if (bl) putchar(' ');                                  /* Output pending blank */
        if (ch > 32) putchar(ch);                               /* Ignore control chars */
        bl=0; np=0;                                             /* Output, and kill flags */
      }
  }
  putchar('\n');                                               /* Finish off */
} /*===== THE END =====*/

```

Technical Correspondence

From: Peter Branson

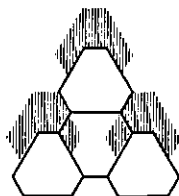
19th May 1988

May I thank Bob Bykerk for the first paragraph of his letter in VECTOR 4.4 about removing all duplicated rows in a matrix. Not only is his idiom more elegant than the one in my letter (VECTOR 4.2), but its simplicity makes it easier to remember. Perhaps more important, it is very much more efficient, and runs some 25 times faster under VS APL.

Here is one example of why it is worth publishing not-so-good code - it helps turn up something better!

Of course with hindsight it is obvious, but I always find that the trouble with hindsight is that it never seems to be available at the times when you really need it!

Peter Branson
Oaklands Cottage
Wray Common
Reigate, Surrey



APL PEOPLE

THE APL EMPLOYMENT AGENCY

- ★ Permanent and Contract Vacancies
- ★ UK and Overseas
- ★ Free Advice on career matters
- ★ Discreet, confidential service

We currently have many openings for programmers
with 1-3 years APL experience (any type).

Contact Jill Moss on 0225 462602 during office hours
or 0225 333618 eves & w/ends

or write to her at The Old Malthouse, Clarence St., Bath BA1 5NS

VSAM file Processing in APL

by John Sullivan

This article is not intended to be an exhaustive account of how to process every type of VSAM file and every requirement that a user may have when using those files. Rather, it is a case-study, designed to show you, the gentle reader, how easy VSAM file processing can be, and why you should not be frightened of it.

For various reasons that need not concern us here, the functions and programs used in the application covered by the case-study are deliberately somewhat restrictive, in that key-lengths are hard-coded into some of the cover functions rather than being held in global variables, and several operations that are normally permitted in general are not permitted in this application (such as file-updates via the alternate index). It is not intended that you should impose the same restrictions on your application, but rather this case-study is intended to guide you to how to use VSAM in the best way for your needs.

Background

In this article I shall be referring to other languages in addition to APL, these are FORTRAN (because I am going to include some FORTRAN code to be called via Associated Processor 11 in the application) and PL/1 (because my VSAM file has to be read by PL/1 in addition to APL). Please bear with me if the parts dealing with these languages are unintelligible to you, because they may be useful to others.

For those of you who remember the article by David Piper in Vector 3.2, covering VSAM file processing in a more general way, I should state that (a) this system was written long before David's article appeared and (b) this system is specific to the VSAM file being used, whereas David's article covered more general concepts. However, I am currently investigating the possibilities of an extremely general set of VSAM processing functions which will combine all of the best points of IBM's supplied workspaces and David Piper's concepts, and it will have a syntax exceedingly similar to another filing system well-known to the APL world. This new system will be described in a future issue of Vector, with the editor's permission.

VSAM concepts

This article is only concerned with VSAM key sequenced data sets (KSDSs) which contain variable-length records that are accessed by means of one or more keys embedded in the record. The length of an existing record can be changed when it is rewritten.

Alternate indexes can be provided for key-sequenced files: their provision makes processing much easier in some circumstances, as explained later.

The VSAM file discussed here contains the following: a 'base cluster' containing a data component and an index component; an 'alternate index cluster' also containing data and index components; an 'alternate index path' which connects the alternate index to the base cluster. The keys in the base cluster must be unique whereas the alternate index keys do not have to be unique (in this application it is essential that they are not).

Application Requirements

There is a requirement to do the following things in the application:

- Close a file (base cluster or alternate index)
- Open a file for input
- Open a file for updates (base cluster only, although you can also do this for alternate index in the general case)
- Read a record from the file by specifying the whole of the primary key
- Read a record from the file by specifying only the first few bytes of the primary key
- Read a record from the file by means of the alternate index
- Read the record immediately following the one you just read
- Read the record immediately preceding the one you just read
- Write a completely new record to the file
- Read a record from the file, amend it and replace it on the file

All of these things can be done in PL/1, although activities can become complicated if you want to do them all, because each alternate-index path needs a separate filename, and you can only read a file in one direction (forwards or backwards), not both. Things are easier in FORTRAN, although you might have some problems with varying-length records and different datatypes in the same record. For FORTRAN a maximum of 8 alternate indexes can be used at any one time; although each alternate index needs a separate file all of them are accessed by the same unit number as the base cluster, and you can read in both directions on the same file. APL will not let you read the file backwards, which is a nuisance, although it is possible to simulate a backwards sequential read by means of FORTRAN subroutines executed through Associated Processor 11. I can see no real reason for not allowing backwards reading in APL: maybe IBM will make this feature available in future releases of the program product.

Setting up the VSAM file

I used the CLIST shown below to set up my VSAM KSDS. You could, of course translate this into JCL, if you really wanted to. This is the basic CLIST; add your own bells and whistles for error-handling.

```

PROC 0
CONTROL MAIN LIST MSG

DEFINE +
  CLUSTER (NAME('PROJECT.VSAMFILE') OWNER(USERID) +
    VOLUME(VOLSER) SHR(4) KEYS(32 0) +
    RECSZ(8192 32600) IMBED REPL FSPC(0 0) SPND) +
  INDEX (NAME('PROJECT.VSAMFILE.INDEX') CYLINDERS(25)) +
  DATA (NAME('PROJECT.VSAMFILE.DATA') CYLINDERS(250)) +
  CATALOG ('USERCAT')
IF &LASTCC = 0 THEN EXIT

REPRO +
  INDATASET(TEMP.VSAMFILE.DATA) +
  OUTDATASET('PROJECT.VSAMFILE')
IF &LASTCC = 0 THEN EXIT

DEFINE +
  AIX (NAME('PROJECT.VSAMFILE.AIX') OWNER(USERID) +
    RELATE('PROJECT.VSAMFILE') +
    VOLUME(VOLSER) SHR(4) KEYS(8 9) +
    RECSZ(512 32600) REPL IMBED FSPC(0 0)) +
  INDEX (NAME('PROJECT.VSAMFILE.AIX.INDEX') CYLINDERS(3)) +
  DATA (NAME('PROJECT.VSAMFILE.AIX.DATA') CYLINDERS(22)) +
  CATALOG ('USERCAT')
IF &LASTCC = 0 THEN EXIT

/* BUILD ALTERNATE INDEX FOR NEW VSAMFILE
BLDINDEX +
  INDATASET('PROJECT.VSAMFILE') +
  OUTDATASET('PROJECT.VSAMFILE.AIX')
IF &LASTCC = 0 THEN EXIT

/* DEFINE ALTERNATE INDEX PATH
DEFINE +
  PATH (NAME('PROJECT.VSAMFILE.PATH') OWNER(USERID) +
    PATHENTRY('PROJECT.VSAMFILE.AIX'))

```

After setting up the VSAM base cluster but before constructing the alternate index(es) you have to put some data into the file. This process of putting data into the empty VSAM file is known as loading, and you stay in the loading state until you close the file. You can do the loading in a number of ways, either by yourself under some kind of program control (using any language that will handle VSAM), or by using the IBM utility REPRO, which is what I did. Whichever method you use, your data MUST be presented in ascending sequence of primary key.

If you decide to load the file yourself, the following points should be noted:

- You MUST open the file for writing (Service request OW)
- You cannot read or erase a record (Service request R or E) while you are loading.
- All the records to be dealt with must be in ascending order of primary key.
- Once you have loaded data into the file you cannot (and do not need to) repeat the process. Therefore you could load a single record consisting completely of blanks (large enough to hold the key and all of the relevant alternate indexes, of course) and then delete it at the first available opportunity. This is what happens implicitly in FORTRAN if you open a new VSAM KSDS with an action of READWRITE.

If you decide to use REPRO, your input data should be in a varying-length sequential dataset, sorted into ascending order of the keys. REPRO is one of IBM's Access Method Services commands so it is very quick.

You must repeat the DEFINE AIX, BLDINDEX and DEFINE PATH commands for each alternate index that you want to set up. You will see that I have set up just one alternate index with an eight-byte key starting at offset 9 (position 10) in the record. You can define your alternate index keys to start anywhere as long as each record is actually long enough to hold the whole of the alternate key.

Accessing the VSAM file

Before doing anything you have to allocate the file and any alternate indexes so that you have a means of referring to them in your programs. This can be done outside APL in a CLIST, or it can be done in the workspace latent expression or a function called by the latent expression. With TSO's ALLOC command you use the cluster name for the base cluster, and the pathname for the alternate index, in the dataset-name parameter.

When you use Auxiliary Processor 123 you will have to share one pair of variables for each index that you wish to access. (You could do everything through a single pair of variables, but it would give rise to a lot of wasted time, opening and closing files, when you want to read different alternate indexes in 'random' order.) APL imposes a maximum of 14 pairs of variables to be shared (and hence files to be open) at any one time. All the pairs of shared variables should be global in the workspace.

There are some rules of thumb that should be followed when writing the cover functions:

- Make your file names logical, so that the programming becomes simpler. For instance, if you have 9 alternate indexes, call their filenames AI1, AI2, ..., AI8, AI9.
- Make your shared-variable names logical for the same reason.

In the example, the base cluster is allocated to file VSAMBC, the alternate index to VSAMAI: if I had more than 1 alternate index I would have called them VSAMAI1, VSAMAI2, etc; and if all files were unrestricted I would have named them differently, but using the same ideas, for example, VSAM0 for the base cluster, VSAM1, VSAM2, etc., for the alternate indexes.

Similarly, the shared variables are CTLb and DATb for the base cluster and CTLa and DATA for the alternate index; other schemes are possible as shown above with filenames.

The functions used in the application are

aiCLOSE..... Closes the alternate index

aiGET..... Reads a record from the alternate index, given the key. In theory, you can read the alternate index using a key of any length up to the keylength that you used when the alternate index was defined. In my case I have insisted that the user must specify the full 8 bytes for the alternate index key.

aiREAD..... Sequentially reads a record from the alternate index in the forwards direction.

aiROPEN Opens the alternate index for read only

bcBKWD Sample function to read the base cluster backwards. Because this is a sample function it is not very fast.

bcCLOSE Closes the base cluster

bcDELETE..... Delete a record from the file via the base cluster. The whole of the primary key must be supplied. File must be open for update.

bcGET Reads a record from the base cluster given the key. The key can be any length between 1 and the keylength defined when the file was set up

bcKF..... Returns the key of the previous successful operation

- bcPUT.....Writes a record to the file through the base cluster. File must be open for update. This function combines the effects of bcUGET and bcUPUT.
- bcREAD.....Sequentially reads a record from the base cluster in the forwards direction
- bcREPLACE.....Replaces a record in the file, which must be open for update
- bcROPENOpen the base cluster for read only
- bcUGET.....As for bcGET, but uses service RU instead of R. File must be open for update
- bcUOPENOpen the base cluster for update
- bcUPUT.....Write a record to the file through the base cluster. File must be open for update and previous operation must have been a read-for-update (bcUGET).
- dbALLOCAllocate and open the VSAM base cluster and alternate index. The relevant filenames are held as elements in nested vector of vectors <dbname>.
- SETUP__VSAMerc Sets up the VSAM error-code matrix. Run once then forget it.
- VSAMchkChecks the return code from a VSAM request.

Reading the file backwards

As mentioned above, APL will not allow you to read a VSAM file sequentially backwards. Sometimes, you need this facility and its absence from the APL system can be rather frustrating. It is possible to read the file backwards, however, by using FORTRAN, although the method is cumbersome, and unless you are going to write specific code for each eventuality for the data in your file it is messy.

What I have done is written a suite of 4 subroutines to open the file, close it, read it via the primary key, and read it sequentially backwards, each read operation reading ONLY the 32-byte primary key. When enough of these keys have been saved in the workspace you can then use APL to read the relevant records by means of these keys. (I did say it was messy, didn't I?) You can, of course, write your subroutines to do the same with an alternate index if you so desire.

The procedure is as follows:

- Close the base cluster and any alternate indexes in APL, which has the effect of writing all amendments from the buffers to the file.
- Open the base cluster (only) in FORTRAN. In FORTRAN read the record from which you want the backwards reading to start, by supplying the subroutine with the key. This is done in order to position the file pointer.
- Now backspace the file twice (once to position at the record you've just read and once to move in front of it). Read the file sequentially (i.e. without specifying a key), returning the key of this record to the workspace.
- Repeat the last step as often as required until your ending condition is satisfied
- Close the file in FORTRAN; reopen the base cluster and alternate indexes in APL
- Continue ...

The FORTRAN subroutines are listed in Appendix 2, and the relevant cover function is amongst the functions in Appendix 1 as bcBKWD.

Points to note

When using the alternate indexes you should note that you could get into trouble if you delete records using the alternate indexes and the base cluster is still open. You should close the base cluster, do your deletions, close and if necessary reopen the alternate index (this makes sure all the deletions are done); then and only then should you reopen everything. Of course, deleting via the base cluster doesn't give you this trouble. In my application we decided that there was no real reason for wanting to delete records via the alternate index, so we didn't include the facility in the cover functions.

You can use any of the pairs of shared variables to add new records to the file, since you have to supply the whole record in any case. If a duplicate key in any of the indexes would result where it is not allowed, you get the usual error message. This is particularly useful if you want to read through all the records with the same nonunique key in an alternate index and then update some of the records (remember with the file open for update, a write request after a RU request will replace the record just read with the new one. This applies no matter which index you used to obtain the record. In my application we decided that there was no real reason for wanting to update via the alternate index, so we didn't include the facility in the cover functions.

Sometimes your VSAM file will run out of space, and further space will have to be allocated. The method is simple. First of all you use REPRO to make a backup sequential

copy of your file, making sure that this has completed correctly before continuing. Then you delete the base cluster (this actually deletes all of the files connected with the base cluster) and run the setup CLIST after increasing the space allocations and making sure that the REPRO step is copying your backup dataset back into the VSAM file. I have done this several times and have had no bother, but be patient since BLDINDEX can take a fair length of time when you have a big file. System failures may cause grief if they occur during this process so do not delete your backup file until everything is O.K. (you can usually delete any half-complete alternate index and pick up again from where the CLIST crashed, which should save some time).

You can also use REPRO to take extracts of the file to give to other people. You can use the PRINT command to obtain a dump of selected parts of the file for documentation and/or debugging purposes. See the Access Methods Services documentation for more details.

Why do you need an alternate index? Consider a simple VSAM file containing the names of your company's sales force. Fields contain payroll number, department, territory covered, sales target, salary, commission, name, address, telephone number(s) - you know the sort of thing that people delight in using for examples. The obvious candidate for the unique primary key is the payroll number. Having set this up, get me a list of all salesmen in the Greater London area without searching the whole file. Or all of those with a salary of £25,000 or more. Of course, this file is small, so reading it in total should not provide too much of a problem with unnecessary processing. But consider a file containing a million records (different contents, of course, unless your company employs 2% of the total population of the U.K.!) and an extract request that results in only a dozen records being selected. That's why you need (multiple) alternate indexes.

Why do you need a VSAM file? I don't know, that's up to you to decide. VSAM is easier than some other filing systems, and easier to handle than most people think, but like all filing systems it has its good points and its bad ones. I started experimenting with VSAM when APL2 arrived with its concept of libraries stored in VSAM files, and when I discovered that there was nothing to be scared of it was natural to add VSAM to the list of problem-solving techniques.

Appendix 1: Function Listings

```

[α]v aiCLOSE.3  ρ: 9  1986-07-23 13.23.19
[0]  aiCLOSE;RC
[1]  α Close the currently open alternate index
[2]  CTLA←i0          α Try empty vector - test for syntax error
[3]  →(0=ρRC+CTLA)/0    α Exit if no shared variables
[4]  →(Λ/12 100012*1000001RC)/Δ1
[5]  CTLA←'C'          α Otherwise try setting a 'C' for CLOSE
[6]  RC←CTLA
[7]  α Ignore VSAM return code for file not open
[8]  Δ1:100022 VSAMchk RC

```

```

[α]v aiGET.3  ρ: 9  1986-07-23 13.24.00
[0]  Z←aiGET KEY
[1]  α Reads a record from the database alternate index, given the key.
[2]  α The file must be open and the key must be correct.
[3]  DES(8*ρKEY)/'Key for alternate index must be 8 bytes long'
[4]  CTLA←'R:',KEY      α Issue GET request
[5]  α Exit with null vector if not found
[6]  ϕ(v/(8,[1,1]4 16)Λ.=2*Z+CTLA)/'→ρZ+''''
[7]  VSAMchk Z          α Check return code
[8]  Z←DATA             α Get data variable if return code OK

```

```

[α]v aiREAD.3  ρ: 5  1986-07-23 13.24.16
[0]  Z←aiREAD
[1]  α Sequentially reads a record from the alternate index
[2]  CTLA←'R'
[3]  VSAMchk CTLA
[4]  Z←DATA

```

```

[α]v aiROPEN.3  ρ: 15 1986-07-23 13.25.08
[0]  aiROPEN;A;N
[1]  α Open alternate index for READ only (Cannot open for update)
[2]  aiCLOSE          α Close any already-open file
[3]  α If variables already shared then don't do them again
[4]  →(2Λ.=DSVO"N+CTLA' DATA')/Δ1
[5]  A←DSVR"N          α Might not be shared properly so retract
[6]  DES(2v.Λ123 DSVO"N)/'Unable to share with AP123'
[7]  A←0 0 1 1 DSVC 1→N    α Set access control
[8]  Δ1:CTLA←'OR VSAMAI'    α Open VSAM file for read only
[9]  100013 VSAMchk A+CTLA
[10] CTLA←'R'            α Sequential read of first item
[11] VSAMchk A+CTLA
[12] CTLA←'KF'           α Key of first item
[13] VSAMchk A+CTLA
[14] aiKEYLEN←ρaiFIRSTKEY+DATA α Store key length, first key

```

```

[A]v bcBKWD.3  p: 25 1988-01-25 15.28.16
[ 0]  Z+bcBKWD B;A;P;Q;T;X
[ 1]  A Read the database base cluster backwards
[ 2]  A Care: this routine loses positioning on BC and AI
[ 3]  A Return the null vector if cannot do it, else return the record
[ 4]  DECLARE CHARACTER VECTOR SIMPLE B
[ 5]  DES(32*pB)/5 3      A Length error if B not 32 bytes
[ 6]  K+'APL DD1 FT03K01'  A File already allocated?
[ 7]  ->(B+K)/Δ2,Z+10      A Yes, so bypass ALLOC statement
[ 8]  K+'ALLOC DD(FT03K01) DS('', (1>dbname), '') SHR'
[ 9]  ->(O+K)/O          A Exit if unable to allocate
[10]  A Database allocated for FORTRAN. So far so good.
[11]  Δ2:bcCLOSE          A Close database for APL
[12]  aiCLOSE            A ... and alternate index
[13]  ->(O+VOPEN 0 3)/O    A Open for FORTRAN, exit if error
[14]  A Try to read record for initial positioning purposes
[15]  A If this fails there is an error somewhere
[16]  ->(O+*A+VGET 0 3 B(32+' '))/ΔCLOSE
[17]  P+2>A              A Save it for comparison purposes
[18]  A+VFRDBK 0 3(32+' ') A Backwards read
[19]  ΔCLOSE:T+VCL0S 0 3  A Close in FORTRAN
[20]  bcROPEN            A Reopen in APL
[21]  aiROPEN            A ... and alternate index
[22]  ->(O+*A)/O          A If error then quick quit
[23]  ->(P= 0+2>A)/O      A If equal then cannot read backwards
[24]  Z+bcGET Q           A Return record as promised

```

```

[A]v bcCLOSE.3  p: 9 1986-07-23 12.43.28
[ 0]  bcCLOSE;RC
[ 1]  A Close the currently open base cluster
[ 2]  CTLb+10            A Try empty vector - test for syntax error
[ 3]  ->(O=μRC+CTLb)/O      A Exit if no shared variables
[ 4]  ->(Λ/12 100012*1000001RC)/Δ1
[ 5]  CTLb+'C'          A Otherwise try setting a 'C' for CLOSE
[ 6]  RC+CTLb
[ 7]  A Ignore VSAM return code for file not open
[ 8]  Δ1:100022 VSAMchk RC

```

```

[A]v bcDELETE.3  p: 12 1986-08-13 15.03.14
[ 0]  Z+bcDELETE KEY;R
[ 1]  A Deletes a record on the file
[ 2]  A The file must be open and the key must be correct.
[ 3]  A Returns 1 for record deleted, 0 otherwise
[ 4]  CTLb+'R:',KEY      A Issue GET request
[ 5]  ->(v/(B,[1.1]4 16)Λ.=2+R+CTLb)/~Z+1 A Not found
[ 6]  VSAMchk R          A Check return code
[ 7]  ->(KEYΛ.= (μKEY)+Z+DATb)/Δ1
[ 8]  ->Z+0              A Error reading file so return
[ 9]  Δ1:CTLb+'E:',KEY   A Erase record
[10]  VSAMchk R+CTLb     A Check return code
[11]  Z+1                A All is OK

```

```

[A]V bcGET.3  ρ: 8  1986-08-28 15.04.07
[0]  Z←bcGET KEY
[1]  A Reads a record from a file, given the key.
[2]  A The file must be open and the key must be correct.
[3]  CTLB←'R:',KEY      A Issue GET request
[4]  A Exit with null vector if not found
[5]  ϕ(√/(8,[1.1]4 16)∧.=2+Z+CTLB)/'→ρZ←''''
[6]  VSAMchk Z      A Check return code
[7]  Z←DATb      A Get data variable if return code OK

```

```

[A]V bcKF.3  ρ: 7  1987-12-11 10.27.21
[0]  Z←bcKF;I
[1]  I←1
[2]  ΔI:CTLB←'KF'
[3]  ϕ(0∧.=CTLB)/'→0,ρZ←DATb'
[4]  ϕ(2<I+1)/'→ρZ←''''
[5]  ϕ(0=ρbcGET 1↑[AV])/'Z←bcREAD'
[6]  →ΔI

```

```

[A]V bcPUT.3  ρ: 11  1986-07-23 13.26.13
[0]  bcPUT X;Q;Z
[1]  A Puts a record on the file
[2]  A The file must be open and the key must be correct.
[3]  CTLB←'RU:',bcKEYLEN+X      A Issue GET request
[4]  →(√/(8,[1.1]4 16)∧.=2+Z+CTLB)/ΔI A Branch if not found
[5]  VSAMchk Z      A Check return code
[6]  →(X= Q←DATb)/0      A Don't write if no change
[7]  ΔI:X[32+;4]←4 IO 0 100 10013;0TS      A Date of last update
[8]  DATb←X      A Put data in data variable
[9]  CTLB←'W'      A Issue PUT request
[10] VSAMchk CTLB      A Check return code

```

```

[A]V bcREAD.3  ρ: 7  1986-07-23 13.26.33
[0]  Z←bcREAD
[1]  A Sequentially reads a record from the base cluster
[2]  A Returns the null vector if EOF
[3]  CTLB←'R'
[4]  ϕ(8 4∧.=Z+CTLB)/'→0,ρZ←''''
[5]  VSAMchk Z
[6]  Z←DATb

```

```

[A]V bcREPLACE.3  ρ: 7  1986-07-23 13.26.43
[0]  Z←bcREPLACE X
[1]  A Replace a record in the file, which must be open for update
[2]  CTLB←'RU:',32+X      A Does record already exist?
[3]  800016 VSAMchk Z+CTLB      A Ignore error if record not found
[4]  DATb←X
[5]  CTLB←'W'
[6]  800028 VSAMchk Z+CTLB      A Return error code to calling function

```

```

[A]v bcROPEN.3 p: 15 1986-07-23 12.44.58
[ 0] bcROPEN;A;N
[ 1] A Open base cluster for READ only
[ 2] bcCLOSE A Close any already-open file
[ 3] A If variables already shared then don't do them again
[ 4] →(2^.=DSVO"N+ 'CTLb' 'DATb')/Δ1
[ 5] A+DSVR"N A Might not be shared properly so retract
[ 6] DES(2v.≠123 (DSVO"N)/'Unable to share with API23'
[ 7] A+0 0 1 1 DSVC 1=N A Set access control
[ 8] Δ1:CTLb+ 'OR VSAMBC' A Open VSAM file for read only
[ 9] 100013 VSAMchk A+CTLb
[10] CTLb+ 'R' A Sequential read of first item
[11] VSAMchk A+CTLb
[12] CTLb+ 'KF' A Key of first item
[13] VSAMchk A+CTLb
[14] bcKEYLEN+ρbcFIRSTKEY+DATb A Store key length, first key

```

```

[A]v bcUGET.3 p: 16 1986-08-11 12.24.53
[ 0] Z+bcUGET KEY;I
[ 1] A Reads a record for update
[ 2] I+1 A Try twice only
[ 3] A The file must be open and the key must be correct.
[ 4] Δ0:CTLb+ 'RU: ',KEY A Issue GET request
[ 5] A Exit with null vector if not found
[ 6] →(v/(8,(1.1/4 16)^.=2+Z+CTLb)/Δ1
[ 7] VSAMchk Z A Check return code
[ 8] →(KEY= (ρKEY)+Z+DATb)/0 A Get data variable if return code OK
[ 9] →(2<I+I+1)/Δ1 A Only try twice
[10] bcUOPEN
[11] →Δ0
[12] A Logical error on GET request causes VSAM to lose positioning
[13] A (See VSAM programmer's guide p.94)
[14] Δ1:bcUOPEN A Reestablish positioning
[15] Z+ '' A Exit with null vector

```

```

[A]v bcUOPEN.3 p: 15 1986-07-23 12.43.12
[ 0] bcUOPEN;A;N
[ 1] A Open base cluster for UPDATE only
[ 2] bcCLOSE A Close any already-open file
[ 3] A If variables already shared then don't do them again
[ 4] →(2^.=DSVO"N+ 'CTLb' 'DATb')/Δ1
[ 5] A+DSVR"N A Might not be shared properly so retract
[ 6] DES(2v.≠123 (DSVO"N)/'Unable to share with API23'
[ 7] A+0 0 1 1 DSVC 1=N A Set access control
[ 8] Δ1:CTLb+ 'OU VSAMBC' A Open VSAM file for read only
[ 9] 100013 VSAMchk CTLb
[10] CTLb+ 'R' A Sequential read of first item
[11] VSAMchk CTLb
[12] CTLb+ 'KF' A Key of first item
[13] VSAMchk CTLb
[14] bcKEYLEN+ρbcFIRSTKEY+DATb A Store key length, first key

```

[A]v dbALLOC.3 p: 9 1988-01-21 14.25.07

```
[0] dbALLOC
[1] bcCLOSE          A Close base cluster
[2] aiCLOSE          A Close alternate index
[3] K+'ALLOC DD(VSAMBC) DS('', (1=dbname), '') SHR REUSE'
[4] DES(E+O+K)/'Unable to allocate VSAM base cluster'
[5] K+'ALLOC DD(VSAMAI) DS('', (2=dbname), '') SHR REUSE'
[6] DES(O+K)/'Unable to allocate VSAM alternate index'
[7] bcROPEN          A Open Base cluster for read
[8] aiROPEN          A Open Alternate index for read
```

[A]v bcUPUT.3 p: 7 1986-07-23 12.44.10

```
[0] bcUPUT.X
[1] A Rewrites a record to the file
[2] A The file must be open for UPDATE, and the previous action MUST
[3] A have been a READ for UPDATE
[4] DATb+X          A Put data in data variable
[5] CTLb+ 'W'       A Issue PUT request
[6] VSAMchk CTLb    A Check return code
```

[A]v SETUPVSAMerc.3 p: 81 1988-01-18 17.30.50

```
[0] SETUPVSAMerc;A
[1] A Set up error-code matrix for VSAM files
[2] A+O 2p''
[3] A+A,[1]4 'More space requested or another volume mounted'
[4] A+A,[1]8 'More than 1 record with specified key'
[5] A+A,[1]16 'Record written into another control area'
[6] A+A,[1]100012 'Command syntax error'
[7] A+A,[1]100013 'VSAM file already open'
[8]
[74]
[75] A+A,[1]800188 'Invalid dataset type'
[76] A+A,[1]800192 'Invalid relative record number'
[77] A+A,[1]800196 'Invalid request for RRDS'
[78] A+A,[1]800200 'Invalid access through a path'
[79] A+A,[1]800204 'Invalid request in Backward mode'
[80] VSAMerc+A
```

[A]v VSAMchk.3 p: 11 1986-07-23 12.52.09

```
[0] X VSAMchk RC;A
[1] A Check return code from a VSAM request
[2] DES(2=DSVO 'CTLb')/'No variables shared with VSAM'
[3] →(O=RC+1000001RC)/O A Exit if return code OK
[4] A If RC is amongst those items specified in X then accept it
[5] q(2=QNC 'X')/'→(RC+X)/O'
[6] A If in VSAMerc then use that message
[7] q(RC+VSAMerc[1])/'→ΔE, pA+VSAMerc[VSAMerc[1];RC;2]'
[8] A Error codes not accounted for by any of the above
[9] A+ 'An error has occurred, codes ', 0 100000, RC
[10] ΔE:DES A
```

Appendix 2: FORTRAN subroutines

Note that these use VS FORTRAN release 4.1 (IBM's version of FORTRAN-77) so they may not resemble the kind of FORTRAN you've come to know and hate!

```

      SUBROUTINE FVOPEN(IERR,IUNIT)
      OPEN (UNIT=IUNIT,IOSTAT=IERR,ACCESS='KEYED',ACTION='READ')
      END
****
      SUBROUTINE FVCLOS(IERR,IUNIT)
      CLOSE (UNIT=IUNIT,IOSTAT=IERR)
      END
****
      SUBROUTINE FVGET(IERR,IUNIT,KEY,KEYR)
      CHARACTER*32 KEY,KEYR
      LOGICAL OPEN
*
* Test if file is open. If not then set return code=1, reason code=22
* (as for APL2 APL23: request to process a file not currently open)
*
      INQUIRE (UNIT=IUNIT,IOSTAT=IERR,OPENED=OPEN)
      IF ( .NOT. OPEN) THEN
         IERR = 65558
         RETURN
      END IF
*
* Read the VSAM file using the specified key.
* If the record we read has a different key then we have lost
* positioning somehow, and we should return with an error code.
* I am using return code 1, with reason code 0, to show that there has
* been an error, but the file has been read. I am also returning the
* key of the record that was read.
*
      READ (UNIT=IUNIT,IOSTAT=IERR,KEY=KEY) KEYR
      IF (KEY .NE. KEYR) IERR = 65536
      END
****
      SUBROUTINE FVRDBK(IERR,IUNIT,KEYR)
      CHARACTER*32 KEYR
      LOGICAL OPEN
      INQUIRE (UNIT=IUNIT,IOSTAT=IERR,OPENED=OPEN)
      IF ( .NOT. OPEN) THEN
         IERR = 65558
         RETURN
      END IF
*
* Backspace once to retrieve the record we've just read.
* Backspace once again to retrieve the previous record.
*
      BACKSPACE (UNIT=IUNIT,IOSTAT=IERR)
      IF (IERR .NE. 0) RETURN
      BACKSPACE (UNIT=IUNIT,IOSTAT=IERR)
      IF (IERR .NE. 0) RETURN

```

```

*
* We only want the key to the record
*
      READ (UNIT=IUNIT,IOSTAT=IERR) KEYR
      END

```

We will also want the following Assembler module, which should be link-edited before the FORTRAN package (i.e. as the primary entry point):

```

FVSAM  CSECT
      XR   15,15              Identify this as EP table
      BCTR 15,14              Return with -1
      DC   A(FIRST)           Start of Entry Point table
      DC   A(16)              Width of Entry Point table
      DC   A(LAST)            Last entry in the table
FIRST  DC   CL8'FVOPEN        ',V(FVOPEN),A(0)
      DC   CL8'FVCLOS        ',V(FVCLOS),A(0)
      DC   CL8'FVGET         ',V(FVGET),A(0)
LAST   DC   CL8'FVRDBK       ',V(FVRDBK),A(0)
      END  FVSAM

```

References:

IBM Manuals:

GC26-4015 MVS/XA VSAM Administration Guide
 GC26-4016 MVS/XA VSAM Administration: Macro Instruction Reference
 GC26-4019 MVS/XA Access Method Services Reference
 SC26-4118 VS FORTRAN Programming Guide, Release 4.1
 SC26-4119 VS FORTRAN Language and Library Reference, Release 4.1
 SH20-9218 APL2 Programming System Services Reference, Release 2

Other:

Using Quad-FX to Facilitate the Use of Auxiliary Processors,
 by David Piper
 (Vector, Vol 3 No. 2, function listings in Vol 3 No. 3)

Getting Data from Vax/Oracle to APL*PLUS/PC

by Adrian Smith

Introduction

Judging by the popular computing press, the flavour of the month in the database world is undoubtedly Oracle, and its favoured environment is the Dec Vax range. At the same time, it has become very clear that the mini-computer will never provide the kind of user-interface that PC users have come to expect. The conclusion of this article is that APL is one tool which just might come to bridge that gap.

With APL, you can build on the concept of the 'workstation', where the Vax is essentially just a database engine (a job it does rather well), with your PC acting as an 'intelligent agent' rather than just a dumb terminal. These notes describe the low-level interface which underpins a plant information system, based on the IBM PS/2-50 (complete with hypertext and windows) and an Oracle database held on a MicroVax-II.

The PC side of the system is entirely written in APL*PLUS/PC, and uses the standard RS232 port to drive a serial line to the Vax. For maximum security of data (not needed in an 'enquiry only' system) you might want to use Ethernet and Decnet-DOS. This would require some coding in C to drive the Ethernet link, and at the moment I have no idea how I would set about this! In terms of response time, simplicity, and ease of installation, I think the simple serial link wins every time.

Making Contact

APL has a number of rather handy add-ins tucked away: one of these is a basic terminal emulator. As I noted in my review of APL*PLUS/PC Rel.7, I have little or no faith in the VT100 mode now offered; maybe you will have better luck! The simple cover function TALK is all you need to get started.

You could dispose of the □CMD into your AUTOEXEC.BAT to save a bit of time.

R←TALK;WAS;JUNK;POKED

```
[1]  A Set up terminal for use with serial connection
[2]  '===== TERMINAL: Use Alt/F8 then <Enter> to exit ====='
[3]  A POKE: FULL DUPLEX; TEXT KBD; BACKSPACE; ALT-HOME[2]
[4]  POKED← 115 118 172 174 175
[5]  WAS← 1 1 127 48 48 □POKE POKED
[6]  JUNK←□CMD 'MODE COM1:9600,N,8,1 >NUL'
[7]  'Press <Enter> to connect ...'
[8]  JUNK←1 □POKE 117 □ JUNK←□
[9]  JUNK←WAS □POKE POKED
[10] R←'End of Terminal Session'
```

▽

While you are testing things out, you can login by hand, but for comfort and convenience you will soon need a simple script such as the function below:

R←LOGIN;CHK

```
[1]      A Run login script starting where necessary ...
[2]      R←'' □ CHK←RSΔCHK □ →0 IF 'SQL>'^.=4↑CHK
[3]      CHK←RSΔFLUSH □ CHK←RSΔCHK □ CHK←□DL 1 □ CHK←RSΔCHK
[4]      →0 IF 'SQL>'^.=4↑CHK
[5]      A Keep checking prompt until we get to SQL>
[6]      →Tsrv IF 'Local>'^.=6↑CHK
[7]      →Vax IF 'User'^.=4↑CHK
[8]      →VMS IF '$'=1↑CHK
[9]      →Dead IF '!!!'^.=3↑CHK
[10]
[10]     A Unknown prompt .... give up gracefully ....
[11]     R←'*** Unknown response .../','CHK, '/LOGON terminated'
[12]     R←'/' ΔTAB R □ →0
[13]     Dead:R←'*** Connection to database failed ***' □ →0
```

```

[14]      A Take system forward from recognised prompt ...

[15] Tsrv:'Login to Decserver ... '
[16]      'name: ' RSΔSR 'CONNECT MYVAX' A      ... DEC nodename
[17] Vax:'Signon to Vax ....'
[18]      R←'word: ' RSΔSR 'SMITH' A      ... Username
[19]      '$ ' RSΔSR 'SECRET' A      ... Password
[20] VMS:'Starting SQL*PLUS from VMS .... '
[21]      R←'$ ' RSΔSR 'SET TERM/UNKNOWN'
[22]      R←'$ ' RSΔSR 'SET TERM/WIDTH=256/NOTAB/ECHO'
[23]      'SQL> ' RSΔSR 'SQLPLUS SCOTT/TIGER'

[24]      A Check that we have reached SQL ... logoff if not!

[25]      CHK←RSΔCHK ◊ →Exit IF 'SQL>'^.=4↑CHK
[26]      R←LOGOUT ◊ →0
[27] Exit:R←'Logged in to SQL*PLUS ...'
[28]      CHK←'SQL> ' RSΔSR 'SET SPACE 0'
[29]      CHK←'SQL> ' RSΔSR 'SET VERIFY OFF'
[30]      CHK←'SQL> ' RSΔSR 'SET HEADING OFF'
[31]      CHK←'SQL> ' RSΔSR 'SET LINESIZE 480'
[32]      R←R CAT SQLΔSYNCH

```

▼

This calls a number of lower-level functions (anything beginning RΔ) to drive the serial port; these will be explained in detail in the next section. Other points worth noting are:

- it flushes the port, and then gives the Vax a couple of pokes with a second's delay in between. This is overkill, but reliable.
- having got through to VMS, it ensures that the terminal is correctly set up. Note the "width=256" setting, all the others are defaults and are just to make sure that no-one else has been there before you.
- having reached SQL*PLUS, you again set some parameters. If you have an SQL id to yourself, you could put all these into LOGIN.SQL and save a bit of to-ing and fro-ing on the line.
- SQLΔSYNCH sets the PC clock to the same time as the clock on the Vax. You probably don't need to do this.

Logging out is much more straightforward. Again you need to find out where you are, and then just send the sign-off commands:

R←LOGOUT;CHK

```
[1]      A Run logout script ...

[2]      Chk:CHK←RSΔFLUSH ◊ CHK←RSΔCHK ◊ CHK←DDL 1 ◊ CHK←RSΔCHK
[3]          →Exit IF 'Local>'^.=6↑CHK
[4]          →Exit IF 'User'^.=4↑CHK
[5]          →SQL IF 'SQL>'^.=4↑CHK
[6]          →VMS IF '$'=1↑CHK
[7]          →Dead IF '!!!'^.=3↑CHK
[8]      SQL:'Logout from SQL*PLUS ... ' ◊ '$' RSΔSR 'EXIT'
[9]      VMS:'Exiting from VMS .... ' ◊ '' RSΔSR 'logoff'
[10]     Exit:R←'Logged out ' ◊ →0
[11]     Dead:R←'***'
      ▽
```

Finally, just in case you do want to fire queries automatically 'on the hour', or simply display an accurate clock on the screen:

R←SQLΔSYNCH;MAT;DTE;DAY;MON;YR;HR;MIN;SEC

```
[1]      A Synchronise PC clock with VAX time ...
[2]      A Note that the seconds are deliberately random, to ensure
[3]      A that queries run 'on the hour' are staggered.
[4]      A Uses preset query QD which gives 'DD MM YY HH MM SS'
[5]      A

[6]      MAT←START 'QD' ◊ R←MAT ◊ →('*'=1↑,MAT)↑0
[7]      MAT←20↑MAT[1;] ◊ DTE←MAT
[8]      DTE[(~DTE<'0123456789')/⌊DTE]←' '
[9]      →(DTE^.= ' ')↑0 ◊ DTE←DTE

[10]     A Use DINT to set DOS system date ...

[11]     DAY←DTE[1] ◊ MON←DTE[2] ◊ YR←1900+DTE[3]
[12]     R←(11008 0 ,YR,DAY+256×MON)DINT 33

[13]     A Use DINT to set DOS system clock ...

[14]     HR←DTE[4] ◊ MIN←DTE[5] ◊ SEC←DTE[6]
[15]     SEC←~1+?60
[16]     R←(11520 0 ,(MIN+256×HR),256×SEC)DINT 33
[17]     R←'PC clock reset to ... ',MAT
      ▽
```

At least all your workstations will agree, even if they are all wrong! One thing I have found useful is a regular check on a system status file (say every 15 min); this normally just says "available", but can be set to things like "shutdown", in which case the workstation will log itself off, even if it is running unattended in a far corner of the factory!

Driving the Serial Port

This is one of those areas where there is very little code, but it has to be right! Everything in the system depends absolutely on these few functions, so I can vouch that they have been rather well tested over the past year or so!

The first essential is to know to whom you are speaking! The easy way to find this out is to hit <Enter> and have a look at the prompt that you get back:

R←RSΔCHK

```
[1]      A Check where we are in the system ....
[2]      RSΔOUT ' ' ◇ R←RSΔFLUSH ◇ R←(~R←[AV[1 8]])/R
[3]      R←TAB R ◇ R←(Rv.#' ')/R ◇ →(0←ρR)↑Dead
[4]      R←R[(ρR)[11];] ◇ R←(R#' ')/R ◇ →0
[5]      Dead:R←'!!!!'
      ▽
```

If you are in VMS, it will come back with "\$", and so on. It clears the input line with RSΔFLUSH, which just keeps collecting characters as long as they keep coming, and uses TAB to kill line-feeds and tabulate on [TCNL:

R←RSΔFLUSH;IN;CT

```
[1]      A Clear anything currently on the line
[2]      CT←1 ◇ R←''
[3]      A Get input buffer. Max 2100 chars ...
[4]      Wait:IN←(0 1 2 2 0 2100)▯ARBIN ' '
[5]      IN←~1↓IN ◇ R←R,IN ◇ →(ρIN)↑Wait
[6]      →(ρR)↑0
[7]      →(50<CT+CT+1)↑Dead ◇ →Wait
[8]      Dead:R←'!!!!'
      ▽
```

(See page 132 for a listing of TAB, and other utility functions.) The process of sending data down the line is not quite as simple as it looks; here is `RSΔOUT`, which should be a one-liner, but isn't:

```
RSΔOUT XX;IN;LEN
-----
```

```
[1]      A Send text to serial port, no reply required.
[2]      XX←XX ◊ LEN+40 ◊ →(LEN≥ρXX)↑Tail
[3]      Send:IN← 1 0 2 0 ◊ARBIN LEN↑XX
[4]      XX←LEN↓XX ◊ →(LEN<ρXX)↑Send
[5]      Tail:IN← 1 0 2 0 ◊ARBIN XX,CHR 13
      v
```

The snag is that if you send too many characters too fast, you start getting batches of BEL characters (◊AV[8]) in the echo. Eventually the Vax gives up on you and just ignores the input altogether. By looping in 40-character chunks you give it time to absorb the input at a sensible rate. Owners of very fast PCs may have to shove a bit of extra delay in here ... I suggest putting a BEL-detector into `RSΔIN` and doing some experiments!

Having fired off your command with `RSΔOUT`, you need a reliable way of knowing that you have collected the entire response. Again, you can look for the characteristic prompt, and simply collect everything that comes up the line until you get a match. It pays to have a timeout as well, just in case your Vax has died, or someone has pulled a plug:

```
R←PARAM RSΔIN MAX;XX;CT;PARAM:FLG;IN;BL
-----
```

```
[1]      A Hang about until something comes up the line.
[2]      A When it does, check for complete message, then return it.
[3]      A <PARAM> is the delimiter string to collect up to ...
[4]      A <MAX> is the number of cycles to wait, before giving up.
[5]      A
[6]      CT←1 ◊ R←'' ◊ BL← 24 70 1 8 ◊WGET 1
[7]      A Get input buffer. Max 2100 chars ...
[8]      Wait:IN←(0 1 2 2 0 2100)◊ARBIN ''
[9]      IN←~1↓IN ◊ FLG←(-12+ρ,PARAM)↑R,IN ◊ →(ρIN)↑Appd
[10]     Null:CT←CT+1 ◊ 24 70 1 3 ◊WPUT 3 0 ◊CT
[11]     →(CT≥MAX)↑Timeup ◊ →Wait
```

```

[12]      A Add input to text buffer, check if it contains target
[13]      A string and go back for more if not.

[14]      Appd:R←R,IN ◇ 24 73 1 5 □WPUT 5 0 $pR
[15]      →(V/FLG □SS PARA)↑Done
[16]      CT←1 ◇ →Wait
[17]      Timeup:→(pR)↑0 ◇ R←'!!!' ◇ →0
[18]      Done: 24 70 1 8 □WPUT BL
      ▽

```

This runs a couple of counters in the lower right corner of the screen, viz the timeout counter and the number of characters collected so far. These were originally put in to help me during debugging, but I have left them in the finished system as they give users some idea of how their request is progressing.

Finally, the workhorse routine RSASR, which sends the command, checks the echo, and returns the tabulated result (obviously less the echo!):

```

R←PARA RSASR TXT;RSP;LEN;CHK
-----
[11]      A Pass text to RS232 and collect reply up to PARA
[12]      A Global ΔTIMEOUT controls clock cycles to wait ... say 500

[13]      TXT←,TXT ◇ PARA←,PARA ◇ LEN←pTXT
[14]      RSΔOUT TXT ◇ RSP←PARA RSΔIN ΔTIMEOUT
[15]      RSP←(~RSP□DAV[1 8])/RSP

[16]      A Check that echo really was ....

[17]      CHK←LEN↑RSP ◇ →(^(CHK←' $',ΔCRLF)✓CHK=TX)↑Ok
[18]      '***** POSSIBLE DATA ERROR ..... ' ◇ TXT ◇ CHK ◇ ' '
[19]      R←BAREIN 'Press <ENTER> to proceed ....'
[101] Ok:R←TAB LEN↓RSP
      ▽

```

What you do with funny data is up to you! This just reports the problem and continues; until I see this happen I refuse to worry about it! The one catch is from the "Password: " prompt, where you don't get any echo of your password (see LOGIN) - hence the escape clause on line[7].

As I noted in the introduction, there is no integrity check on received data. One easy thing to do is to set the Vax to transmit on even parity; you then need a version of □AV

with all the odd characters displaced by 128. Everything you receive should occur in this, and a quick look-up and index will get you back to the bottom 128. Of course you will need to shift outgoing characters the other way so that the Vax only sees even parity at its end!

Preparing and Sending an SQL Query

I think I will again start at the top level function, and work down; this after all is the way the workspace was written! The basic requirement is to pass a character vector to SQL, and get usable data back into your workspace. An example could be:

```
MAT←SQL 'Select ename,deptno,salary from emp'
```

```
      MAT
JONES | 20| 12876|
HARRIS | 20| 7890|
KING   | 50| 76000|
```

```
SQL>
```

This differs in two ways from the default format returned by SQL. Firstly there are no headings, and no space between columns; secondly, there is a delimiter character between each column, and a closing delimiter on the end. The reason for the lack of headings is that you will want to do your own anyway, and they can play havoc with the column widths. Also, at 9600 baud, the less junk you send down the line the better!

The delimiters are simply to make it easy for you to split out the columns as separate APL variables; there are numerous ways of doing this, but an unambiguous set of markers is obviously a sensible starting point.

I hope that most if this code is self explanatory; the detail of the called functions follows:

R←FLG SQL QRY;lab;CT;POS;PMPT;LSIZE;FM;HARDCR

```

[1]      A Send query to SQL and return data ...
[2]      A FLG defaults to 1 ... query may be re-wrapped if so.

[3]      R←LOGIN ◊ →(''=1↑,R)↑0
[4]      LSIZE←160 ◊ ⚡(0=UNC 'FLG')/'FLG←1'

[5]      A Skip wrapping if FLG is -1
[6]      →(FLG=-1)↑Lp

[7]      A Compress out spare blanks, and ravel. Sets HARDCR.
[8]      QRY←SQL△PREP QRY

[9]      A Take query and pass to SQL in bite-size lines.
[10]     A Response of 'ln-no' is awaited, and checked.

[11]     QRY←LSIZE SQL△WRAP QRY

[12]     A Peel any COL/BREAK/COMPUTE commands off the front ...

[13]     LpC:→lab+(ρHARDCR)ITERATE EndC,CT+1
[14]     PMPT←'SQL> ' RSΔSR DTB QRY[HARDCR[CT];]
[15]     EndC:→lab[CT+CT+1]
[16]     QRY←(∼(1↑ρQRY)◊HARDCR)≠QRY
[17]     Lp:→lab+((1↑ρQRY)-1)ITERATE End,CT+1
[18]     PMPT←5↑ 3 0 ⚡CT+1 ◊ PMPT←PMPT RSΔSR DTB QRY[CT;]
[19]     End:→lab[CT+CT+1]
[20]     QRY←QRY[CT;]

[21]     A Pass last line, and capture result of entire query ...

[22]     Tail:R←'SQL> ' RSΔSR DTB QRY
        V

```

Note that we start with a LOGIN; most of the time this will just get the "SQL> " prompt straight back, and do nothing. However it is always worth checking that your Vax is still there! Assuming you have omitted the left argument, the function now sets about minimizing the number of characters in your query. The first stage is to take out any nice indentation, and to ensure that the obligatory semi-colon is there at the end:

R←SQLΔPREP QRY;POS;MSK;CHK

```

[1]      A Prepare a query for transmission
[2]      A Ensure that SQL commands (e.g. COL) go on a separate line.
[3]      A records these lines in <HARDCR> ....

[4]      HARDCR←0 ◇ →(2>ρρQRY)↑Vec
[5]      CHK←ΔUPPER QRY[;13]
[6]      POS←(V/CHK^.=0 3 3 ρ'COLCOMBRE')/11↑ρQRY
[7]      QRY←QRY,' ' ◇ QRY[POS;(ρQRY)[2]]←'r'
[8]      HARDCR←POS
[9]      QRY←~1ΔNEATEN(,QRY),' '
[10]     Vec:QRY←QRY SQLΔSSR 'from◇FROM'
[11]     QRY←DLB QRY
[12]     POS←(QRY USS 'FROM')/1ρQRY ◇ →(ρPOS)↓Skip
[13]     →('DEL'^.=3↑QRY)↑Skip
[14]     →('INSERT'^.=6↑QRY)↑Skip

[15]     A Insert ':' between columns to give delimited result ...
[16]     POS←POS[1] ◇ QRY←((POS↑QRY)SQLΔSSR ',◇','|',''),POS↓QRY

[17]     A Add a closing delimiter unless 'Select *'
[18]     →('*'∈8↑QRY)↑Skip
[19]     QRY←QRY SQLΔSSR ' FROM◇ ','|' FROM'

[20]     A Ensure it ends with a semi-colon ...
[21]     Skip:R←QRY,(';'≠~1↑QRY)/';'
      ▽

```

There is also some subtle patching of the query, to ensure that you get those column delimiters returned to you. We look for the word 'from' as in:

Select empno,dept,salary from emp;

... and patch in the delimiter symbol wherever we find a comma:

Select empno,'|',dept,'|',salary,'|' from emp;

... and at the end of the list for good measure. Commas within '()' are left well alone, otherwise expressions like:

Select tochar(date,'DDMMYY'),info from junk;

... are well and truly wrecked!

```
R←OLDSTR SQLASSR CMD;s1;s2;d;p
-----
```

```
[1] A SQL Replace strings within OLDSTR.
[2] d←' ' ⋄ R←OLDSTR
[3] →(ps1←(1+p←CMD)d)↑CMD)↓0
[4] s2←p↓CMD
[5] →(pp←(OLDSTR DSS s1)/1pOLDSTR)↓0
[6] p←(1,(ps1)≤1↓p-1↓p)/p

[7] A Ignore strings which fall within parentheses ....
[8] →(pp←(~p∈(≠\OLDSTRε'()')/1pOLDSTR)/p)↓0
[9] R←(~(1pOLDSTR)ε1+,pε.,1ps1)/OLDSTR
[10] p←(p-0,+1(p)ε(pps1)-ps2)ε.,+1+1ps2
[11] R←(~(1(pR)+(1↑pp)×ps2)ε,p)\R
[12] R[,p]←(p,p)ps2
▽
```

The resulting vector can now be re-wrapped into a reasonable line length. Anything up to the terminal width is OK. Note the one catch, which is that any SQL*PLUS commands (which are simply formatting instructions and have nothing to do with SQL itself) must go on separate lines.

```
PARA←WID SQLΔWRAP QRY;POS;WID
-----
```

```
[1] AREWRAP SQL QUERY TO FIT GIVEN WIDTH.
[2] A Targets for line breaks are ' , '
[3] A Lines ended with 'r' are preserved

[4] PARA←(0,WID)p' '
[5] Loop: POS←(1+WID)1/(ΦWID↑QRY)1', '
[6] POS←WID-POS-1 ⋄ POS←POS11+(POS↑QRY)1'r'
[7] PARA←PARA;WID↑POS↑QRY
[8] QRY←POS↓QRY ⋄ QRY←(+//^(5↑QRY)ε' r')↓QRY ⋄ →(0≠pQRY)↑Loop
▽
```

The commands and lines of SQL are then sent to SQL*PLUS; this returns a "line-number" prompt until it sees the final semi-colon. Hence the check on line[18] that we have got the prompt back before the next line is fired off. The final line closes the query, and we get the full result back, along with a final "SQL> " prompt.

What you do to handle possible errors is up to you. I tend to check the result with a function like:

```
RC←SQLΔCHK R
-----
[1] A Check for SQL errors
[2] RC←1
[3] →(R ΔIOTA 'ERROR')↑0
[4] →(R ΔIOTA 'unknown')↑0
[5] RC←2
[6] →(R ΔIOTA 'no rec')↑0
[7] RC←0
    ∇
```

... and if I get a zero back, I know I can strip the last two lines which will simply be a blank, followed by the "SQL> " prompt. On the other hand something like "no records selected" may not be an error at all, so you can simply hand it back to the user. Real problem messages like "ERROR: table or view does not exist" should obviously be reported back, with some helpful packaging, such as the phone number of your help desk!

Capturing the Result as APL Variables

This is the final stage, which will let you interface SQL fully into a PC-based system. Of course you will not have nested arrays to play with, but you can get very similar results with data-handling functions such as my "DB-385" utilities. These run off simple tables of variable names, such as:

```
DBDEFN
ΔNAME CH[20] .... name
ΔDEPTNO NUM .... department number
ΔSALARY NUM .... gross salary
```

```
    MAT
JONES      | 20| 12876|
HARRIS     | 20| 7890|
KING       | 50| 76000|
```

SQL>

```
    DBDEFN SQLΔSPLIT MAT
    ρΔNAME
3 20
    +/ΔSALARY
96766
```

Here are the two remaining functions to carve up your result array into variables:

R←DBDEFN SQLADB QRY;RC;REPLY

```

[1]      R Send query to SQL and assign into the variables in DBDEFN
[2]      R
[3]      REPLY←O SQL QRY
[4]      RC←SQLCHK REPLY ⋄ →(Err,Empty)IF 1 2 =RC
[5]      DBDEFN SQLASPLIT REPLY
[6]      R←'OK' ⋄ →0
[7]      Err:R←'*** SQL ERROR: ',,REPLY ⋄ →0
[8]      Empty:R←'*** No records selected ***'
      v

```

NL SQLASPLIT MAT;POS;LEN;DPTH;TYPE;lab;FORM

```

[1]      A Split out columns of table into variables.
[2]      POS←MAT[1;1] ⋄ MAT←(MAT[;POS]=')')/MAT
[3]      POS←(MAT[1;]=')')/1~1⊖MAT
[4]      LEN←~1+~1⊖(POS,0)-0,POS
[5]      POS←~1⊖0,POS
[6]      A Convert NL to the correct format and note whether number,char,dat
[7]      NL←(~2↑ 1 1 ,⊖NL)⊖NL
[8]      WIDTH←(~1↑⊖NL)⊖( ' ^.=NL)⊖1 ⋄ DPTH←(1↑⊖NL)⊖⊖POS
[9]      TYPE←'NCD'⊖,(DPTH,1)↑ΔLJUST(0,WIDTH)⊖NL
[10]     NL←(DPTH,WIDTH)↑NL
[11]     FORM← 4 9 ⊖36↑',ΔMATEXEC'
[12]     Lp:→lab←(⊖POS)ITERATE End,CT+1
[13]     ⊖NL[CT;1,'←',FORM[TYPE[CT];1,' MAT[;',(⊖POS[CT]),'+⊖',(⊖LEN[CT]),'
[14]     End:→lab[CT+CT+1]
      v

```

A more general strategy would be to ask SQL to "describe" the table, and then use this information to break the data down. I have so far avoided this, simply to cut down the amount of data flying up and down the line.

Summary

Using these functions, or something like them, you can seriously contemplate harnessing the power of APL and the PC environment to provide a far more flexible and easy to use route to SQL-based data than anything currently available in packages like SQL*FORMS.

Appendix: Other Useful Functions.

Here is a selection of the utilities used in the system. Please excuse any missing ones, I have long ceased to notice such things in my code; they just get used as if they were part of the language!

```

MAT←TAB VEC
-----

[1]  A SET UP RETURNED DATA AS TABLE
[2]  A Kill line-feeds, and leading CRs

[3]  VEC←(VEC#QAV[111])/VEC
[4]  VEC←(+//^(12↑VEC)=UTCNL)↓VEC
[5]  MAT←UTCNL ΔTAB VEC
    ▽

R←A CAT B;MAX
-----

[1]  A Superimpose any two char things

[2]  A←(¬2↑ 1 1 ,ρA)ρA ◇ B←(¬2↑ 1 1 ,ρB)ρB
[3]  MAX←(¬1↑ρA)↑¬1↑ρB
[4]  R←(((¬1↓ρA),MAX)↑A);((¬1↓ρB),MAX)↑B
    ▽

R←DTC MAT;WID
-----

[1]  A Delete trailing cols

[2]  WID←+/v\Φ' 'v.≠MAT
[3]  R←((1↑ρMAT),WID)↑MAT
    ▽

R←DTR MAT;LEN
-----

[1]  A Delete trailing rows

[2]  LEN←+/v\ΦMATv.≠' '
[3]  R←(LEN,(1↓ρMAT))↑MAT
    ▽

R←DTB VEC
-----

[1]  A Delete trailing blanks from VEC

[2]  R←(+//v\Φ' '≠VEC)↑VEC
    ▽

```

Interfacing APL to the Apple Macintosh

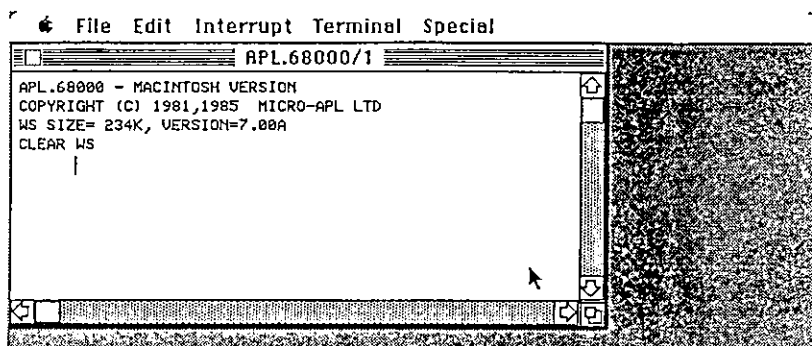
by Richard Nabavi
MicroAPL Ltd

As computers have evolved since APL was first implemented in the sixties, APL interpreter writers have developed the human interface to APL in line with the hardware and systems software available to them. Whilst modern APLs still include features which were originally designed for the old IBM 2741 printing terminals, users are now more likely to rely on sophisticated full-screen editors and session managers than to employ exclusively the old del editor and line-by-line editing of the early systems.

However, it is still true today that programming in APL is done mainly using tools which are based on VDUs or PC screens which typically have about 25 lines and 80 or 132 columns, with a 'modal' interaction where the system presents a single-tasking interface to the user. In these systems, you are perhaps either editing a function, or in desk-calculator mode, or are running a program, and you have to finish one thing before switching mode to do something else. But more up-to-date computers such as the Apple Macintosh have the capability to offer much more sophisticated and flexible user interfaces, and APL has to adapt if it is to take advantage of the increased capabilities which are today on offer.

For the past couple of years, MicroAPL has been working on versions of APL which do employ the conventions and exploit the power of the latest generation of powerful micro-computers. Our first product based on this was APL.68000 for the Apple Macintosh, and this was followed by implementations for the Atari ST and the Commodore Amiga. Not surprisingly, when you have to do something three times, you learn quite a lot along the way, and we felt that our Amiga implementation of APL was considerably better than our initial Macintosh version. We have therefore gone back and re-written a considerable part of the user interface of the Macintosh version of APL.68000, and this article is designed to share some of the insights which we believe we have gained in this iterative development process.

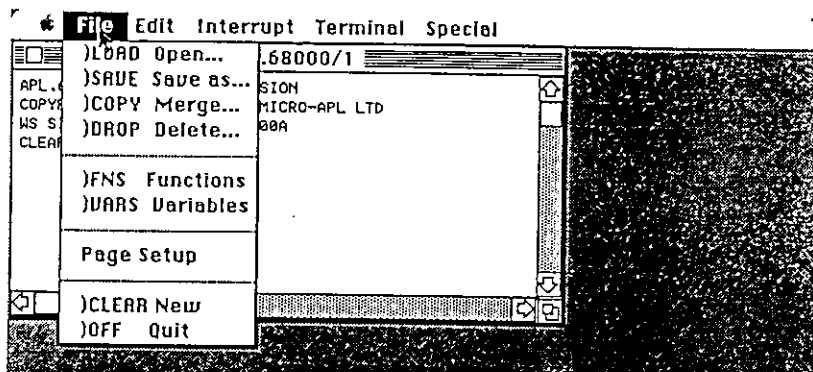
Unlike a conventional computer terminal, a Macintosh task displays results to the user within a window, such as this one:



The row of names at the top (the "Menu bar") is used, in conjunction with a mouse, to control the environment. More of that later. The window itself consists of a central area where text and graphics are displayed, a title (here "APL.68000/1") and a number of controls. These include the "Close box" in the top left of the window, the Scroll bars along the bottom and right-hand edges of the window, and the Size box which is at the bottom right-hand corner.

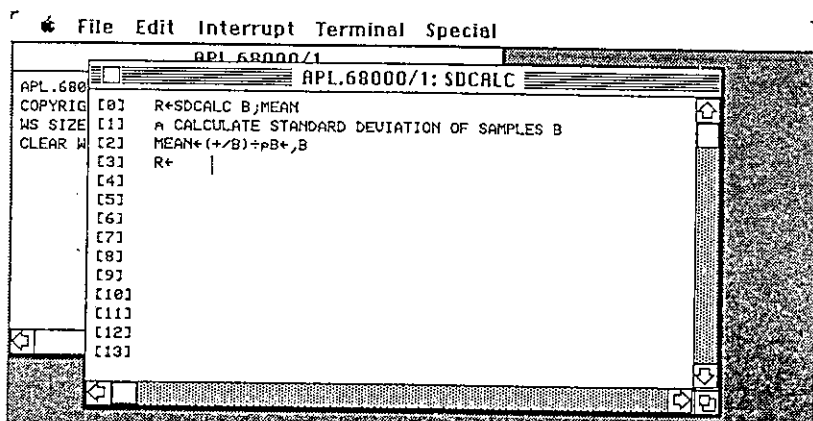
A Macintosh window can be regarded conceptually as a view of part of a larger logical area of text and graphics. By dragging the Size box with the mouse, you can increase or decrease the window size, thus revealing more or less of the data underneath. Using the Scroll bars, you can move the window over the data. Thus, APL.68000's initial output (as shown in the above example) appears in a Macintosh window, where the user can scroll backwards and forwards, as well as horizontally, to view the results which APL has displayed.

As well as controlling the window itself, the user can use the mouse to open up Menu options in the Menu bar:

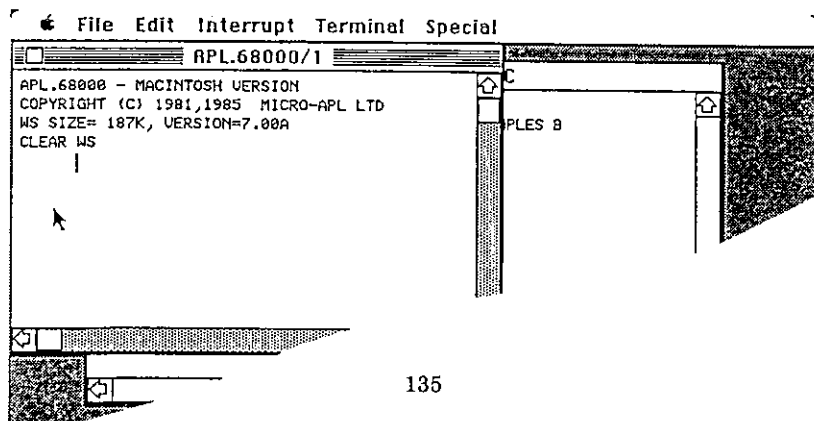


Here we show the File Menu, which allows familiar system commands such as)SAVE and)LOAD to be performed with much less typing effort than APLers are used to. If you select the)LOAD menu item, a special dialog box (a particular kind of window) will appear, prompting you with a display of available workspaces. Although people who are unused to using a mouse take a little while to get used to it, most people find that (at least for this kind of selection), it is very fast and convenient indeed - a couple of clicks is all that is needed to load a workspace or carry out other, more unusual, commands.

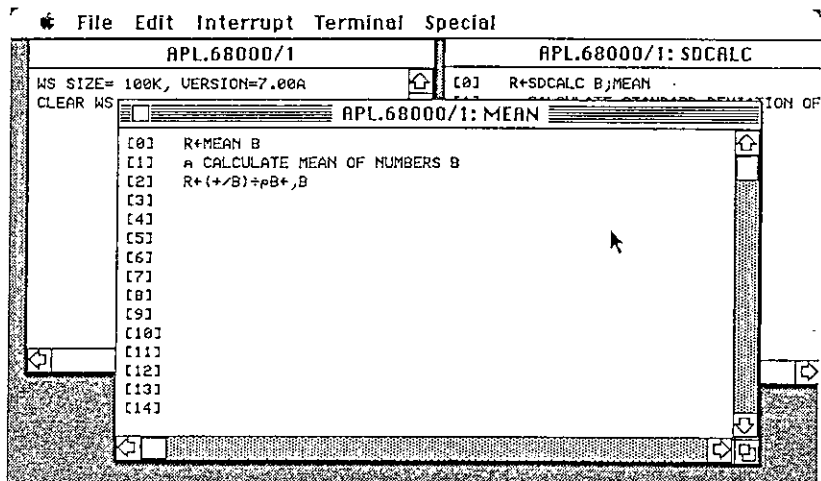
Of course, the windowing environment is not really very interesting until you start having several windows on the screen. Opening a function (or variable) for editing is an obvious example:



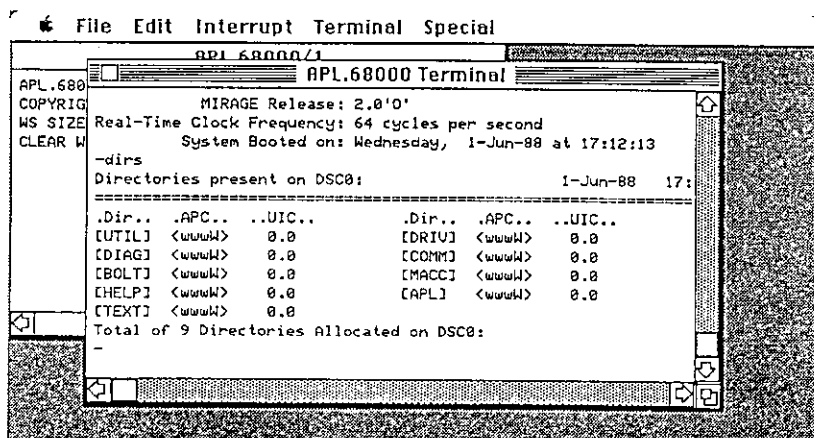
Here, we have used the Edit menu to open a new function called SDCALC. Notice that a new window has opened for the editing process, but the APL.68000/1 window is still there in the background. In fact, if we click the mouse on the APL.68000/1 window, it will come to the front, and we could if we liked enter any APL command:



The function SDCALC is still sitting in its window, and in fact it is very easy to use this approach to try out APL expressions in desk-calculator mode, and use the excellent Macintosh "Cut and Paste" facility to paste them into the APL function editor without re-typing. And of course you don't need to restrict yourself to editing only one function at a time, for example if you wish to move some code from one function to another:



Multiple windows are also very handy if you are communicating with a remote mainframe or super-micro computer. Using the Terminal menu, you can literally open a window on the wider world outside your Macintosh:



Naturally, you can work on your own local APL session in the APL.68000 window whilst simultaneously keeping an eye on what the mainframe is doing, perhaps keeping the Terminal window very small whilst you wait for that SQL query to complete...

Technical digression: Getting several quarts out of one pint pot

The above description and screen shots show that the use of multiple windows allows you to appear to be doing several things simultaneously. In fact, though, the Macintosh was, until recently, a single-tasking machine. Apple have partially addressed the issue of multi-tasking with a product called MultiFinder, but the facilities provided by this are rather limited. And yet, it has always seemed to us that a single-tasking system could not really exploit the true potential of a windowing environment, and our experience of the Amiga (which is a true multi-tasking machine) confirmed how useful this is for APL. So how is it that on a single-tasking machine you can simultaneously edit several functions, execute APL expressions, and run a terminal emulation task in another window?

The answer is that, to understand the Macintosh and its competitors, you have to invert the standard view of how an application (such as an APL interpreter) runs on the machine.

In a conventional machine, such as an IBM-AT running PC-DOS, an APL interpreter would execute an expression, and then call a sub-routine to get the next line of input. That sub-routine would call the operating system to get characters from the keyboard. When the line of input was complete, the subroutine would exit and control would return to the interpreter.

In a typical Macintosh application, the main control is the "Event Loop". The application loops round asking the operating system what "events" have occurred. Examples of events are:

- A key has been pressed
- The mouse has been clicked in a menu bar
- A disk has been inserted
- A window, belonging to the application, needs updating because the user has enlarged it, or another window has disappeared from in front of it.

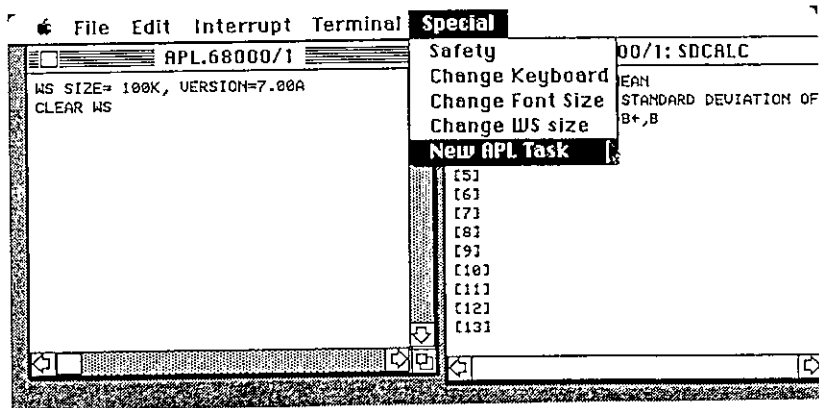
Most events are associated with a window. For example, a key has been pressed, but which window is selected (ie front-most and the one which the user thinks he is working in)? According to which window it is, the key might mean a character to be sent to the remote mainframe, or a character to be inserted in a function in the

full-screen editor, or a character to be assembled into an APL expression to be submitted to the APL interpreter for evaluation. In this way of looking at things, the APL interpreter is a subroutine which is called when a line of user input needs to be executed, and this is just one of the kinds of event which need to be handled.

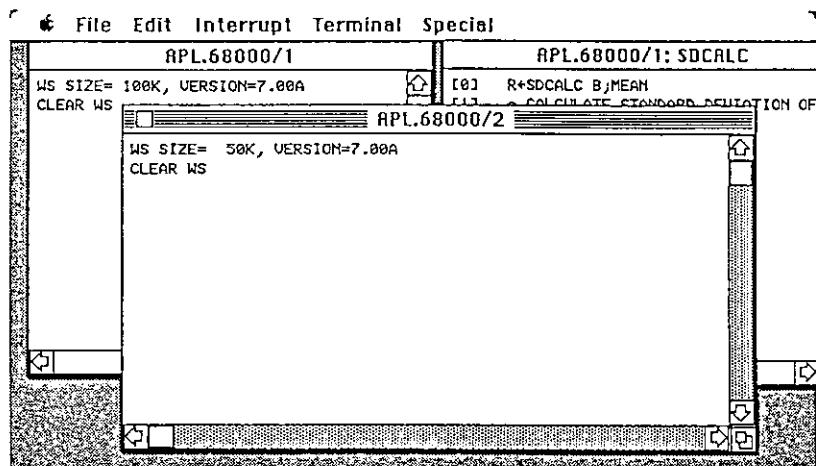
This in fact is the logical structure used by APL.68000 on the Macintosh. Of course, it is not really that simple, and in fact event handling accounts for about a quarter of the total code size. In particular, calling the APL interpreter as a subroutine is fine, except that the user may have requested a matrix invert of a 100 by 100 array, and that takes rather a long time. Fortunately, APL periodically checks for attention (break) while it is executing, and this attention check call can be used to run through the event loop while APL is in control. Hence, the apparently "simultaneous" operation in several different windows is the result of responding many times per second to events which occur in different contexts.

If we can multi-task, why not multi-task?

As con-men since time began have always known, if a trick works on one sucker, it will work on another sucker. And so, since we can use the event loop to achieve effective multi-tasking between APL, the editing windows, and the terminal emulator, why not use the same method to achieve multi-tasking in the sense of running several APL tasks simultaneously? By doing this, the full potential of the windowing environment is reached, in a way which transcends the limited multi-tasking support of MultiFinder. So, the Special menu now includes an item which makes another APL window appear:



The new APL task can run simultaneously with other APL tasks, and the user directs input at the task he wishes by selecting the appropriate window with the mouse:



So, with Version 7 of APL.68000, the Macintosh interface is integrated with APL to the extent that (if you have enough memory), you can have up to four APL tasks, each in its own window, and each task can have associated with it one or more editing windows (up to four in all), and in addition there might be a terminal emulation window, a dynamic map of the keyboard layout (which is user-definable), and, if that is not enough, up to ten user-defined windows defined by the APL application. The different APL tasks can share variables with each other, and can also work in background mode without a window at all (unless an error occurs, in which case a window appears to show the user the error).

Now, incidentally, you can see why the initial APL window is titled "APL.68000/1", and also why editing windows have the APL.68000 task number as well as the object name in the title. Otherwise, it does get very confusing!

And the rest of the interface...

I have concentrated above on APL's use of windows on the Macintosh, but there is a lot more which I do not have space to elaborate on. This includes changing font sizes, user-defined dialog and alert boxes, the Quickdraw interface, cutting and pasting with other (non-APL) applications, and implementation of the sound synthesizer (it was a fine moment when Bach's *Crab Canon*, downloaded from an Amiga, was first heard on the Macintosh - see *Vector* Vol 1 No. 4. P60). Above all, perhaps, there is the complex and interesting problem of how you allow APL applications to take advantage of all these features, whilst APL.68000 itself is in control of the machine. But that forms the subject of another paper - which will be presented at APL-ication in September.

**British APL Association
Software Library****Software Submission (1)**

Please copy and fill in this form for EACH disk you submit.

Details corresponding to items flagged (*) will NOT be made publically available, but are for our records only. Please Use BLOCK CAPITALS for all items except number 18.

0. Submission date: _____
1. Name of donor: _____ 2. Daytime phone(*): _____
3. FULL address(*): _____

4. Electronic mail addresses(*): _____
5. Short disk title: _____
6. Brief description of disk : _____

7. List target machine: _____
8. List additional software required: _____
9. Indicate special hardware requirements: _____
10. Is user documentation provided on the disk?(Y/N): _____
11. List titles of any paper documentation included with your submission

12. Is this documentation, or any other, available to users upon
application to you?(Y/N): _____ Please give details: _____

13. Does the disk include any form of payment request from users of the
software?(Y/N): _____ Please give details: _____

14. Does your submission constitute a 'demonstration disk' in that it
demonstrates software that is available for purchase?(Y/N): _____
15. If the TARGET machine is NOT a DOS-based PC, does the disk include
instructions for transfer to the target machine?(Y/N): _____

British APL Association Software Library

Order Form

To: Dave Ziemann
Flat 3, 63 Queens Crescent
London NW5 4ES, ENGLAND

PLEASE SUPPLY THE DISKS CIRCLED BELOW:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35
36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53
54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71
72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89
90	91	92	93	94	95	96	97	98	99	100							

TOTAL NUMBER OF DISKS ORDERED (please double-check): _____

at 2 Pounds per disk : _____ BAA Member's rate

OR at 3 Pounds per disk : _____ Non-member's rate

I want to join the BAA, and enclose
the 10 Pounds annual membership fee : _____

Postage and handling : _____ 1.00

Add 2 Pounds for orders outside the UK: _____

Total order - remittance included: _____

Your signature: _____

Your name: _____

Full address: _____

Please make cheques payable to the British APL Association.

Payment can be made in US Dollars - pay 1.5 US Dollars for each Pound.

All orders must include payment - allow 30 days for delivery.

BAA membership year runs from 1 May - 30 April. People joining part way through the year will receive appropriate back numbers of Vector.

Software is accepted by the British APL Association on good faith and we do not vouch for or make any claims regarding donated software. The BAA shall not be held responsible for damage caused by the use of library software, and shall not be liable in any way in the event that submitted material proves to be the subject of third party copyright. Your signature above accordingly indemnifies the BAA from all liability.

Index to Advertisers

APL Booklist (Ed Shaw)	22
APL People	31,58,103
Cocking and Drury	16,17,76
Dyadic Systems Ltd	2
Excel	10
HMW Computing	20,100
MicroAPL	32,38

All queries regarding advertising in VECTOR should be made to the advertising editor, Cathy Dargue, at the following address:

Cathy Dargue,
60 Downhall Ley,
Buntingford,
Herts SG9 9TL

Tel: 0707-325161

BRITISH APL ASSOCIATION

Membership Application Form

Please read the membership information in the inside front cover of VECTOR before completing this form. Use photocopies of this form for multiple applications. The membership year runs from 1st May – 30th April.

Name: _____
Department: _____
Organisation: _____
Address Line 1: _____
Address Line 2: _____
Address Line 3: _____
Address Line 4: _____
Post or zip code: _____
Country: _____
Telephone Number: _____

Membership category applied for (tick one):	87/88
Non-voting student membership (UK only)	£ 5
UK private membership	£ 10
Overseas private membership	£ 18 \$ 27
Airmail supplement (not needed for Europe)	£ 8 \$ 12
Corporate membership	£ 85
Corporate membership Overseas	£140 \$210
Sustaining membership	£360

For student applicants:

Name of course: _____
Name and title of supervisor: _____
Signature of supervisor: _____

PAYMENT

Payment should be enclosed with membership applications in the form of a UK sterling cheque or postal order made payable to "The British APL Association". Corporate or sustaining member applicants should contact the Treasurer in advance if an invoice is required. Please enclose a stamped addressed envelope if you require a receipt.

Please send the completed form to the Treasurer at the following address:

The British APL Association
c/o Specialist Groups Liaison Officer
The British Computer Society
13 Mansfield Sreet, LONDON W1M 0BP.

The British APL Association

The British APL Association is a Specialist Group of the British Computer Society. It is administered by a Committee of officers who are elected by a postal ballot of Association members prior to the Annual General Meeting. Working groups are also established in areas such as activity planning and journal production. Offers of assistance and involvement with any Association matters are welcomed and should be addressed in the first instance to the Secretary.

1988/89 Committee

Chairman:	Peter Donnelly 0420-87024	Dyadic Systems Ltd., Park House, The High Street, ALTON, Hants GU34 1EN
Secretary:	Graham Parkhouse 0483-571281 (x2379)	Dept. of Mech. Eng., University of Surrey, GUILDFORD GU2 5XH.
Treasurer:	Phillip Goacher 03727-21282	27 Downs Way, EPSOM, Surrey
Activities:	Jonathan Martin 01 562-5697	(S499) British Airways, PO Box 10, Heathrow Airport, HOUSLOW, Middlesex TW6 2JA
Journal Editor:	Adrian Smith 0904-653071 (x3382)	Brook House, Gilling East, YORK.
Education:	Dr Alan Sykes 0792-295296	Dept of Management Science University College of Swansea, Singleton Park, SWANSEA SA2 8PP
Publicity:	Cathy Dargue 0707-325161 (x2418)	60 Downhall Ley, BUNTINGFORD, Herts SG9 9TL
Technical:	Vacant post	
Recruitment:	Val Lusmore 0225-62601	17, Barton Street, BATH, Avon
Projects:	David Eastwood 01-922 8866	MicroAPL Ltd South Bank Technopark, 90 London Road, LONDON SE1 6LN

Journal Working Group

Jonathan Barman	0374-588835
Anthony Camacho	0727-60130
Cathy Dargue	0707-325161
Val Lusmore	0225-62602
Adrian Smith	0904-653071
David Ziemann	01-436 9481

Activities Working Group

Phil Goacher	03727-21282
Maurice Jordan	01-562 3090
David Parker	01-834 2333

Typeset using MS Word 3.0 on Xerox 3700

Printed in England by Short-Run Press Ltd, Exeter

VECTOR

VECTOR is the quarterly Journal of the British APL Association and is distributed to Association members in the UK and overseas. The British APL Association is a Specialist Group of the British Computer Society. APL stands for "A Programming Language" - an interactive computer language noted for its elegance, conciseness and fast development speed. It is supported on many timesharing bureaus and on most machines, mini and micro computers.

SUSTAINING MEMBERS

The Committee of the British APL Association wish to acknowledge the generous financial support of the following Association Sustaining Members. In many cases these organisations also provide manpower and administrative assistance to the Association at their own cost.

APL People
Queen Anne House
Chalfonts St, BATH, BA1 2NE
Tel:0225-337392

Cooking & Drury Ltd
185 Tottenham Court Rd
LONDON, W1P 9LE
Tel:01-435 9421

HMW Consulting Ltd
Hamilton House
1 Temple Avenue
LONDON, EC4Y 0HA
Tel:01-353 4212
Fax:01-353 3325

Isis Product Ltd
Eagle House
73 Clapham Common Southside
LONDON SW4 9DD
Tel:01-873 0354

Matechmics Systems Ltd
Unit 212, 62 Tipton Road
LONDON SE21 8DE
Tel:01-949-6238

J.P. Sharp Associates
40 Dean Court St
LONDON SW1H 0DX
Tel:01-322-7033

APL Software Tech
14 Rosewood Avenue
Alveston, BRISTOL BS12 2PP
Tel:0454-415737

Dyadic Systems Ltd
Park House, The High Street
ALTON, Hants
Tel:0420-87024

APL Impetus Ltd
Rusper, Sandy Lane
Ivy Hatch, SEVENOAKS
Kent TN15 0PD
Tel:0732-885126

Marcia Software Ltd
Aston Science Park
Love Lane
BIRMINGHAM B7 4BJ
Tel:021-352-5096

MicroAPL Ltd
South Bank Technopark
90 London Road,
LONDON SE1 6LN
Tel:01-922 8886

Peter Cynix Systems
213 Goldhurst Terrace
LONDON NW6 3ER
Tel:01-624-7013
Mobile:0850-377863

